

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO CUỐI KỲ NỘI DUNG BÀI TẬP
THIẾT KẾ LUẬN LÝ SỐ
LỚP: CE118.P13

GIẢNG VIÊN HƯỚNG DẪN: TS. LÂM ĐỨC KHẢI

TP. HỒ CHÍ MINH – NĂM 2024

LỜI CẢM ƠN

Trước tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Lâm Đức Khải, giảng viên bộ môn Thiết kế luận lý số, Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ Thông tin.

Chính nhờ sự hướng dẫn tận tình, chỉ bảo chu đáo và những góp ý quý báu từ thầy trong suốt quá trình học tập và thực hiện bài báo cáo, chúng em mới có thể hoàn thành tốt nhiệm vụ được giao.

Quãng thời gian học tập cùng thầy đã giúp chúng em không chỉ tiếp thu thêm nhiều kiến thức chuyên môn bổ ích mà còn học hỏi được những kinh nghiệm thực tiễn giá trị. Đồng thời, thầy đã giúp chúng em rèn luyện các kỹ năng làm việc nhóm, tư duy logic, và giải quyết vấn đề – những hành trang quan trọng để chúng em phát triển bản thân và vững bước trong tương lai.

Chúng em xin chân thành cảm ơn thầy đã luôn nhiệt huyết, tạo động lực để chúng em nỗ lực hoàn thiện bài báo cáo này.

Môn học Thiết kế luận lý số không chỉ mang lại cho chúng em những kiến thức lý thuyết hữu ích mà còn là cơ hội để vận dụng và thực hành, giúp chúng em hiểu sâu hơn về lĩnh vực mình đang theo đuổi. Với sự cố gắng của cả nhóm cùng sự hỗ trợ từ thầy, bài báo cáo đã được hoàn thành đúng thời hạn.

Tuy nhiên, do kiến thức và kinh nghiệm thực tế của nhóm còn nhiều hạn chế, bài báo cáo không tránh khỏi những thiếu sót. Chúng em kính mong thầy xem xét và đóng góp ý kiến để bài làm của nhóm được hoàn thiện hơn.

Một lần nữa, chúng em xin gửi lời tri ân chân thành đến thầy và kính chúc thầy dồi dào sức khỏe, thành công trong sự nghiệp giảng dạy.

TP. Hồ Chí Minh, thứ ba ngày 3 tháng 12 năm 2024

Nhóm sinh viên thực hiện:

Phạm Anh Nguyên

Lê Minh Hùng

Văn Công Gia Luật

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting or typing. There are no margins, text, or other markings on the page.

THÀNH VIÊN NHÓM THỰC HIỆN

STT	MSSV	Họ và tên	Tỷ lệ đóng góp	Ghi chú
1	22520980	Phạm Anh Nguyên	35%	
2	22520506	Lê Minh Hùng	35%	
3	22520830	Văn Công Gia Luật	30%	

CÁC NỘI DUNG BÀI TẬP ĐÃ THỰC HIỆN

STT	TÊN BÀI
1	Textbook exercises
2	4-bit up/down binary counter with parallel load
3	BCD counter
4	Shift register with parallel load (SRwPL)
5	Register-file with 1 write port and 2 read ports
6	RAM 16x8
7	Stack 16x8
8	Queue 16x8
9	One accumulator
10	One's-Count
11	Square-root approximation

12	Register assignment generated by the graph-partitioning algorithm
13	Datapath connectivity
14	SRA Datapath generated by prioritized partitioning
15	Connection merging in SRA datapath
16	Datapath with pipelined functional unit
17	Datapath pipelining
18	Timing diagram for datapath pipeline with pipelined functional units

MỤC LỤC

LỜI CẢM ƠN.....	2
NHẬN XÉT CỦA GIẢNG VIÊN.....	3
DANH SÁCH HÌNH VẼ.....	12
Bài 1. Textbook exercises.....	17
1.1 (Sequential synthesis) Design a parity checker.....	17
1.2 (Sequential synthesis) Design a recognizer.....	19
Bài 2: 4-bit up/down binary counter with parallel load.....	22
2.1. Tổng quan thiết kế:.....	22
2.2. Tín hiệu điều khiển:.....	22
2.3. Thiết kế:.....	23
2.3.1. Half Adder with Storage (HAS):.....	23
2.3.2. Thiết kế hoàn thiện:.....	24
2.4. Waveform:.....	25
Bài 3: BCD Counter.....	26
3.1. Tổng quan thiết kế:.....	26
3.1.1. Mục Tiêu:.....	26
3.1.2. Nguyên tắc hoạt động:.....	26
3.2. Thiết kế:.....	26
3.3. Waveform:.....	27
Bài 4. Shift register with parallel load (SRwPL).....	28
4.1. Tổng quan thiết kế:.....	28
4.2. Thiết kế:.....	28
4.2.1. Các khối D-flipflop:.....	28
4.2.2. Khối chọn tín hiệu input:.....	29
4.2.3. Thiết kế hoàn thiện:.....	29
4.3. Waveform:.....	30
Bài 5. Register-file with 1 write port and 2 read ports.....	31
5.1. Tổng quan thiết kế:.....	31
5.2. Thiết kế:.....	31
5.2.1. Decoder 2 to 4:.....	31
5.2.2. Register file cell:.....	33
5.2.3. Hàng dữ liệu:.....	33
5.2.4. Thiết kế hoàn thiện:.....	33
5.3. Waveform:.....	34

Bài 6: Random Access Memory (Ram 16x8).....	36
6.1. Tổng quan thiết kế:.....	36
6.2. Thiết kế:.....	36
6.2.1. Memory Cell:.....	36
6.2.2. Ram 4x4:.....	37
6.2.3. Ram 16x8:.....	39
6.3. Waveform:.....	40
Bài 7. Stack 16x8.....	41
7.1. Tổng quan thiết kế:.....	41
7.2. Thiết kế:.....	41
7.2.1. 4-bit up/down binary counter:.....	41
7.2.2. Khởi tín hiệu báo trạng thái:.....	42
7.2.3. Thiết kế hoàn thiện:.....	42
7.3. Waveform:.....	43
Bài 8. Queue 16x8.....	44
8.1. Tổng quan thiết kế:.....	44
8.2. Thiết kế:.....	46
8.2.1. Bộ so sánh:.....	46
8.2.2. Thiết kế hoàn thiện:.....	46
8.3. Waveform:.....	47
Bài 9. One accumulator.....	48
9.1. Tổng quan thiết kế:.....	48
9.2. Thiết kế Datapath:.....	48
9.3. Thiết kế Controller:.....	49
9.4. Waveform:.....	51
Bài 10. One's-Count.....	52
10.1. Tổng quan thiết kế:.....	52
10.2. Thiết kế Datapath:.....	52
10.3. Thiết kế Controller:.....	53
10.3.1. Sơ đồ trạng thái:.....	53
10.3.2. Tín hiệu Control:.....	53
10.3.3. Nextstate:.....	54
10.3.4. Currentstate:.....	55
10.3.5. Output Block:.....	55
10.4. Thiết kế tổng:.....	56
10.5. Waveform:.....	56

Bài 11. Square-root approximation.....	57
11.1. Tổng quan thiết kế:.....	57
11.2. Thiết kế Datapath:.....	57
11.2.1. Khối Full Adder:.....	57
11.2.2. Khối Cộng 2 số 8 bit:.....	58
11.2.3. Khối Trừ 2 số 8 bit:.....	59
11.2.4. Khối Cộng/Trừ 2 số 8 bit:.....	60
11.2.5. Khối trị tuyệt đối:.....	60
11.2.6. Khối Min Max:.....	61
11.2.7. Thiết kế tổng:.....	62
11.3. Waveform:.....	62
Bài 12. Register assignment generated by the graph-partitioning algorithm.....	63
12.1. Tổng quan thiết kế:.....	63
12.2. Thiết kế Datapath:.....	65
12.2.1. Khối Mux 4 to 1:.....	65
12.2.2. Khối Mux 8 to 1:.....	65
12.2.3. Khối Register 1x8:.....	66
12.2.4. Hoàn thiện Datapath:.....	66
12.3. Thiết kế Controller:.....	67
12.3.1. Tín hiệu Control.....	67
12.3.2. Khối Next State.....	69
12.3.3. Khối Current State:.....	69
12.3.4. Khối tín hiệu Output cho thiết kế Graph partitioning:.....	69
12.3.5. Hoàn thiện khối Controller:.....	70
12.4. Thiết kế Hoàn thiện:.....	70
12.5. Waveform:.....	70
Bài 13. Datapath connectivity.....	71
13.1. Tổng quan thiết kế:.....	71
13.2. Thiết kế Datapath:.....	72
13.2.1. Khối AU1: Add/Abs/Sub/Max.....	72
13.2.2. Khối AU2: Abs/Min.....	73
13.2.3. Khối Register 8x8:.....	74
13.2.4. Khối Register 4x8: từ Register 4x4.....	74
13.2.5. Hoàn thiện Datapath:.....	75
13.3. Thiết kế Controller:.....	75

13.3.1. Bảng trạng thái và các tín hiệu điều khiển:.....	75
13.3.2. Khối tín hiệu Output cho thiết kế Datapath connectivity:.....	77
13.3.3. Hoàn thiện khối Controller:.....	77
13.4. Thiết kế Hoàn thiện:.....	78
13.5. Waveform:.....	78
Bài 14. SRA Datapath generated by prioritized partitioning.....	79
14.1. Tổng quan thiết kế:.....	79
14.2. Thiết kế Datapath:.....	81
14.2.1. Khối AU1: Abs/Max.....	81
14.2.2. Khối AU2: Add/Abs/Sub/Min.....	81
14.2.3. Hoàn thiện Datapath:.....	82
14.3. Thiết kế Controller:.....	82
14.3.1. Thiết kế trạng thái kế tiếp:.....	84
14.3.2. Thiết kế trạng thái hiện tại:.....	84
14.3.3. Khối tín hiệu Output cho thiết kế SRA Datapath:.....	85
14.3.4. Hoàn thiện Controller:.....	85
14.4. Thiết kế Hoàn thiện:.....	86
14.5. Waveform:.....	86
Bài 15. Register merging.....	87
15.1. Tổng quan thiết kế:.....	87
15.2. Thiết kế Datapath:.....	89
15.2.1. Khối Abs_Max:.....	89
15.2.2. Khối Abs_Min_Add_Sub:.....	89
15.2.3. Hoàn thiện Datapath:.....	90
15.3. Thiết kế Controller:.....	91
15.3.1. Bảng trạng thái:.....	92
15.3.2. Tín hiệu Control:.....	92
15.3.3. Khối NextState:.....	93
15.3.4. Khối CurrentState:.....	94
15.3.5. Khối Output:.....	94
15.3.6. Hoàn thiện Controller:.....	94
15.4. Thiết kế hoàn thiện:.....	95
15.5. Waveform:.....	95
Bài 16. Datapath with pipelined functional unit.....	96
16.1. Tổng quan mạch.....	96
16.2. Thiết kế Datapath:.....	97

16.2.1. AU:.....	98
16.2.2. Control Adder8bit.....	99
16.2.3. Adder8bit.....	99
16.3. Thiết kế Controller:.....	100
16.3.1. Các khối điều khiển MUXS1 (Điều khiển MUX 1).....	100
16.3.2. Bảng trạng thái Control:.....	101
16.3.3. Khối NextState.....	101
16.3.4. Khối CurrentState.....	102
16.3.5. Hoàn thiện Controller:.....	102
16.4. Thiết kế Hoàn thiện.....	103
16.5. Waveform.....	103
Bài 17. Datapath pipelining.....	104
17.1. Tổng quan thiết kế:.....	104
17.2. Thiết kế Datapath:.....	106
17.2.1. Khối AU1 để tính Abs/Min/Max:.....	106
17.2.2. Khối AU2 để tính Abs/Sub/Max:.....	106
17.2.3. Hoàn thiện Datapath:.....	107
17.3. Thiết kế Controller:.....	108
17.3.1. Bảng trạng thái và các tín hiệu điều khiển:.....	108
17.3.2. Hoàn thiện khối Controller:.....	110
17.3.3. Hoàn thiện Controller:.....	111
17.4. Thiết kế Hoàn thiện:.....	112
17.5. Waveform:.....	113
Bài 18. Datapath pipeline with pipelined functional units.....	114
18.1. Tổng quan thiết kế:.....	114
18.2. Thiết kế Datapath:.....	115
18.2.1. Khối AU1:.....	115
18.2.2. Khối AU2:.....	116
18.2.3. Hoàn thiện Datapath:.....	117
18.3. Thiết kế Controller:.....	119
18.3.1. Tín hiệu control:.....	119
18.3.2. Khối NextState:.....	120
18.3.3. Khối Current State:.....	121
18.3.4. Khối Output:.....	121
18.3.5. Khối tín hiệu hoàn thành:.....	122
18.3.6. Hoàn thiện Controller:.....	122

18.4. Thiết kế Hoàn thiện:	123
18.5. Waveform:	123

DANH SÁCH HÌNH VẼ

Hình 1.1: Sơ đồ chuyển trạng thái.....	17
Hình 1.2: Thiết kế khối NEXTSTATE.....	18
Hình 1.3: Thiết kế khối CURRENTSTATE.....	18
Hình 1.4: Thiết kế hoàn thiện.....	18
Hình 1.5: Waveform 1 text book exercise.....	19
Hình 1.6: Sơ đồ chuyển trạng thái.....	19
Hình 1.7: Thiết kế khối NEXTSTATE.....	20
Hình 1.8: Thiết kế khối CURRENTSTATE.....	21
Hình 1.9: Thiết kế khối OUTPUT.....	21
Hình 1.10: Waveform 2 text book exercise.....	21
Hình 2.1: Thiết kế khối HAS.....	23
Hình 2.2: Thiết kế mạch 4-bit up/down binary counter with parallel load.....	24
Hình 2.3: Waveform.....	25
Hình 3.1: Thiết kế mạch 4-bit up/down binary counter with parallel load.....	27
Hình 3.2: Waveform.....	27
Hình 4.1 Tín hiệu điều khiển.....	28
Hình 4.2: Thiết kế.....	29
Hình 4.3: Thiết kế.....	29
Hình 4.4: Thiết kế.....	30
Hình 4.5: Waveform.....	30
Hình 5.1: Khối Decoder 2 to 4.....	32
Hình 5.2: Khối RFC.....	33
Hình 5.3: Thiết kế 1 hàng trong Register.....	33
Hình 5.4. Thiết kế tổng của Register 4x4.....	34
Hình 5.5: Waveform của register 4x4.....	34
Hình 6.1: Thiết kế khối MC.....	36
Hình 6.2: Thiết kế Ram 4x4.....	38
Hình 6.3: Thiết kế Ram 16x8.....	39
Hình 6.4: Waveform Ram 16x8.....	40
Hình 7.1: Tín hiệu điều khiển chức năng khối Stack.....	41
Hình 7.2: Tín hiệu control.....	41
Hình 7.3: Thiết kế 4-bit up/down binary counter.....	42
Hình 7.4: Thiết kế bộ check trạng thái Stack.....	42
Hình 7.5: Thiết kế Stack.....	43

Hình 7.6: Waveform1 Stack.....	43
Hình 7.7: Waveform2 Stack.....	44
Hình 8.1: Tín hiệu điều khiển Queue.....	45
Hình 8.2: Bộ so sánh 2 số.....	46
Hình 8.3: Thiết kế Queue.....	47
Hình 8.4: Waveform Queue.....	47
Hình 9.1: Thiết kế bộ cộng 2 số 7 bit.....	48
Hình 9.2: Thiết kế bộ Datapath tính tổng từ 1 đến input.....	48
Hình 9.3: Sơ đồ trạng thái One accumulator.....	49
Hình 9.4: Bảng trạng thái và output.....	49
Hình 9.5: Khối Next State.....	50
Hình 9.6: Khối Current State.....	50
Hình 9.7: Khối Output.....	50
Hình 9.8: Thiết kế khối Control.....	50
Hình 9.9: Thiết kế tổng.....	51
Hình 9.10: Waveform one Accumulator.....	51
Hình 10.1: Thiết kế mạch One's-count.....	52
Hình 10.2: Sơ đồ trạng thái.....	53
Hình 10.3: Bảng trạng thái kế tiếp và output.....	53
Hình 10.4: Thiết kế khối Control.....	54
Hình 10.5: Khối Next State.....	54
Hình 10.6: Khối Current State.....	55
Hình 10.7: Khối Output.....	56
Hình 10.8: Thiết kế tổng.....	56
Hình 10.9: Waveform One's-count.....	56
Hình 11.1: ASM Chart.....	57
Hình 11.2: Thiết kế bộ Full Adder.....	58
Hình 11.3: Thiết kế mạch cộng 2 số 8 bit.....	59
Hình 11.4: Thiết kế mạch trừ 2 số 8 bit.....	60
Hình 11.5: Thiết kế khối AddorSub.....	60
Hình 11.6: Thiết kế khối trị tuyệt đối.....	61
Hình 11.7: Khối MinMax.....	62
Hình 11.8: Thiết kế tổng.....	62
Hình 11.9: Waveform.....	62
Hình 12.1: ASM Chart.....	63
Hình 12.2: Khối Mux 4 to 1.....	65

Hình 12.3: Khối Mux 8 to 1.....	65
Hình 12.4: Khối Register 1x8.....	66
Hình 12.5: Thiết kế Datapath.....	67
Hình 12.6: Bảng trạng thái tiếp theo và output.....	67
Hình 12.7: Sơ đồ chuyển trạng thái.....	68
Hình 12.8: Khối Next State.....	69
Hình 12.9: Khối Current State.....	69
Hình 12.10: Khối Output.....	70
Hình 12.11: Khối Control.....	70
Hình 12.11: Thiết kế tổng.....	70
Hình 12.12: Waveform.....	70
Hình 13.1. Merging alternative (simple merging operator).....	71
Hình 13.2: Thiết kế khối AU1.....	73
Hình 13.3: Thiết kế khối AU2.....	74
Hình 13.4: Thiết kế Register 8x8.....	74
Hình 13.5: Thiết kế Register 4x8.....	75
Hình 13.6: Thiết kế Datapath.....	75
Hình 13.7: Khối Output.....	77
Hình 13.8: Khối Control.....	78
Hình 13.9: Thiết kế tổng.....	78
Hình 13.10: Waveform.....	78
Hình 14.1: ASM Chart.....	79
Hình 14.2: Giải thuật Graph Partitioning thiết kế Functional Unit Sharing.....	80
Hình 14.3: Thiết kế khối AU1.....	81
Hình 14.4: Thiết kế khối AU2.....	82
Hình 14.5: Thiết kế khối Datapath.....	82
Hình 14.6: Thiết kế khối NEXTSTATE.....	84
Hình 14.7: Thiết kế khối CURRENTSTATE.....	84
Hình 14.8: Thiết kế khối Output cho SRA Datapath.....	85
Hình 14.9: Thiết kế khối Controller.....	85
Hình 14.10: Thiết kế hoàn chỉnh mạch SRA Datapath.....	86
Hình 14.11: Waveform SRA Datapath.....	86
Hình 15.1: ASM Chart.....	87
Hình 15.2: Khối Abs_Max.....	89
Hình 15.3: Khối Abs_Min_Add_Sub.....	90
Hình 15.4: Datapath.....	90

Hình 15.5: Bảng trạng thái kế tiếp.....	92
Hình 15.6: bảng tín hiệu Control ứng với mỗi trạng thái.....	92
Hình 15.7: Sơ đồ chuyển trạng thái.....	93
Hình 15.8: Khối Next State.....	93
Hình 15.9: Khối Current State.....	94
Hình 15.10: Khối Output.....	94
Hình 15.11: Khối Control.....	94
Hình 15.12: Mạch tổng.....	95
Hình 16.1: ASM Chart.....	96
Hình 16.2: Timing diagram.....	97
Hình 16.3: Thiết kế khối Datapath.....	98
Hình 16.4: Thiết kế khối AU.....	99
Hình 16.5: Thiết kế khối Control Adder8bit.....	99
Hình 16.6: Thiết kế khối Adder8bit.....	99
Hình 16.7: Thiết kế khối S1.....	100
Hình 16.8: Thiết kế khối S2.....	100
Hình 16.9: Bảng trạng thái kế tiếp và output.....	101
Hình 16.10: Khối NextState.....	101
Hình 16.11: Khối Current State.....	102
Hình 16.12: Khối Control.....	102
Hình 16.13: Mạch tổng.....	103
Hình 16.14: Waveform Datapath with pipelined functional.....	103
Hình 17.1. Biểu đồ ASM cho thiết kế Datapath pipelining.....	104
Hình 17.2 Giản đồ thời gian dựa trên biểu đồ ASM.....	105
Hình 17.3: Khối AU1 (Abs/Min/Max).....	106
Hình 17.4. Thiết kế khối AU2 để tính Abs/Sub/Max.....	107
Hình 17.5. Thiết kế Datapath hoàn chỉnh cho thiết kế Datapath pipelining... ..	107
Hình 17.6. Sơ đồ chuyển trạng thái cho thiết kế Datapath pipelining.....	109
Hình 17.7. Khối tín hiệu Output cho thiết kế Datapath Pipeline.....	110
Hình 17.8. Mạch trạng thái kế tiếp.....	111
Hình 17.9. Mạch trạng thái hiện tại.....	111
Hình 17.10. Mạch kiểm soát đầu ra.....	111
Hình 17.11. Controller hoàn thiện cho thiết kế Datapath pipelining.....	112
Hình 17.12. Thiết kế hoàn thiện của Datapath pipelining.....	112
Hình 17.13. Waveform của thiết kế Datapath pipelining.....	113
Hình 18.1. Timing diagram for datapath pipeline with pipelined functional..	115

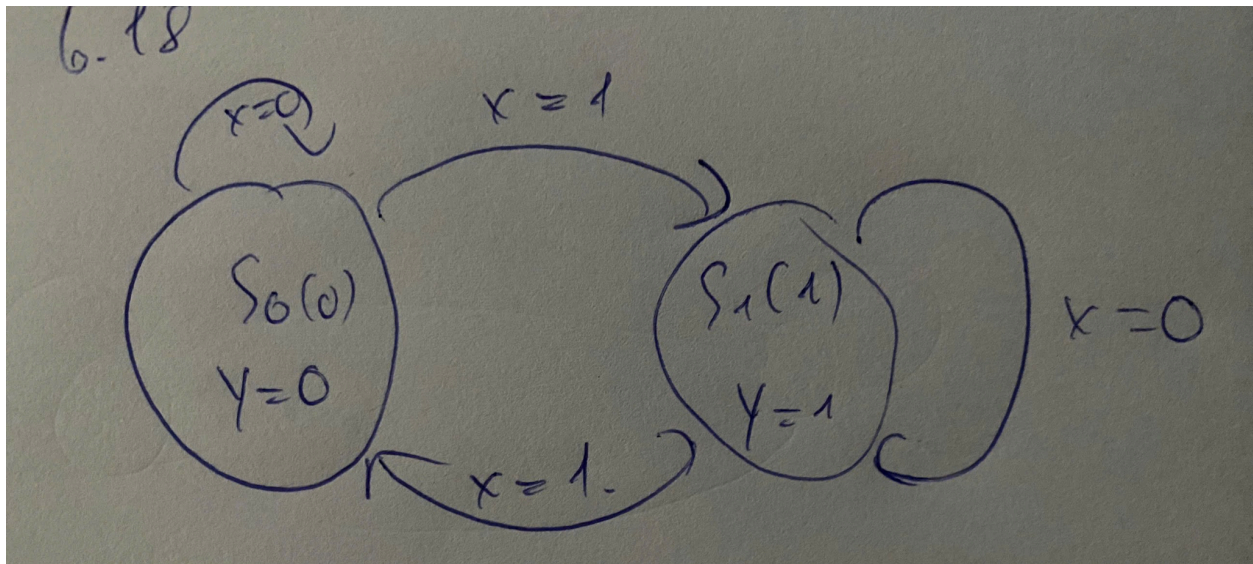
Hình 18.2. Khối AU1 (Abs/Min/Max).....	116
Hình 18.3. Khối AU2 (Max/Add/Sub).....	116
Hình 18.3. Datapath.....	117
Hình 18.4. Tín hiệu control ứng với từng trạng thái.....	119
Hình 18.5: Sơ đồ chuyển trạng thái.....	120
Hình 18.6. Khối Next State.....	121
Hình 18.8. Khối Output.....	122
Hình 18.9. Khối tín hiệu hoàn thành.....	122
Hình 18.10. Khối Control.....	123
Hình 18.11. Thiết kế tổng.....	123
Hình 18.12. Waveform.....	123

Bài 1. Textbook exercises

1.1 (Sequential synthesis) Design a parity checker

Design a parity checker that counts the number of 1's in the input stream. This checker asserts its output Y if it has received an odd number of 1's on the input X. An asynchronous Reset signal returns the parity checker into its initial state. As storage elements use only D flip-flops.

Máy trạng thái:



Hình 1.1: Sơ đồ chuyển trạng thái.

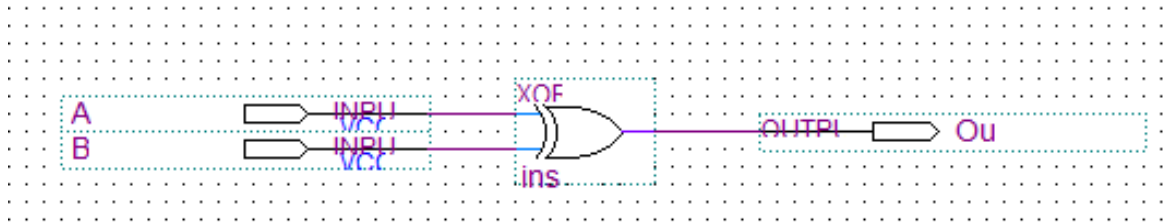
Bảng trạng thái:

Bảng 1.1. Bảng trạng thái kế tiếp cho khối NEXTSTATE

PRESENT STATE Q0	NEXT STATE Q0(next)		OUTPUTS Y
	Cnt = 0	Cnt = 1	
0	0	1	0
1	1	0	1

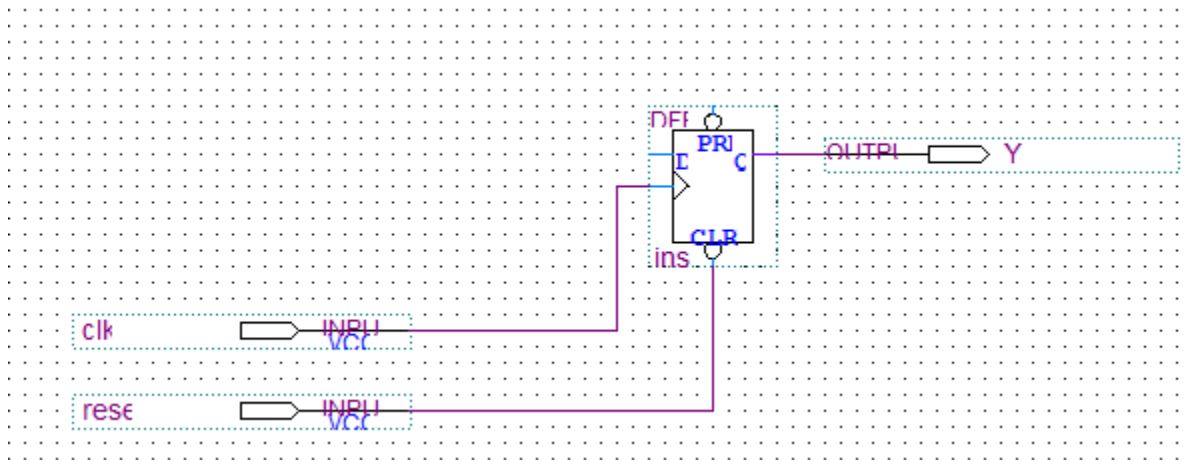
Next State Equations: $D = X \oplus Q$

Khởi trạng thái kế tiếp:



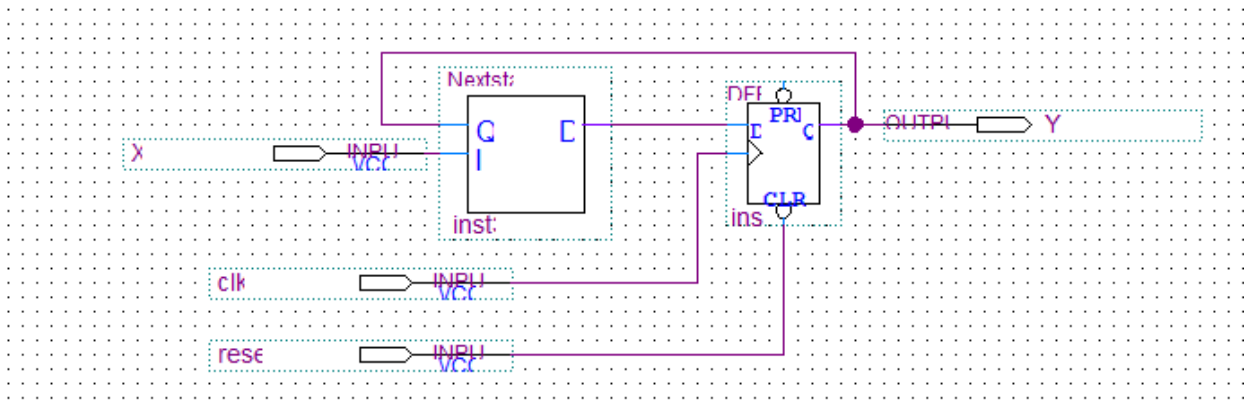
Hình 1.2: Thiết kế khối NEXTSTATE.

Khởi trạng thái hiện tại:



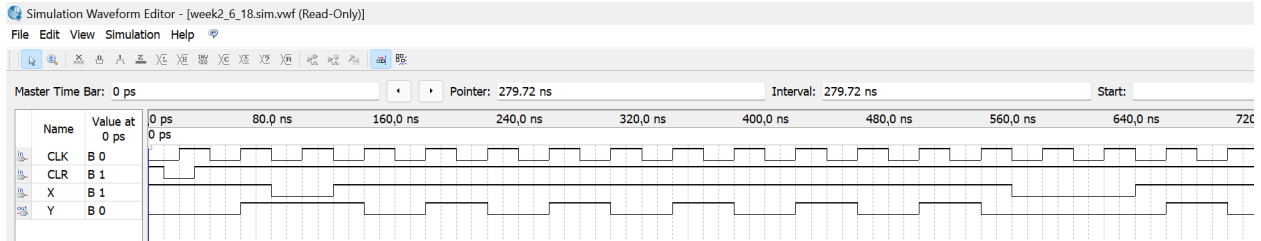
Hình 1.3: Thiết kế khối CURRENTSTATE.

Thiết kế hoàn thiện:



Hình 1.4: Thiết kế hoàn thiện.

Waveform:

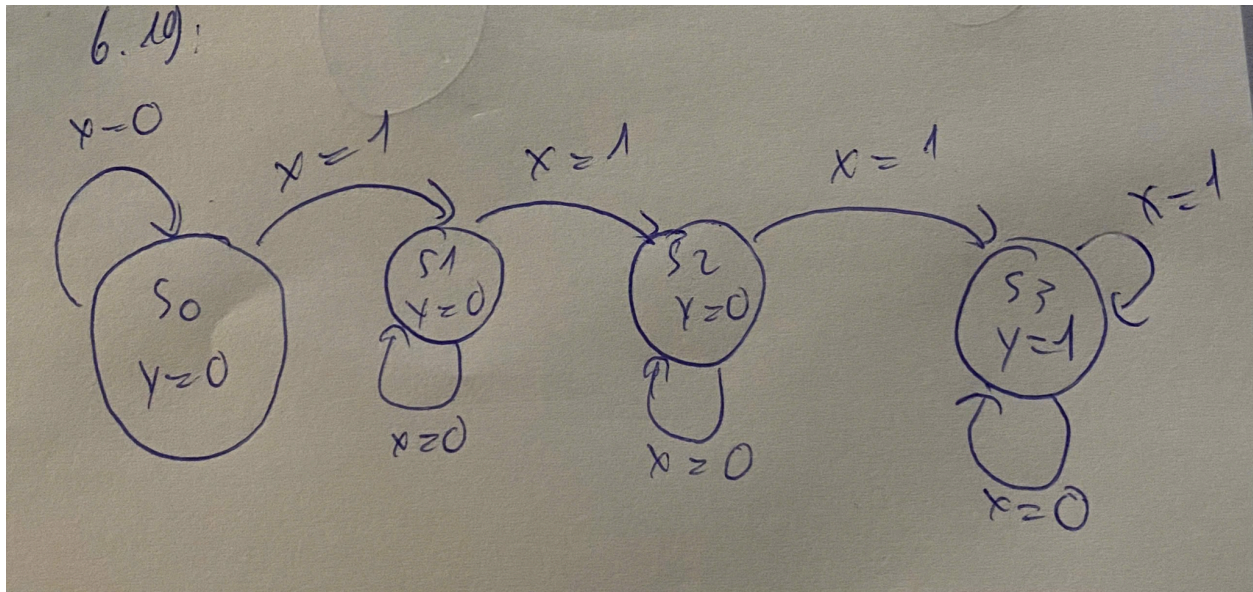


Hình 1.5: Waveform 1 text book exercise

1.2 (Sequential synthesis) Design a recognizer

Design a recognizer that recognizes an input sequence that has at least three 1's. The recognizer has a single input X and a single output Y, in addition to an asynchronous Reset signal. The recognizer sets the output Y to 1 if the input signal X equals 1 at least three clock cycles after Reset was disasserted.

Máy trạng thái:



Hình 1.6: Sơ đồ chuyển trạng thái.

Bảng trạng thái:

Bảng 1.2. Bảng trạng thái kế tiếp cho khối NEXTSTATE

PRESENT STATE Q1Q0	NEXT STATE Q1(next)Q0(next)		OUTPUTS Y
	Cnt = 0	Cnt = 1	
00	00	01	0
01	01	10	0
10	10	11	0
11	11	11	1

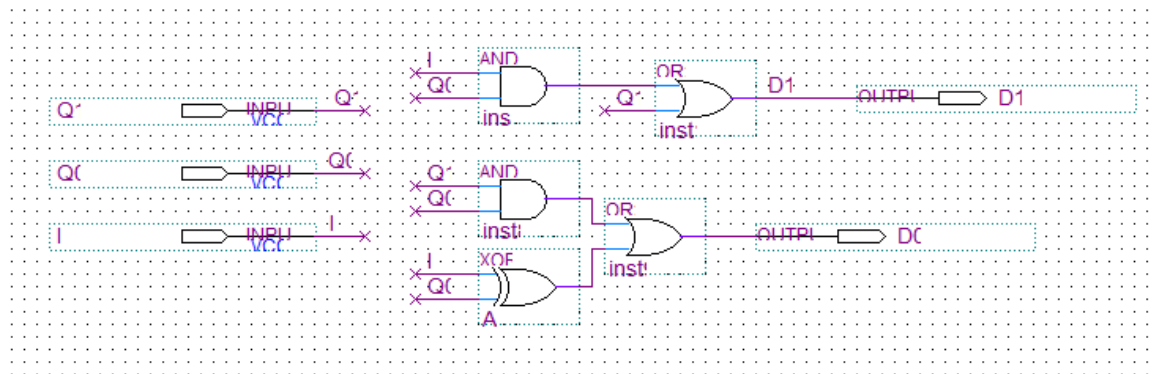
Next state equation:

$$Q1(next) = Q1 + CntQ0$$

$$Q0(next) = Cnt'Q0 + CntQ0' + Q1Q0$$

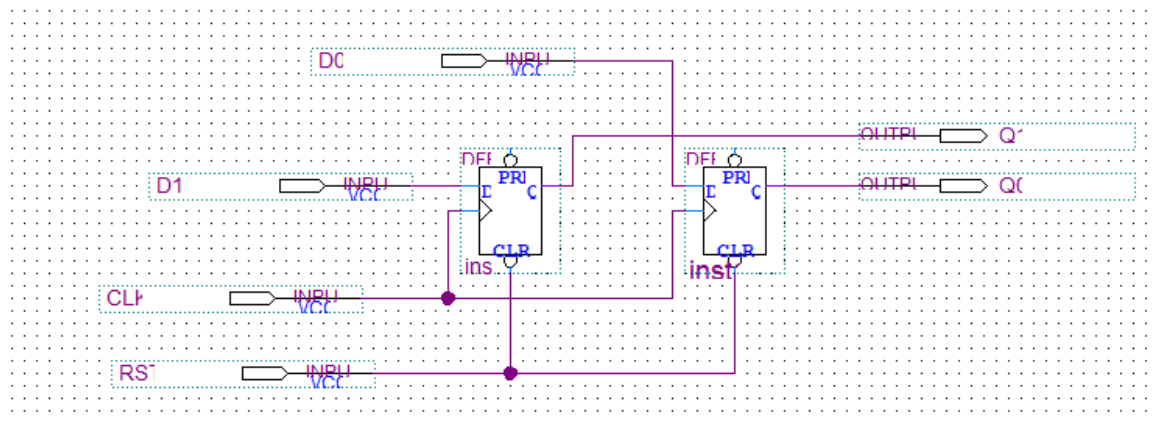
$$Y = Q1Q0$$

Khối trạng thái kế tiếp:



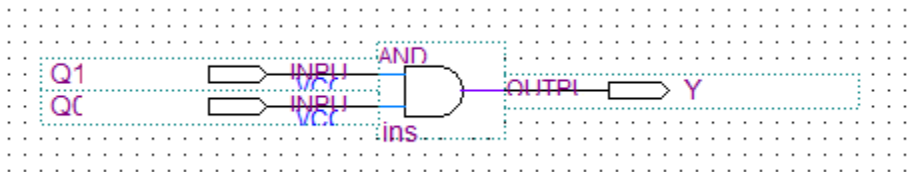
Hình 1.7: Thiết kế khối NEXTSTATE.

Khởi trạng thái hiện tại:



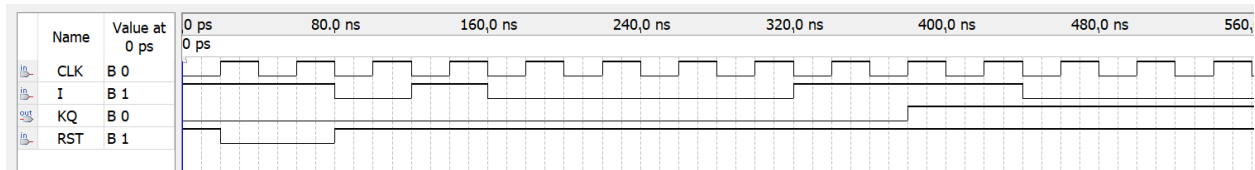
Hình 1.8: Thiết kế khối CURRENTSTATE.

Khối Output:



Hình 1.9: Thiết kế khối OUTPUT

Waveform:



Hình 1.10: Waveform 2 text book exercise

Bài 2: 4-bit up/down binary counter with parallel load

2.1. Tổng quan thiết kế:

Bộ đếm nhị phân 4-bit lên/xuống với khả năng tải song song (4-bit Up/Down Binary Counter with Parallel Load) là một mạch số được sử dụng để đếm tăng hoặc giảm giá trị nhị phân 4-bit, đồng thời cho phép nạp giá trị đầu vào từ bên ngoài vào bộ đếm.

- Đầu vào:
 - + D: tín hiệu điều khiển Up/Down.
 - + E: tín hiệu cho phép khởi hoạt động hay không.
 - + Load: tín hiệu cho phép việc ghi input vào hay không
- Đầu ra:
 - + Trạng thái kế tiếp của khối.

2.2. Tín hiệu điều khiển:

Bảng 2.1. Bảng tín hiệu điều khiển

Load	E	D	Operations
0	0	X	No change
0	1	0	Count up
0	1	1	Count down
1	X	X	Load the input

2.3. Thiết kế:

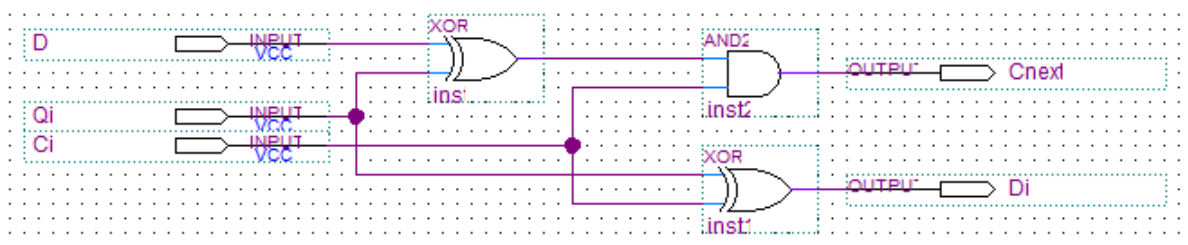
2.3.1. Half Adder with Storage (HAS):

- HAS truth table

Bảng 2.2. Bảng sự thật cho khối HALFADDER

D	Qi	Ci	Ci+1	Di
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	0

- Thiết kế:

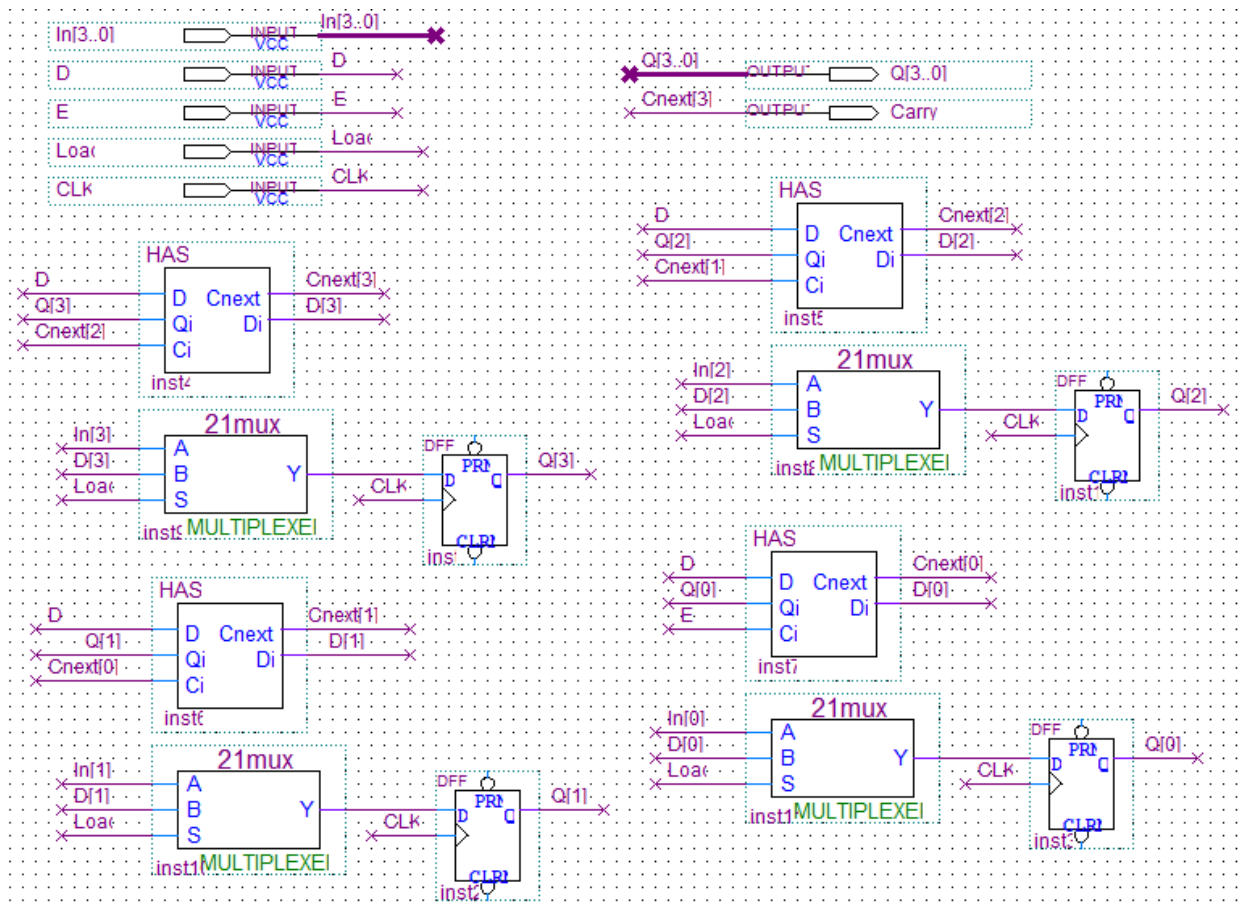


Hình 2.1: Thiết kế khối HAS

- Chức năng:
 - + Thực hiện phép cộng bit đơn với đầu vào từ các tín hiệu Ci, D, và Qi, đồng thời tạo ra các đầu ra là:

- **Ci+1:** Bit nhớ (carry-out) được tạo ra từ phép cộng.
 - **Di:** Kết quả của phép cộng lưu trữ hoặc logic kết hợp.
- + HAS vừa thực hiện chức năng **tính toán cơ bản (phép cộng)**, vừa **lưu trữ trạng thái** để phục vụ các phép toán tuần tự

2.3.2. Thiết kế hoàn thiện:



Hình 2.2: Thiết kế mạch 4-bit up/down binary counter with parallel load

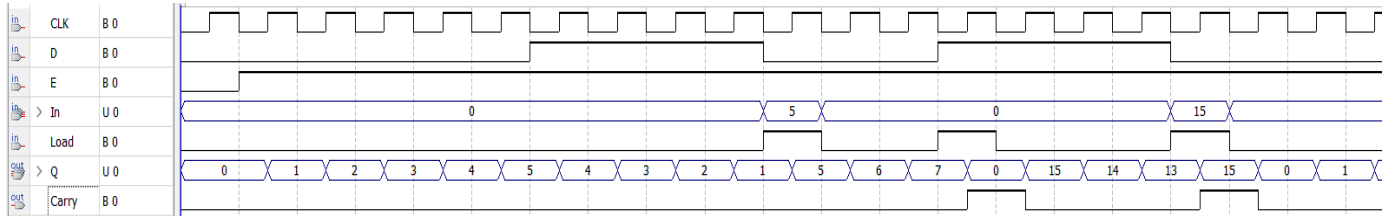
Nguyên tắc hoạt động:

- + Sử dụng mux 2_1 để chọn giữa chức năng count down/up (Khối HAS) hoặc Load input.
- + Ban đầu load input dựa vào việc chọn chân In[x] ở các mux21, sau đó thực hiện đếm lên/xuống và lưu kết quả vào flipflop sau đó tiếp tục sử dụng kết quả trước đó được lưu trong flipflop để tiếp tục đếm
- + Tín hiệu Load:

+ Tín hiệu D:

+ Tín hiệu E:

2.4. Waveform:



Hình 2.3: Waveform

Giải thích Waveform:

- Từ chu kỳ thứ 2 bật E = 1 (Enable):
- Với tín hiệu **D = 0**, mạch bắt đầu đếm lên từ 0 và đạt giá trị **5**.
- Sau đó, tín hiệu **D** chuyển sang **1**, khiến mạch đổi hướng và bắt đầu đếm xuống (5 -> 1).
- Khi mạch đếm xuống đến giá trị **1**, tín hiệu **Load = 1** xuất hiện, và mạch tải giá trị **Input = 5** để tiếp tục đếm lên tới 7 với **D = 0**.
- Mạch đếm lên đến **7**, gặp tín hiệu **Load = 1**, và lần này tải giá trị **Input = 0**. Mạch chuyển sang đếm xuống từ **0** đến **13** với **D = 1**.
- Khi đạt đến **13**, tín hiệu **Load = 1** xuất hiện, mạch nạp giá trị **Input = 15** và bắt đầu lại chu kỳ đếm lên với **D = 0**.

Bài 3: BCD Counter

3.1. Tổng quan thiết kế:

3.1.1. Mục Tiêu:

Thiết kế một bộ đếm BCD (Binary-Coded Decimal) đếm lên hoặc đếm xuống trong dải giá trị từ 0 đến 9 (theo hệ thập phân). Bộ đếm sử dụng khối 4-bit Up/Down Binary Counter với khả năng nạp song song (parallel load).

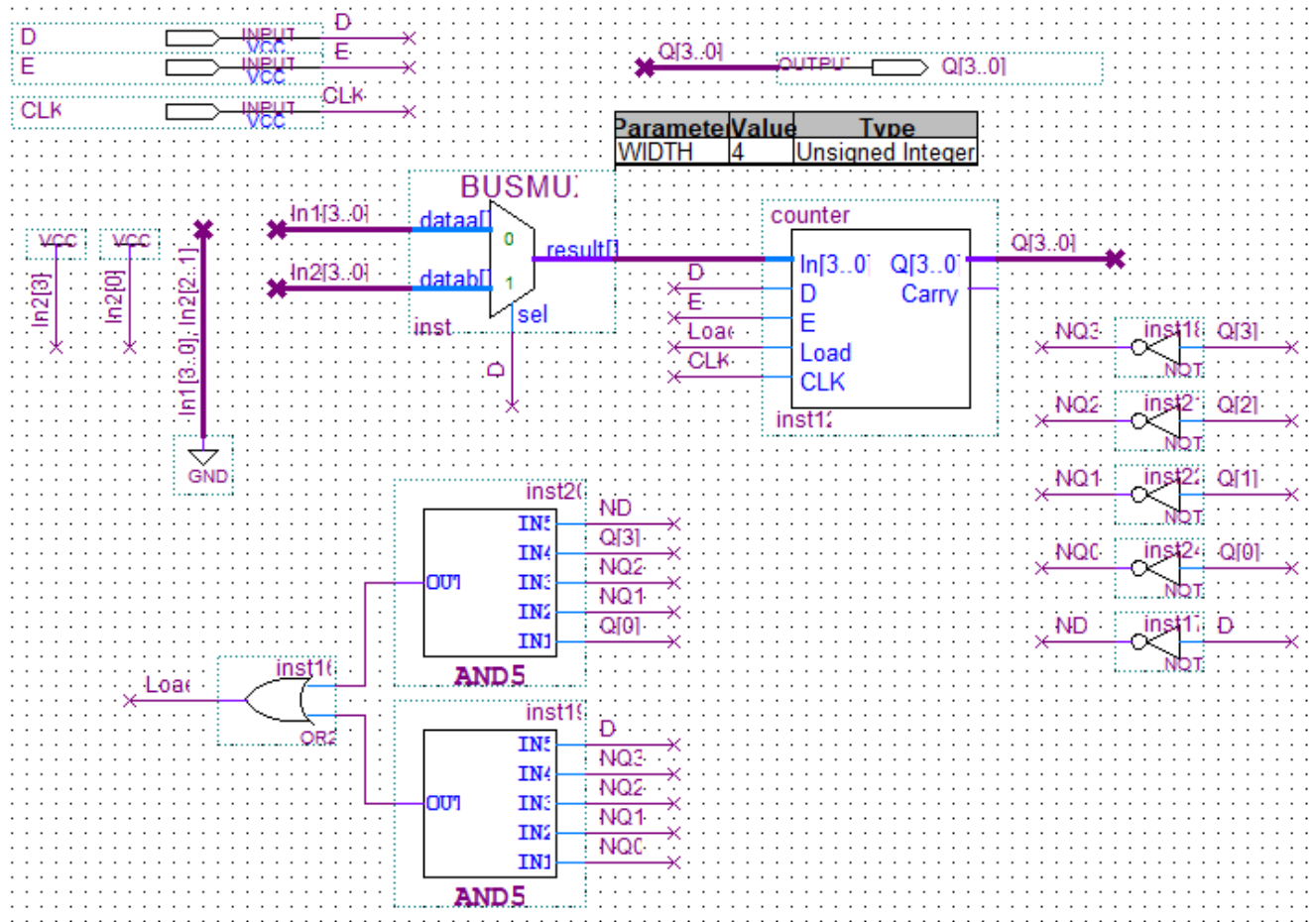
- Đầu vào:
 - + D: tín hiệu điều khiển việc đếm lên hay xuống.
 - + E: tín hiệu cho phép khối hoạt động được không.
 - + Xung clock.
- Đầu ra:
 - + Trạng thái kế tiếp.

3.1.2. Nguyên tắc hoạt động:

- **Bộ đếm tăng (Up Counter)** nạp giá trị 0 khi nội dung bộ đếm đạt 9 (1001).
- **Bộ đếm tăng/giảm (Up/Down Counter)** nạp giá trị 0 khi nội dung bộ đếm đạt 9 (1001) và bit điều hướng D=0.
- **Bộ đếm tăng/giảm (Up/Down Counter)** nạp giá trị 9 khi nội dung bộ đếm đạt 0 (0000) và bit điều hướng D=1.

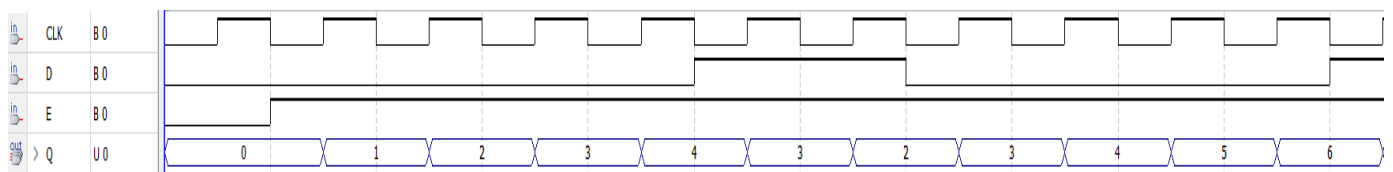
3.2. Thiết kế:

Sử dụng 4-bit up/down binary counter with parallel load.



Hình 3.1: Thiết kế mạch 4-bit up/down binary counter with parallel load.

3.3. Waveform:



Hình 3.2: Waveform

Giải thích:

- Tín hiệu E bật lên từ chu kỳ thứ 2. Mạch hoạt động từ chu kỳ thứ 2.
- Khi D = 0, mạch đếm lên từ 0 đến 4.
- Khi D = 1, mạch đếm xuống từ 4 đến 2. Sau đó, D = 0 mạch đếm lên từ 2 tới 6.

Bài 4. Shift register with parallel load (SRwPL)

4.1. Tổng quan thiết kế:

Input:

- + Dữ liệu đầu vào ($I[3:0]$, I_R , I_L).
- + $S[1:0]$: tín hiệu điều khiển chức năng.

Output:

- + Trạng thái kế tiếp.

Các tín hiệu điều khiển:

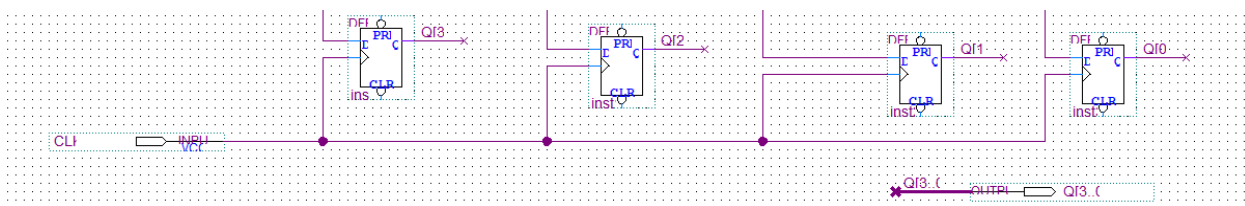
Present state		Operation	Next state			
S_1	S_0		Q_3	Q_2	Q_1	Q_0
0	0	No change	Q_3	Q_2	Q_1	Q_0
0	1	Load input	I_3	I_2	I_1	I_0
1	0	Shift left	Q_2	Q_1	Q_0	I_R
1	1	Shift right	I_L	Q_3	Q_2	Q_1

Hình 4.1 Tín hiệu điều khiển.

4.2. Thiết kế:

4.2.1. Các khối D-flipflop:

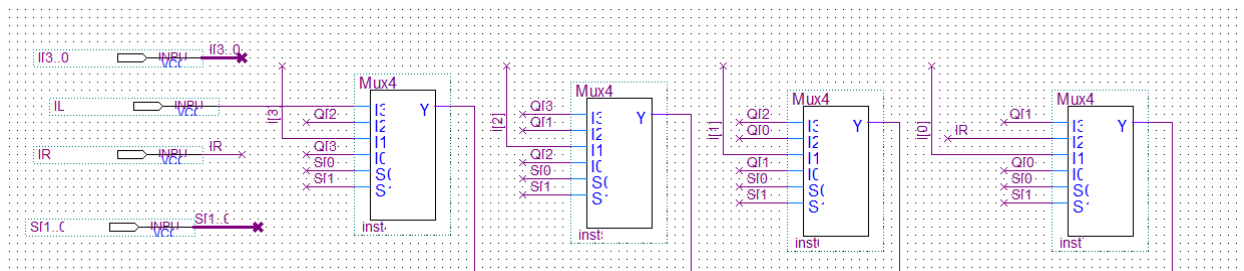
- + Chức năng:
 - Lưu trữ dữ liệu tại mỗi bước xung clock (Clk).
 - Dữ liệu đầu ra (Y_3, Y_2, Y_1, Y_0) phản ánh trạng thái của các flip-flop sau mỗi chu kỳ xung nhịp.



Hình 4.2: Thiết kế

4.2.2. Khối chọn tín hiệu input:

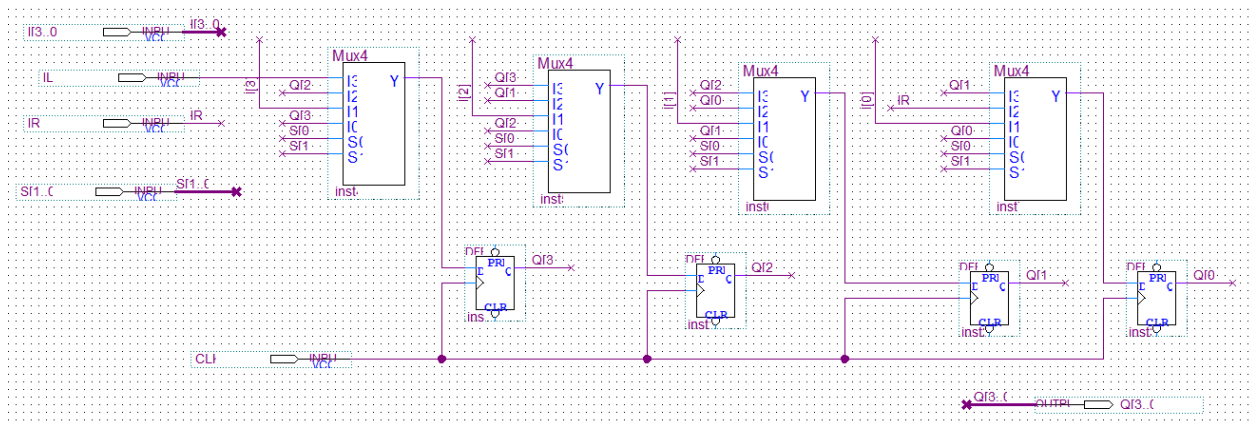
- + Các khối Mux 4-1 có nhiệm vụ chọn 1 trong 4 input tùy thuộc vào chức năng.
 - Dữ liệu từ bit bên trái (IL) hoặc bên phải (IR).
 - Dữ liệu từ các đầu ra trước đó (Q3,Q2,Q1,Q0).
 - Tín hiệu dữ liệu trực tiếp từ các đầu vào (I3,I2,I1,I0).
- + Dựa trên tín hiệu điều khiển S1,S0 bộ chọn xác định nguồn dữ liệu nào được chuyển đến đầu vào của các flip-flop.



Hình 4.3: Thiết kế

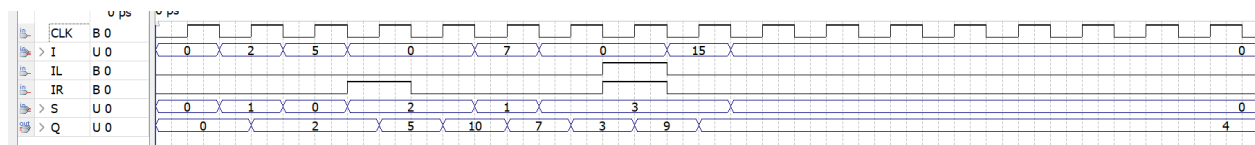
4.2.3. Thiết kế hoàn thiện:

- + Chức năng: Mạch này hoạt động như một **bộ dịch và lưu trữ 4 bit**, với khả năng thực hiện các chức năng:
 - Lưu trữ dữ liệu hiện tại.
 - Dịch dữ liệu sang trái hoặc phải.
 - Nạp dữ liệu mới từ các ngõ vào song song.



Hình 4.4: Thiết kế

4.3. Waveform:



Hình 4.5: Waveform

Giải thích:

- Ở chu kỳ đầu, tín hiệu điều khiển $S = 00$ (no change) không có sự thay đổi gì.
- Ở chu kỳ thứ 2, $S = 01$ (Load input) cho phép ghi input = 2 vào. Sau đó $S = 00$ (No change) không thay đổi gì cả.
- Ở chu kỳ 3, $S = 10$ (Shift left) với $IR = 1$ ta được 5.
- Ở chu kỳ 4, $S = 10$ (Shift left) với $IR = 0$ ta được 0.
- Ở chu kỳ 5, $S = 01$ (Load input) ghi input = 7 vào.
- Ở chu kỳ 6, $S = 10$ (Shift right) với $IL = 0$ ($7 \gg 1 = 3$)
- Ở chu kỳ 7, $S = 10$ (Shift right) với $IL = 1$ ta được 9.

Bài 5. Register-file with 1 write port and 2 read ports

5.1. Tổng quan thiết kế:

Mục tiêu thiết kế **register file 4x4** là tạo một cấu trúc lưu trữ gồm 4 thanh ghi, mỗi thanh ghi có độ rộng 4-bit, hỗ trợ **1 cổng ghi** và **2 cổng đọc** hoạt động song song. Hệ thống cho phép truy xuất dữ liệu nhanh, đồng bộ với tín hiệu clock, đảm bảo hiệu quả.

Đầu vào:

- + Dữ liệu cần lưu trữ.
- + Địa chỉ lưu trữ dữ liệu
- + Các tín hiệu điều khiển, đọc/ghi dữ liệu

Đầu ra:

- + Dữ liệu được truy xuất.

5.2. Thiết kế:

5.2.1. Decoder 2 to 4:

Đầu vào:

- + E: chân enable cho phép decoder hoạt động hay không.
- + A1, A0: hai chân tín hiệu .

Đầu ra:

- + Q0, Q1, Q2, Q3: tích cực mức cao (1) khi được chọn.

+

Bảng kích thích:

Bảng 5.1. Bảng tín hiệu các trạng thái kích thích

E	A1	A0	Q0	Q1	Q2	Q3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

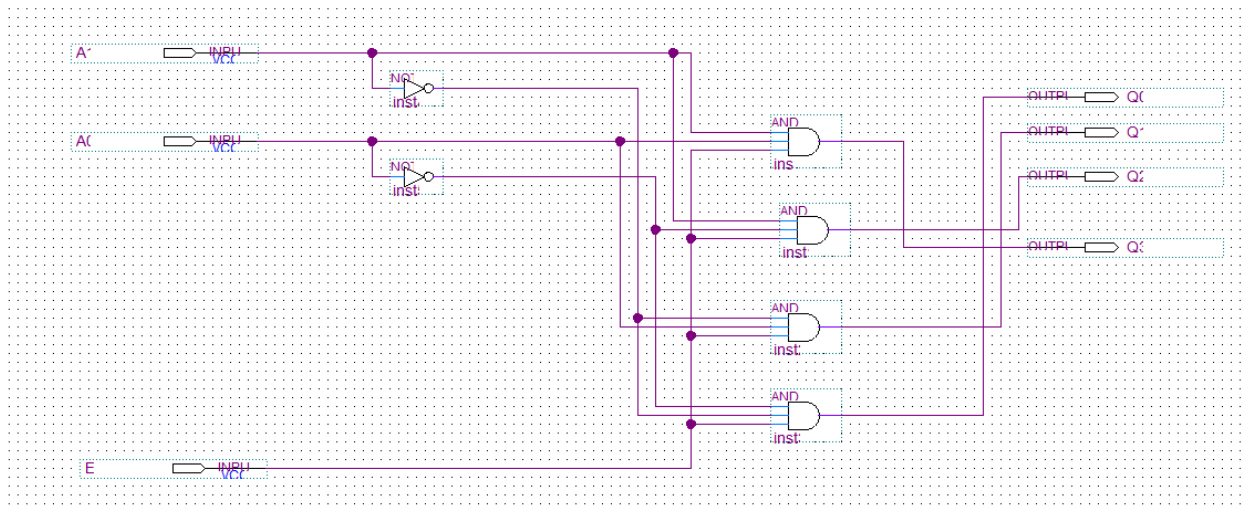
Phương trình kích thích:

$$Q0 = EA1'A0'$$

$$Q1 = EA1'A0$$

$$Q2 = EA1A0'$$

$$Q3 = EA1A0$$



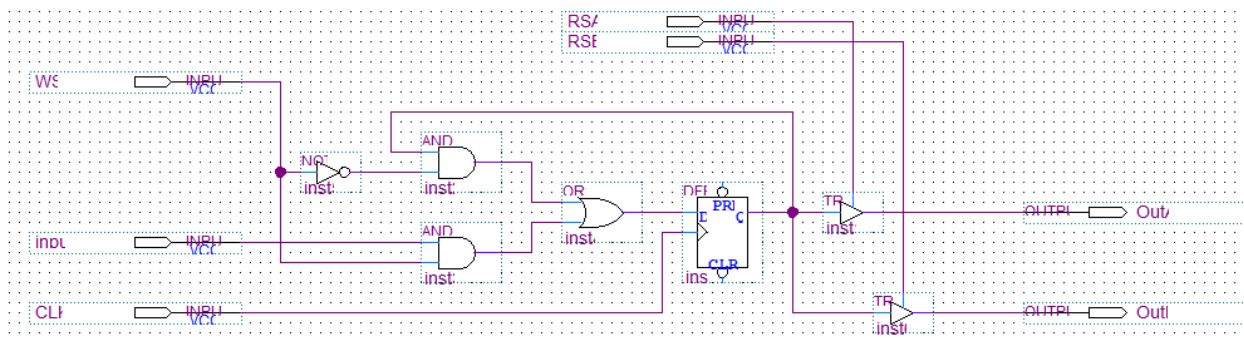
Hình 5.1: Khối Decoder 2 to 4.

5.2.2. Register file cell:

Chức năng: Có chức năng lưu dữ liệu, cho phép ghi và đọc.

Nguyên lý hoạt động:

- + Sử dụng D-flipflop để lưu trữ dữ liệu mỗi khi có Clk cạnh lên.
- + $WE = 1$ thì cho phép ghi input vào D-flipflop
- + $RSA = 1$ thì bật tri cho phép dữ liệu ra
- + $RSB = 1$ thì bật tri cho phép dữ liệu ra .

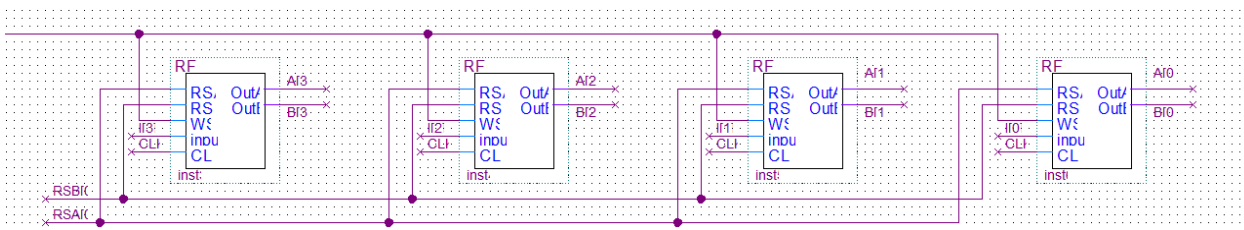


Hình 5.2: Khối RFC.

5.2.3. Hàng dữ liệu:

Chức năng:

- + Lưu dữ liệu 4 bit vào 1 dòng sử dụng 4 khối RFC cùng 1 dòng (Cùng tín hiệu WE, RSA, RSB).



Hình 5.3: Thiết kế 1 hàng trong Register.

5.2.4. Thiết kế hoàn thiện:

Chức năng:

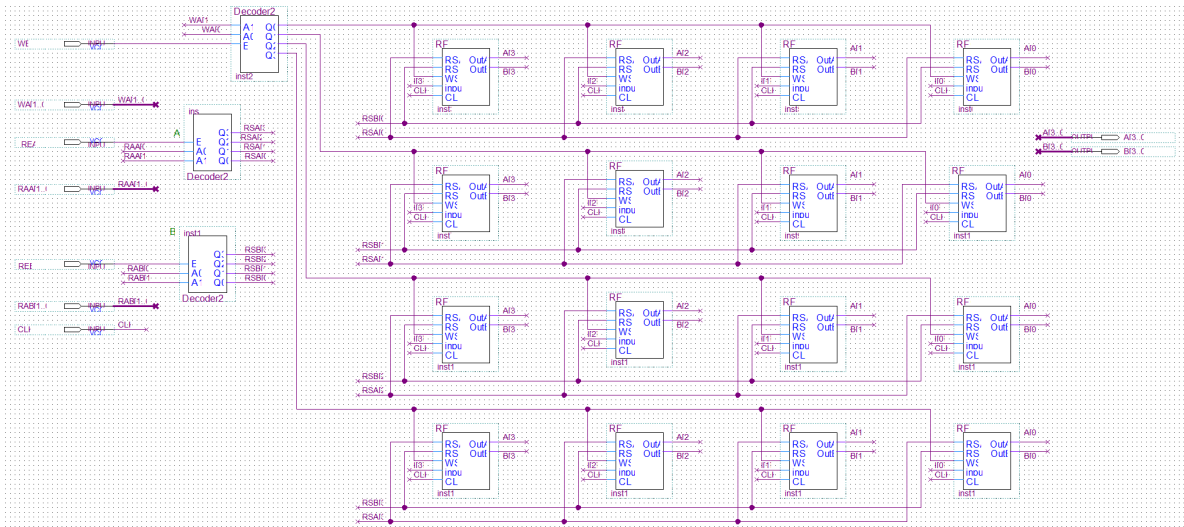
- + Sử dụng khối Decoder 2 to 4 để xác định dòng nào được ghi vào và đọc dữ liệu ở dòng nào. Có thể lưu trữ 4 bit trên mỗi dòng.

Đầu vào:

- + $I[3..0]$: dữ liệu đầu vào với độ rộng 4 bit.
- + WE: cho phép ghi dữ liệu vào Register.
- + $WA[1..0]$: tín hiệu có độ rộng 2 bit chứa địa chỉ ghi dữ liệu vào.
- + REA, REB: tín hiệu cho phép đọc dữ liệu từ Register.
- + $RAA[1..0]$, $RAB[1..0]$: tín hiệu có độ rộng 2 bit chứa địa chỉ đọc dữ liệu ra.
- + CLK: cấp xung clock cho mạch.

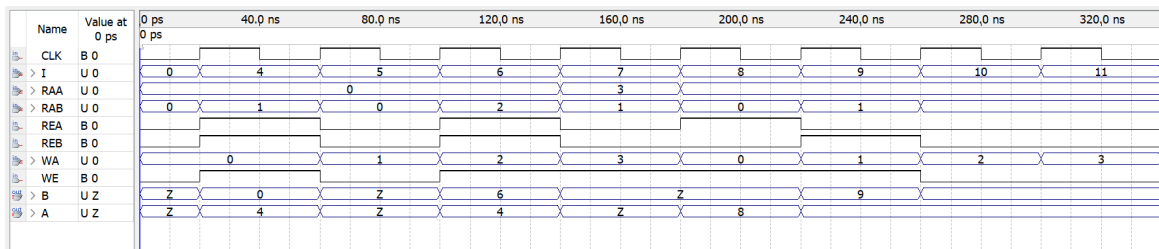
Đầu ra:

- + $A[3..0]$, $B[3..0]$: đầu ra có độ rộng 4 bit.



Hình 5.4. Thiết kế tổng của Register 4x4.

5.3. Waveform:



Hình 5.5: Waveform của register 4x4.

- Tại chu kỳ thứ 1: tín hiệu WE = 1 cho phép ghi Input = 4 vào register tại địa chỉ WA = 0. Đồng thời tín hiệu REA = 1 và REB = 1 lần lượt cho phép đọc dữ liệu tại địa chỉ RAA = 0 và RAB = 1 có được giá trị trả về lần lượt là A = 0 và B = 4.

- Tại chu kỳ thứ 2: tín hiệu $WE = 0$ không cho phép ghi dữ liệu mới vào Register, tín hiệu $REA = 0$ và $REB = 0$ không cho phép đọc dữ liệu từ Register.
- Tại chu kỳ thứ 3: tín hiệu $WE = 1$ cho phép ghi Input = 6 vào register tại địa chỉ $WA = 2$. Đồng thời tín hiệu $REA = 1$ và $REB = 1$ lần lượt cho phép đọc dữ liệu tại địa chỉ $RAA = 0$ và $RAB = 2$ có được giá trị trả về lần lượt là $A = 4$ và $B = 6$.
- Tại chu kỳ thứ 4: Tín hiệu $WE = 1$ cho phép ghi Input = 7 vào register tại địa chỉ $WA = 3$. Tín hiệu $REA = 0$ và $REB = 0$ không cho phép đọc dữ liệu từ Register.
- Tại chu kỳ thứ 5: tín hiệu $WE = 1$ cho phép ghi Input = 6 vào register tại địa chỉ $WA = 2$. Đồng thời tín hiệu $REA = 1$ cho phép đọc dữ liệu tại địa chỉ $RAA = 0$ có được giá trị trả về là $A = 8$. Tín hiệu $REB = 0$ không cho phép đọc dữ liệu.
- Tại chu kỳ thứ 6: tín hiệu $WE = 1$ cho phép ghi Input = 9 vào register tại địa chỉ $WA = 1$. Đồng thời tín hiệu $REB = 1$ cho phép đọc dữ liệu tại địa chỉ $RAB = 1$ có được giá trị trả về là $A = 9$. Tín hiệu $REA = 0$ không cho phép đọc dữ liệu.

Bài 6: Random Access Memory (Ram 16x8)

6.1. Tổng quan thiết kế:

RAM (Random Access Memory) 16x8 là một bộ nhớ truy cập ngẫu nhiên có dung lượng 16 từ (words), mỗi từ có độ dài 8 bit.

Tín hiệu điều khiển:

Bảng 6.1. Bảng tín hiệu điều khiển

CS	RWS	Operations
0	X	No change
1	0	Read
1	1	Write

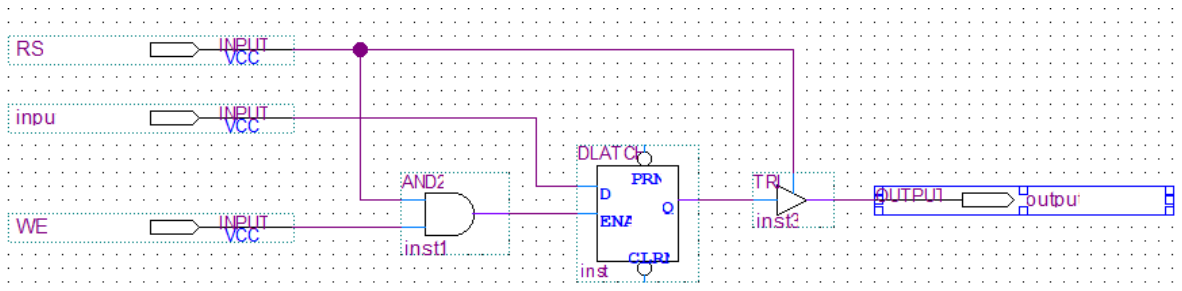
6.2. Thiết kế:

6.2.1. Memory Cell:

Cấu tạo: Sử dụng khối DLatch để lưu dữ liệu.

Nguyên lý hoạt động:

- + Khi RS (Row Select) = 1 và WE = 1 (Write Enable) đồng thời thì tín hiệu ở mức cao cho phép DLatch ghi dữ liệu mới.
- + Khi RS = 1 thì Tri-State bật cho phép dữ liệu ra.



Hình 6.1: Thiết kế khối MC.

6.2.2. Ram 4x4:

Đầu vào:

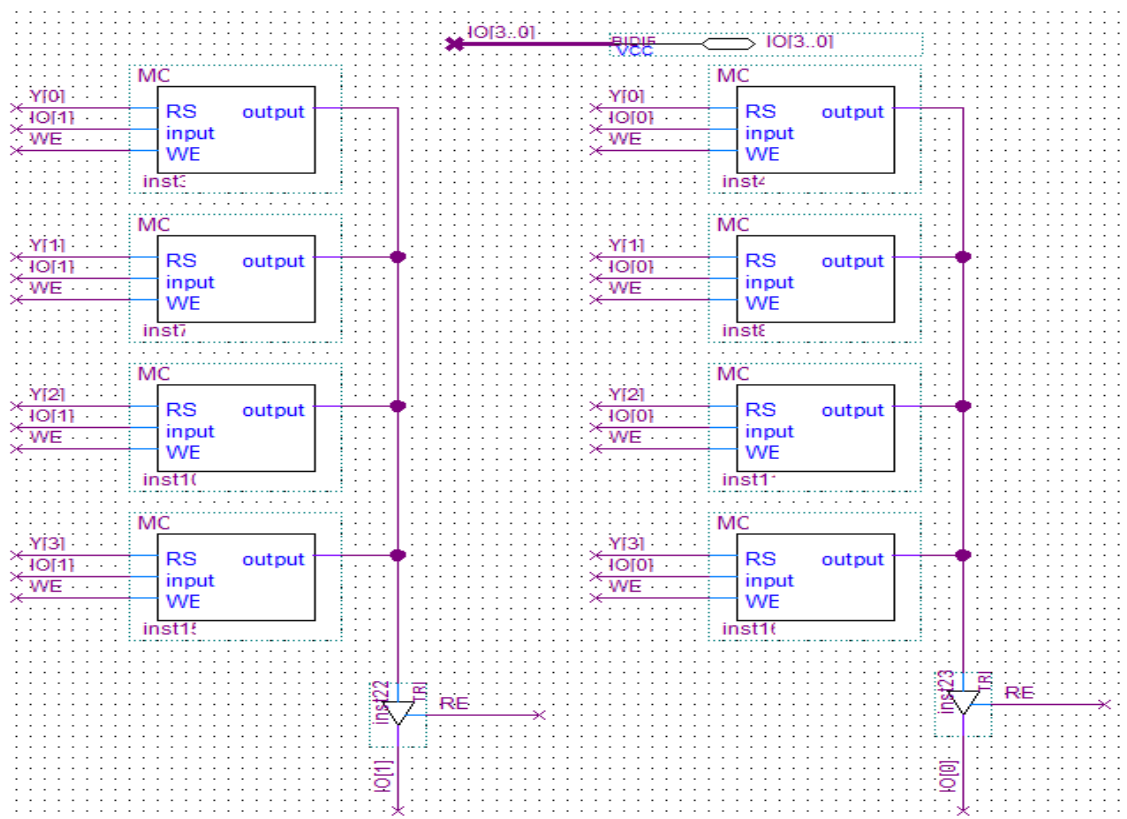
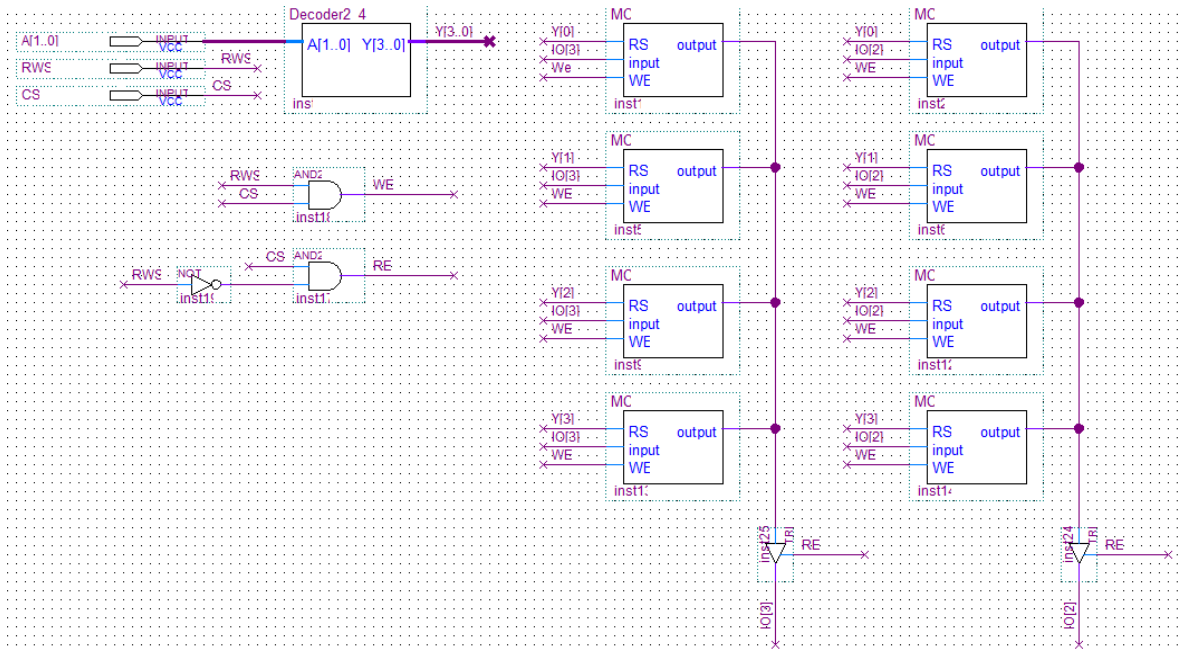
- + A[1..0]: tín hiệu có độ rộng 2 bit để xác định địa chỉ cần đọc/ghi.
- + RWS: tín hiệu điều khiển cho phép đọc/ghi của ram.
- + CS: cho phép các hoạt động của RAM.
- + IO[3..0]: Đầu vào dữ liệu 4 bit.

Đầu ra:

- IO[3..0]: Đầu ra dữ liệu 4 bit.

Cấu tạo:

- + 4 dòng mỗi dòng có chứa 4 khối MC để lưu trữ dữ liệu 4 bit.
- + Sử dụng Decoder 2 to 4 để xác định dòng nào sẽ được lưu dựa vào tín hiệu A1, A0
- + Điều khiển các tín hiệu WE, RE bằng RWS (Read/Write Select) và CS (Chip Select).
 - RWS = 1: Write.
 - RWS = 0: Read.
 - Khi RWS và CS đồng thời bằng 1 thì Ram sẽ thực hiện chức năng tương ứng với tín hiệu RWS.

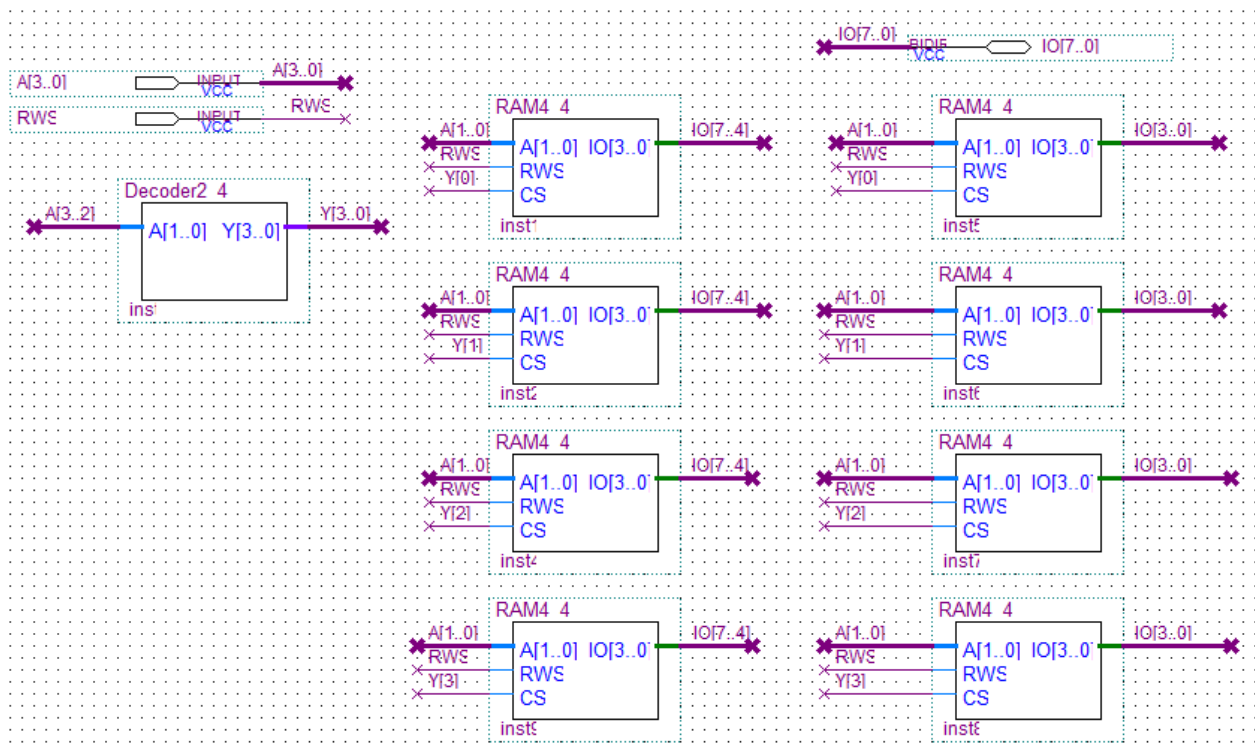


Hình 6.2: Thiết kế Ram 4x4.

6.2.3. Ram 16x8:

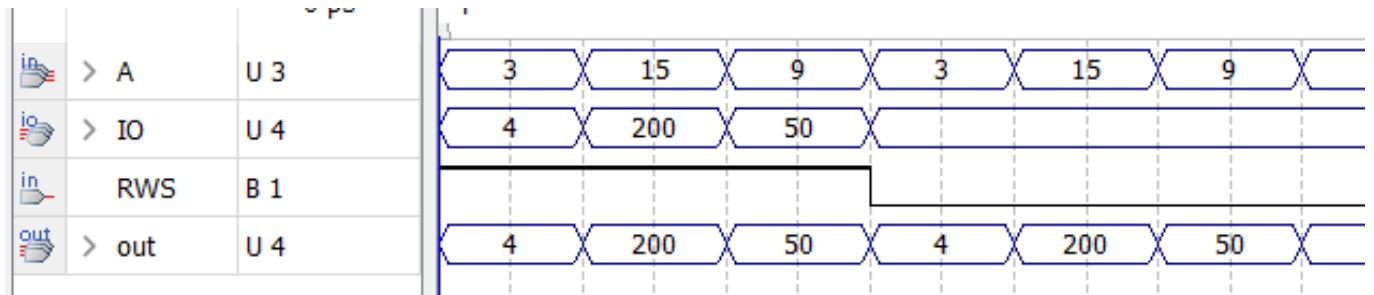
Nguyên lý hoạt động:

- + Có 16 dòng mỗi dòng lưu được 16 bit.
- + Sử dụng tín hiệu A3, A2, A1, A0 để chọn dòng ghi và đọc tương ứng.
 - A3, A2 sử dụng khối decoder 2 to 4 để chọn các khối Ram 4x4 ở các dòng.
 - A1, A0 sử dụng để chọn các dòng trong khối Ram 4x4 ở dòng đã chọn bằng A3, A2
- + Sử dụng tín hiệu RWS để chọn chức năng Read/Write.



Hình 6.3: Thiết kế Ram 16x8.

6.3. Waveform:



Hình 6.4: Waveform Ram 16x8.

Giải thích:

- Ở chu kỳ 1, RWS = 1 (write) ghi Input = 3 vào hàng 3 (A = 3) của Ram.
- Ở chu kỳ 2, RWS = 1 (write) ghi Input = 200 vào hàng 15 (A = 15) của Ram.
- Ở chu kỳ 3, RWS = 1 (write) ghi Input = 50 vào hàng 9 (A = 9) của Ram.
- Ở chu kỳ 4, RWS = 0 (read) đọc input ở hàng 3 (A = 3) được 4.
- Ở chu kỳ 5, RWS = 0 (read) đọc input ở hàng 15 (A = 15) được 200.
- Ở chu kỳ 6, RWS = 0 (read) đọc input ở hàng 9 (A = 9) được 50

Bài 7. Stack 16x8

7.1. Tổng quan thiết kế:

Stack (ngăn xếp) là một cấu trúc dữ liệu hoạt động theo nguyên lý **LIFO (Last In, First Out)**. Trong thiết kế phần cứng, stack thường được triển khai bằng cách sử dụng RAM và các bộ đếm (counter) để quản lý việc đọc, ghi dữ liệu. Sau đây là tổng quan về thiết kế khối stack sử dụng RAM 16x8 cùng 2 bộ đếm: **Top** và **Top-1**.

Tín hiệu điều khiển:

Operation table:

Push/Pop	Enable	Operations
X	0	No change
0	1	Push
1	1	Pop

Hình 7.1: Tín hiệu điều khiển chức năng khối Stack.

Control table:

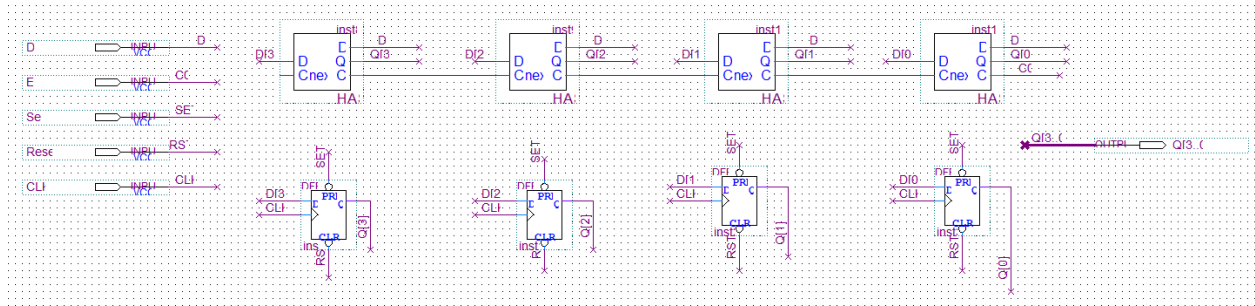
		Selector control	Memory controls		Counter controls	
Push/Pop Enable		S	CS	RWS	D	E
X	0	X	0	0	X	0
0	1	1	1	1	0	1
1	1	0	1	0	1	1

Hình 7.2: Tín hiệu control.

7.2. Thiết kế:

7.2.1. 4-bit up/down binary counter:

(đã được giới thiệu ở các phần trên.)

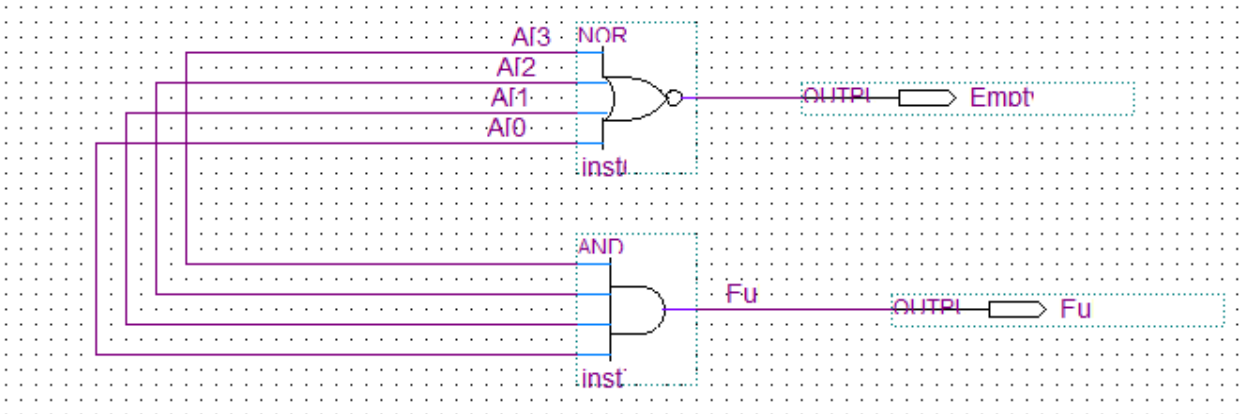


Hình 7.3: Thiết kế 4-bit up/down binary counter.

7.2.2. Khối tín hiệu báo trạng thái:

+ Cách hoạt động:

- Empty = 1 khi $A[3..0] = 0000$ tức là chưa có dữ liệu nào được ghi vào Stack.
- Full = 1 khi $A[3..0] = 1111$ tức là đã ghi full dữ liệu của stack.



Hình 7.4: Thiết kế bộ check trạng thái Stack.

7.2.3. Thiết kế hoàn thiện:

Đầu vào:

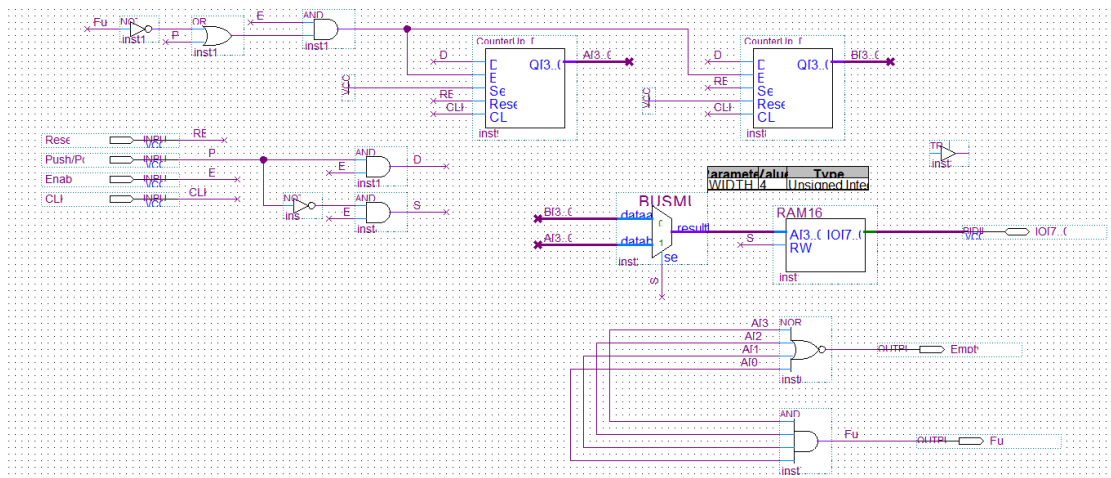
- + Reset: Tín hiệu reset 2 khối counter.
- + Push/Pop: Tín hiệu điều khiển chức năng của Stack.
- + Enable: Tín hiệu cho phép Stack hoạt động hay không.
- + CLK: xung clock
- + IO[7..0]: Input nhập vào khi push

Đầu ra:

- + IO[7..0]: Output của Stack khi pop
- + Empty: Tín hiệu thông báo Stack đang trống.
- + Full: Tín hiệu thông báo Stack đã đầy.

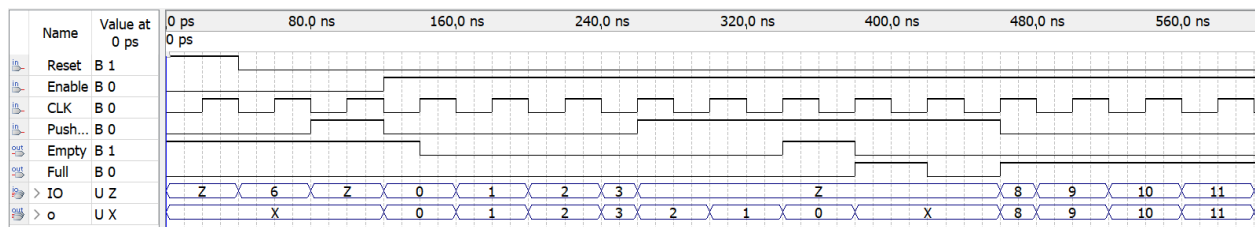
Nguyên lý hoạt động:

- + Sử dụng 2 khối Counter hoạt động song song. Khối ban đầu (Top) được reset về 0000, khối thứ 2 (Top - 1) được set về 1111.
- + Khi đếm khối (Top) sẽ đếm trước (Top - 1) 1 đơn vị.
- + Sử dụng 1 khối Selector để chọn tín hiệu từ khối Top và Top - 1. Nếu Push thì nhận tín hiệu từ khối Top và ngược lại.
- + Khi tín hiệu Enable thì Stack mới hoạt động.



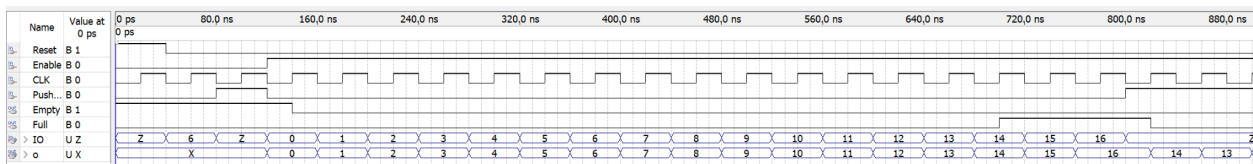
Hình 7.5: Thiết kế Stack.

7.3. Waveform:



Hình 7.6: Waveform1 Stack.

- Từ chu kì thứ 1 đến chu kì thứ 3: tín hiệu Enable = 0 không cho phép stack hoạt động. Do đó tín hiệu Empty = 1 cho thấy Stack đang trống.
- Từ chu kì thứ 4 đến chu kì thứ 6: tín hiệu Push/Pop = 0 cho phép ghi dữ liệu vào Stack từ bus IO, tín hiệu Empty = 0 (stack đang có dữ liệu).
- Từ chu kì thứ 7 đến chu kì thứ 11: tín hiệu Push/Pop = 1 cho phép đọc dữ liệu từ Stack và xuất ra bus IO. Tại chu kì thứ 9 dữ liệu đã được đọc hết nên tín hiệu Empty = 1 và sau đó không còn dữ liệu Bus IO = X.



Hình 7.7: Waveform2 Stack.

- Thực hiện việc kiểm tra tín hiệu FULL: bắt đầu nạp dữ liệu từ clock thứ 4 đến clock thứ 19 nạp giá trị từ 0 đến 14 tổng cộng có 15 giá trị tương ứng với 15 lần nạp và FULL = 1 tại lúc nạp giá trị thứ 14 và sau đó nạp thêm giá trị 15 và 16 khi stack đã full và xuất ra thì giá trị ở vị trí 15 vẫn là 14.

Bài 8. Queue 16x8

8.1. Tổng quan thiết kế:

Queue (hàng đợi) là một cấu trúc dữ liệu hoạt động theo nguyên lý **FIFO (First In, First Out)**, nghĩa là dữ liệu được thêm vào trước sẽ được lấy ra trước. Trong phần cứng, queue thường được triển khai bằng RAM và các bộ đếm để quản lý vị trí đầu và cuối của hàng đợi. Dưới đây là thiết kế queue sử dụng **RAM 16x8** và hai bộ đếm: **Front** và **Back**.

Operation table:

<i>Read/Write</i>	<i>Enable</i>	<i>Operations</i>
<i>X</i>	<i>0</i>	<i>No change</i>
<i>0</i>	<i>1</i>	<i>Read</i>
<i>1</i>	<i>1</i>	<i>Write</i>

Hình 8.1: Tín hiệu điều khiển Queue.

Control table:

Empty	Full	R/W	Enable	S	CS	RWS	Front	Back
x	x	x	0	x	0	x	0	0
0	0	0	1	1	1	0	1	0
0	0	1	1	0	1	1	0	1
0	1	0	1	1	1	0	1	0
0	1	1	1	x	0	x	0	0
1	0	0	1	x	0	x	0	0
1	0	1	1	0	1	1	0	1
1	1	0	1	x	0	x	0	0
1	1	1	1	x	0	x	0	0

$S = R/W \cdot \text{Enable}$

$CS = (\text{Empty} \cdot R/W + \text{Full} \cdot R/W) \cdot \text{Enable}$

$RWS = R/W \cdot \text{Enable}$

$\text{Front} = \text{Empty} \cdot R/W \cdot \text{Enable}$

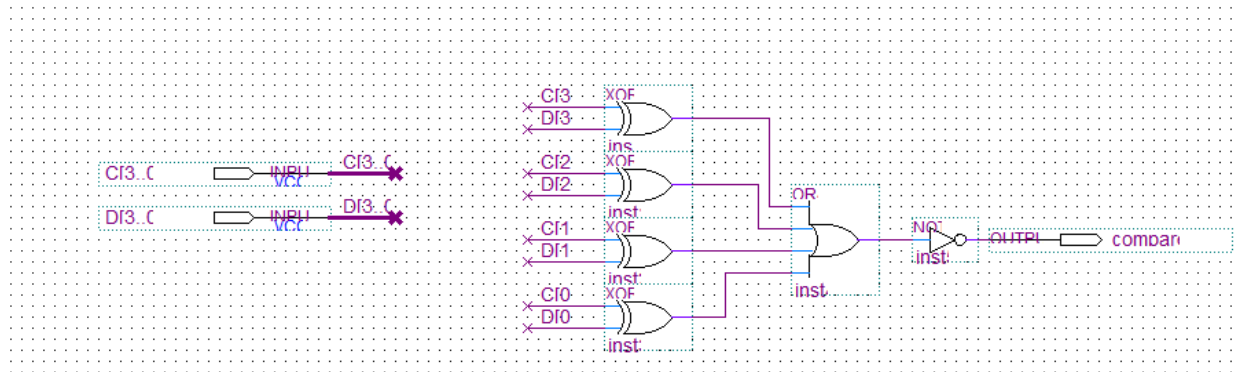
$\text{Back} = \text{Full} \cdot R/W \cdot \text{Enable}$

8.2. Thiết kế:

8.2.1. Bộ so sánh:

So sánh C (4 bit) và D (4 bit):

- + $C = D$, output = 1.
- + $C \neq D$, output = 0.



Hình 8.2: Bộ so sánh 2 số.

8.2.2. Thiết kế hoàn thiện:

Đầu vào:

- + CLK: tín hiệu xung clock cấp cho mạch.
- + Reset: tín hiệu reset các bộ đếm.
- + ENABLE: tín hiệu cho phép mạch hoạt động
- + READ_WRITE: tín hiệu chọn đọc/ghi dữ liệu.
- + IO[7..0]: Input nhập vào.

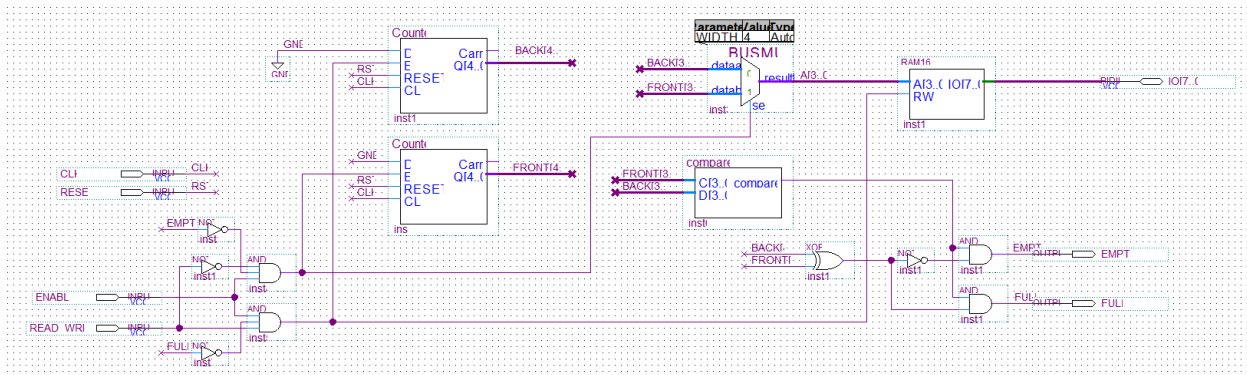
Đầu ra:

- + IO[7..0]: Output xuất ra.
- + FULL: Tín hiệu báo Queue đang đầy.
- + EMPTY: Tín hiệu báo Queue đang trống

Nguyên lý hoạt động:

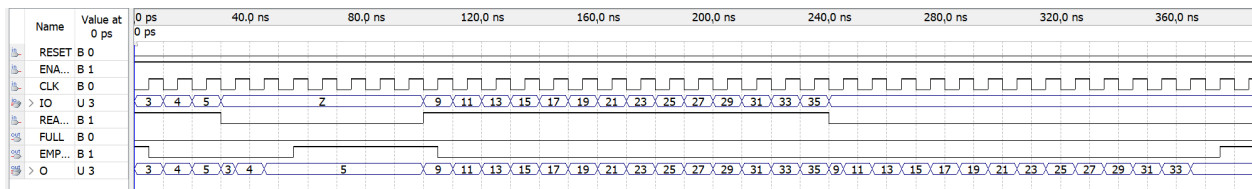
- + Sử dụng 2 khối Counter (Font, Back). Ban đầu reset cả 2 khối về 0000.
- + Sử dụng tín hiệu Read/Write để điều khiển chức năng Read hoặc Write:
 - Khi Read/Write = 1: thực hiện việc Write, khối Back tăng lên 1.

- Khi Read/Write = 0: thực hiện việc Read, khối Front tăng lên 1.
- + Sử dụng khối Selector để chọn tín hiệu của 1 trong 2 khối Front, Back.



Hình 8.3: Thiết kế Queue.

8.3. Waveform:



Hình 8.4: Waveform Queue.

- Trước chu kỳ 1: Empty = 1 vì queue chưa có dữ liệu.
- Từ chu kỳ 1 đến 3: với tín hiệu Read_Write = 1 cho phép ghi dữ liệu 3,4,5 từ IO vào Queue. Empty = 0 vì queue đã có dữ liệu.
- Từ chu kỳ 4 đến chu kỳ 10: với tín hiệu Read_Write = 0 cho phép đọc dữ liệu từ queue và xuất ra IO. Ở đây chỉ cần 3 chu kỳ để đọc hết dữ liệu được ghi vào từ chu kỳ 1 - 3 của queue và sau đó Empty= 1 biểu thị queue trống.
- Từ chu kỳ 11 trở đi Queue tiến hành ghi 13 lần dữ liệu vào và sau đó đọc ra đúng thứ tự đã ghi vào

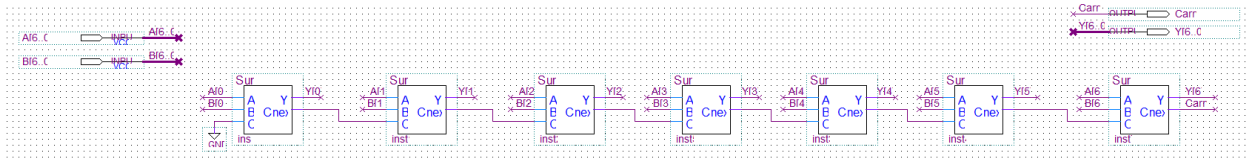
Bài 9. One accumulator

9.1. Tổng quan thiết kế:

Mạch tính tổng từ 1 đến một số nguyên dương đầu vào 4 bit (giá trị từ 0 đến 15) được triển khai bằng cách sử dụng một **khối Sum (7 bit)**, một **Counter** để tạo dãy số từ 1 đến giá trị đầu vào, và một **Register** để lưu trữ giá trị tổng.

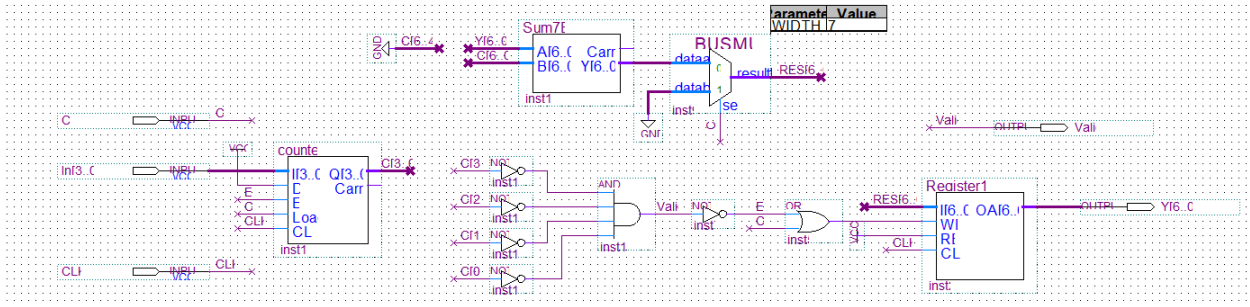
9.2. Thiết kế Datapath:

Khối Cộng 2 số 7 bit:



Hình 9.1: Thiết kế bộ cộng 2 số 7 bit.

Hoàn thiện Datapath:



Hình 9.2: Thiết kế bộ Datapath tính tổng từ 1 đến input.

Thiết kế:

- + Gồm 3 phần chính: Counter, Register, Sum7bit

Đầu vào:

- + C: tín hiệu điều khiển
- + In[3:0]: tín hiệu đầu vào có độ rộng 4 bit.
- + CLK: tín hiệu xung clock cấp cho mạch.

Đầu ra:

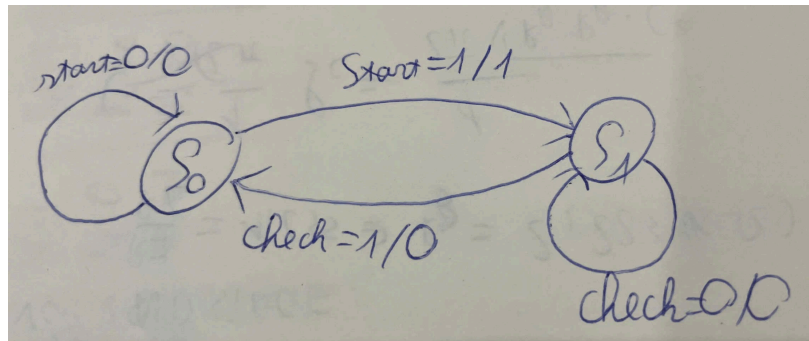
- + Y[6:0]: kết quả tổng từ 1 đến In[3:0].
- + Valid: tín hiệu báo valid input.

Nguyên lý hoạt động:

- + Nạp giá trị từ **In[3:0]** vào **counter** (bộ đếm).
- + Đồng thời lưu giá trị từ **In[3:0]** vào **register** (thanh ghi).
- + Trong mỗi chu kỳ:
 - Bộ đếm (**counter**) giảm giá trị (count down).
 - Giá trị từ **counter** được cộng với nội dung của **register**.
 - Kết quả phép cộng được lưu ngược lại vào **register**.
- + Khi **counter** giảm xuống **0**, dừng quá trình xử lý.
- + Xuất kết quả cuối cùng từ **register** ra ngoài.

9.3. Thiết kế Controller:

Sơ đồ trạng thái:



Hình 9.3: Sơ đồ trạng thái One accumulator.

Bảng trạng thái:

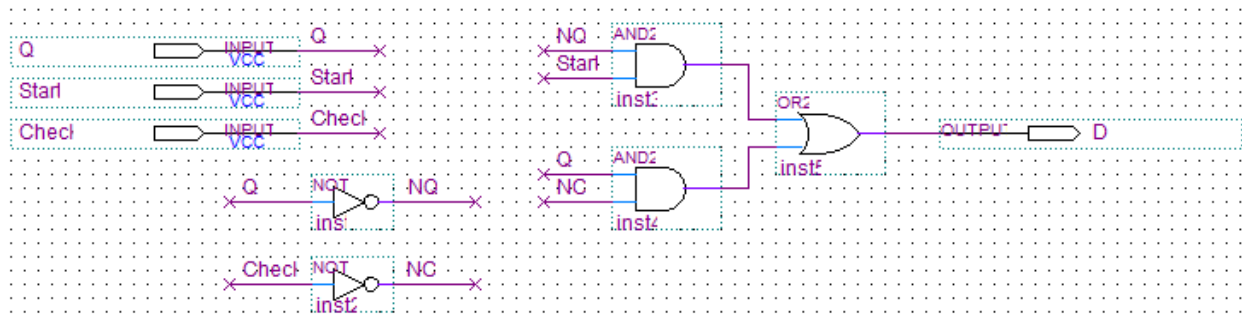
State	Current State	Next State / S			
		Start = 1		Start = 0	
		Check = 0	Check = 1	Check = 0	Check = 1
	Q	D	D	D	D
S0	0	1 / 1	1 / 1	0 / 0	0 / 0
S1	1	1 / 0	0 / 0	1 / 0	0 / 0

Hình 9.4: Bảng trạng thái và output.

$$D = Q'Start + QCheck'$$

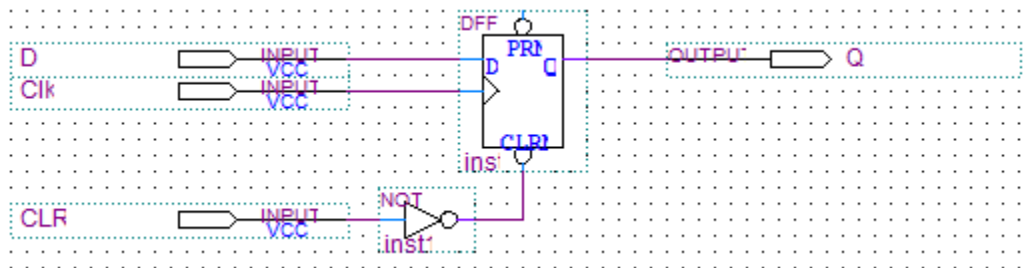
$$S = Q'Start$$

Khối Next state:



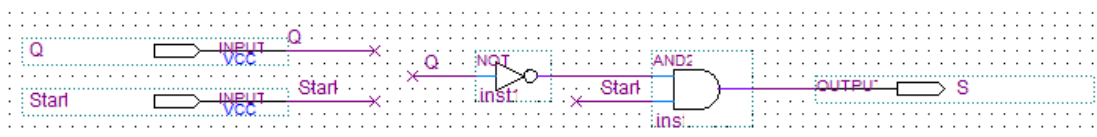
Hình 9.5: Khối Next State.

Khối Current state:



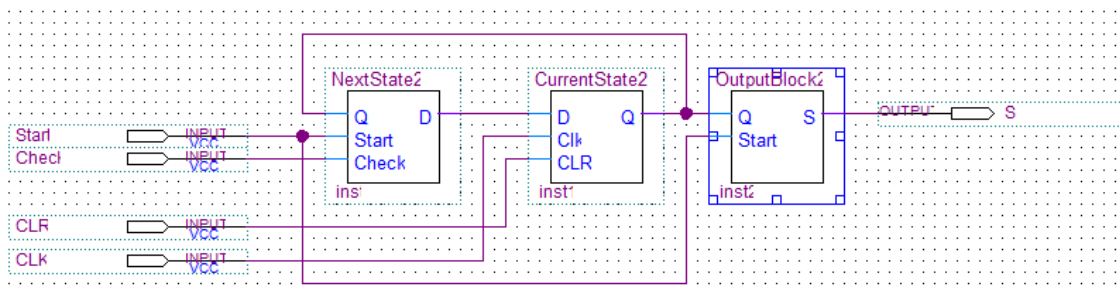
Hình 9.6: Khối Current State.

Khối tín hiệu Output điều khiển:



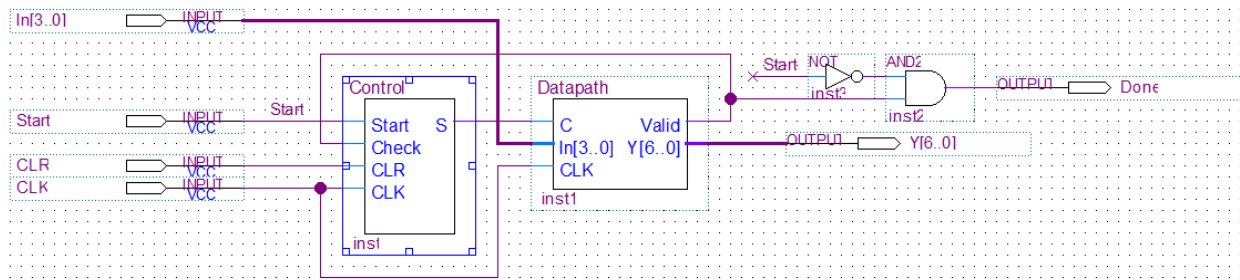
Hình 9.7: Khối Output.

Hoàn thiện Controller:



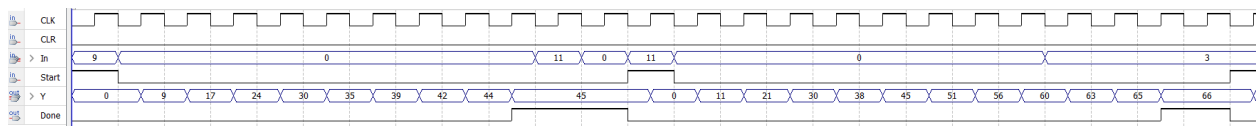
Hình 9.8: Thiết kế khối Control.

Thiết kế hoàn thiện:



Hình 9.9: Thiết kế tổng.

9.4. Waveform:



Hình 9.10: Waveform one Accumulator.

Giải thích:

- Ở chu kỳ đầu, Start = 1 cho phép ghi input = 9 vào. Ta thực hiện tính tổng từ 1 đến 9.
- Tới chu kỳ 10, tín hiệu done bật lên, kết quả của phép tính tổng từ 1 đến 9 là 45.
- Sau chu kỳ 10, ta có thể bật start = 1 và tiếp tục nhập input = 11 vào.
- Sau đó mạch tiếp tục tính và done = 1 thông báo đã tính xong với output là 66.

Bài 10. One's-Count

10.1. Tổng quan thiết kế:

Mạch đếm số bit '1' trong một số nhị phân 8 bit có thể được thiết kế bằng cách sử dụng các khối **Shift Register with Parallel Load (SRwPL)** và **Counter**.

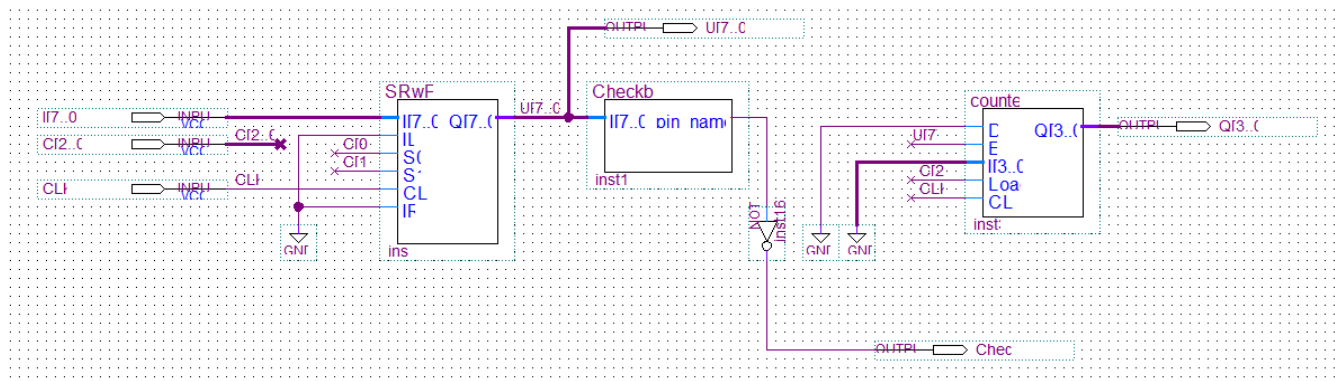
10.2. Thiết kế Datapath:

Thiết kế:

- + Gồm 3 khối chính: SRwPL, Counter, Check

Nguyên lý hoạt động:

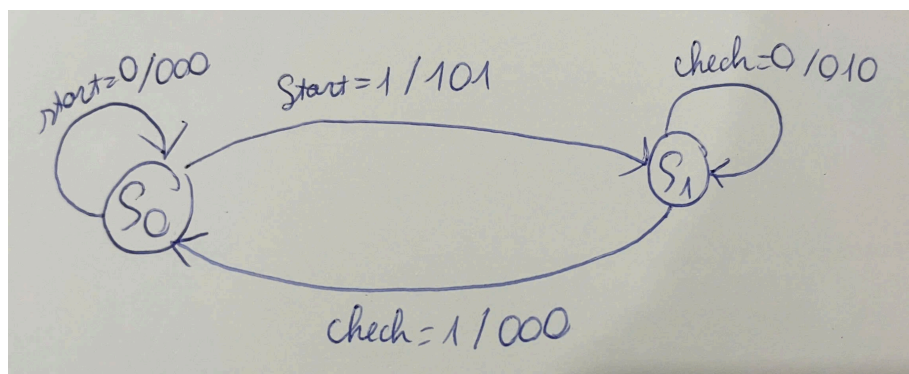
- + Tín hiệu C[2..0] dùng để điều khiển hoạt động của khối Datapath
 - C[2] điều khiển hoạt động load của khối Control.
 - C[1..0] điều khiển khối SRwPL.
- + Shift Left liên tục, nếu bit cuối bằng 1 thì bật counter đếm lên 1.
- + Khối tín hiệu Check nếu không có bit 1 nào trong dữ liệu sẽ đưa ra tín hiệu để gửi tới khối Control.



Hình 10.1: Thiết kế mạch One's-count.

10.3. Thiết kế Controller:

10.3.1. Sơ đồ trạng thái:



Hình 10.2: Sơ đồ trạng thái.

10.3.2. Tín hiệu Control:

State	Current State	Next State / C[2:0]			
		Start = 1		Start = 0	
		Check = 0	Check = 1	Check = 0	Check = 1
	Q	D	D	D	D
S0	0	1 / 101	1 / 101	0 / 000	0 / 000
S1	1	1 / 010	0 / 000	1 / 010	0 / 000

Hình 10.3: Bảng trạng thái kế tiếp và output.

$$D = Q'Start + QCheck'$$

$$C[2] = C[0] = Q'Start$$

$$C[1] = QCheck'$$

Thiết kế:

- + Gồm 3 khối chính: NextState, CurrentState, OutputBlock

Chức năng:

- + Với mỗi trạng thái sẽ đưa ra các tín hiệu tương ứng để điều khiển khối Datapath.'

Input:

- + Tín hiệu Start.
- + Xung clock (CLK)

- + Tín hiệu Control.

Hình 10.4: Thiết kế khối Control.

10.3.3. Nextstate:

Chức năng:

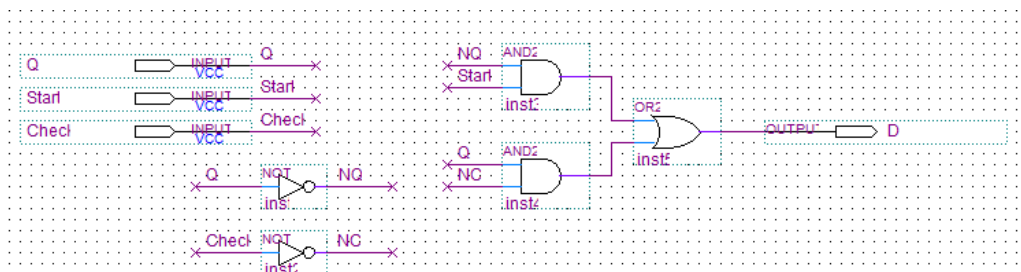
- + Tính toán trạng thái kế tiếp của hệ thống dựa trên trạng thái hiện tại và tín hiệu đầu vào.
- + Được định nghĩa bởi hàm chuyển trạng thái (**State Transition Function**) trong FSM.

Đầu vào:

- + Tín hiệu trạng thái hiện tại (**CurrentState**).
- + Tín hiệu đầu vào (**Inputs**).

Đầu ra:

- + Trạng thái kế tiếp (**NextState**), được gửi đến khối D-flipflop để lưu trữ trong chu kỳ tiếp theo.



Hình 10.5: Khối Next State.

10.3.4. Currentstate:

Chức năng:

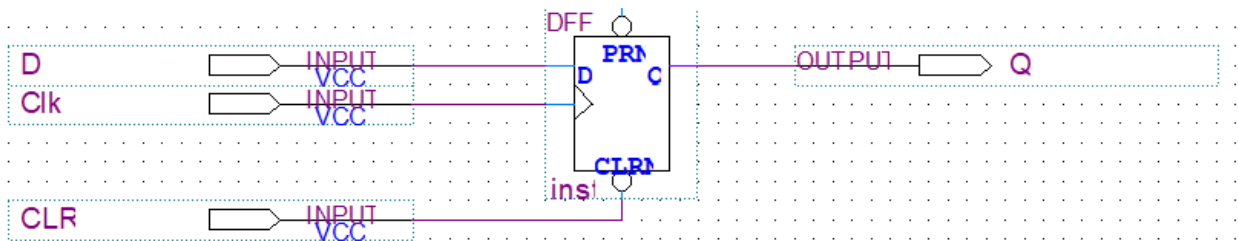
- + Lưu giữ trạng thái hiện tại của hệ thống và cung cấp thông tin trạng thái này cho các khối khác.
- + Được cập nhật vào mỗi chu kỳ clock bởi trạng thái kế tiếp từ khối **NextState**.

Đầu vào:

- + Tín hiệu trạng thái kế tiếp (**NextState**).
- + Tín hiệu clock (để đồng bộ cập nhật trạng thái).

Đầu ra:

- + Trạng thái hiện tại (**CurrentState**) để cung cấp cho các khối **NextState** và **OutputBlock**.



Hình 10.6: Khối Current State.

10.3.5. Output Block:

Chức năng:

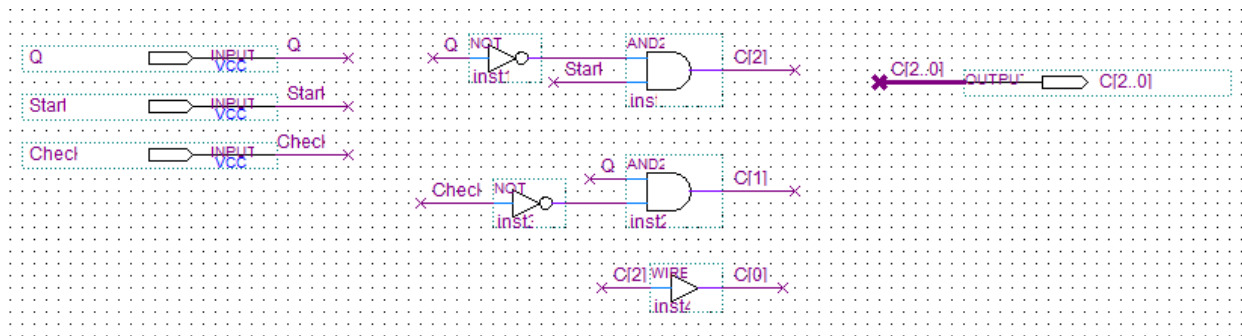
- + Tạo tín hiệu đầu ra của hệ thống dựa trên trạng thái hiện tại và (tùy chọn) tín hiệu đầu vào.

Đầu vào:

- + Trạng thái hiện tại (**CurrentState**).
- + (Tùy chọn) Tín hiệu đầu vào (**Inputs**, nếu là Mealy Machine).

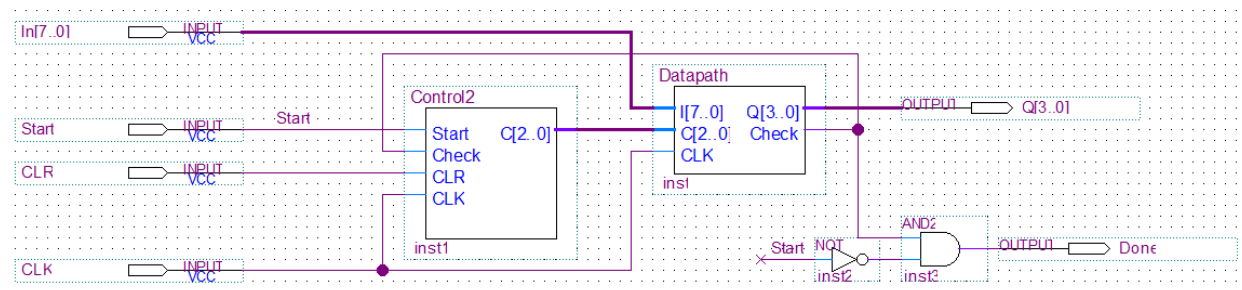
Đầu ra:

- + Tín hiệu điều khiển hoặc dữ liệu đầu ra của hệ thống (**Outputs**).



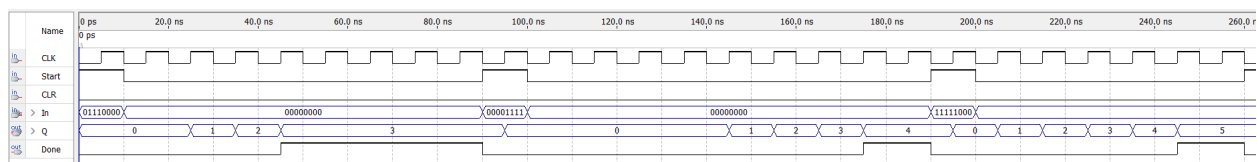
Hình 10.7: Khối Output.

10.4. Thiết kế tổng:



Hình 10.8: Thiết kế tổng.

10.5. Waveform:



Hình 10.9: Waveform One's-count.

Giải thích:

- Ở chu kỳ đầu, Start = 1 cho phép nhập input (01110000) vào.
- Mạch sẽ đếm số bit 1 của input và khi tín hiệu Done = 1 thông báo đã tính xong và cho output = 3.
- Sau đó, ta có thể bật Start = 1 và tiếp tục nhập input vào.
- Với Input = 00001111, mạch tính output 4.
- Với Input = 11111000, mạch tính output 5.

Bài 11. Square-root approximation

11.1. Tổng quan thiết kế:

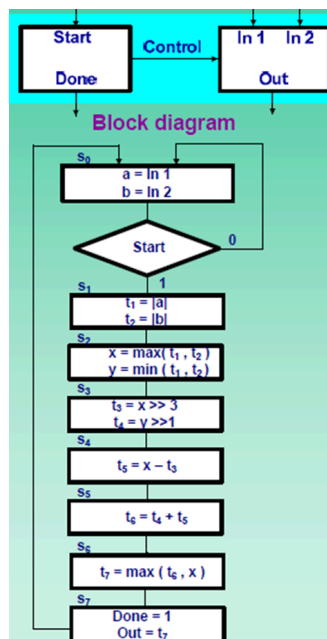
Mạch tính $\sqrt{a^2 + b^2}$ bằng cách sử dụng các mạch đơn giản dựa vào giải thuật dưới:

Giải thuật tính xấp xỉ căn bậc hai:

$$\sqrt{a^2 + b^2} \approx \max((0,875x + 0,5y), x)$$

where $x = \max(a, b)$
 $y = \min(a, b)$

ASM Chart:



Hình 11.1: ASM Chart

11.2. Thiết kế Datapath:

11.2.1. Khối Full Adder:

Chức năng:

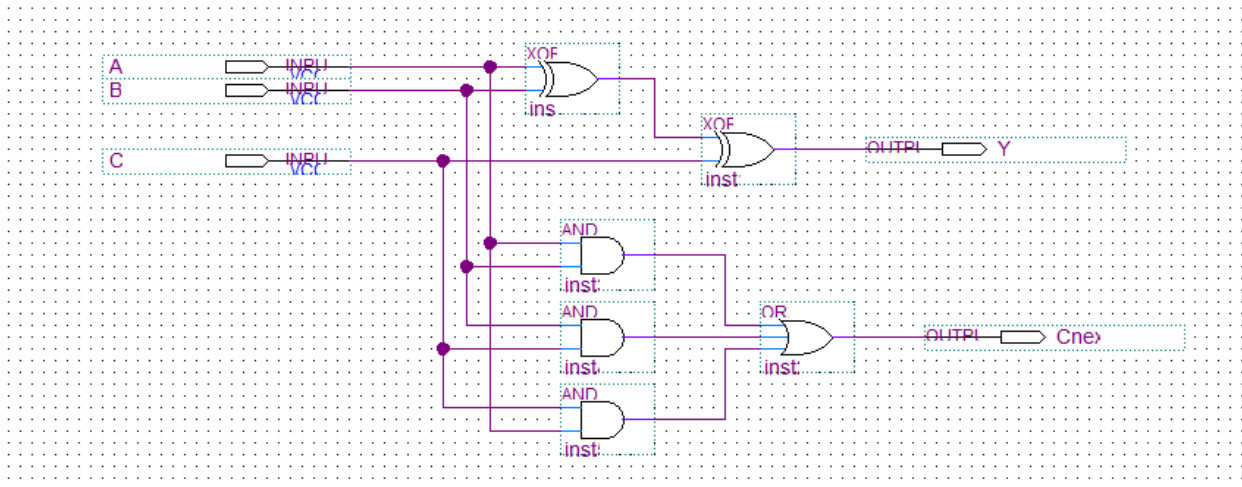
- + là một mạch tổ hợp có chức năng thực hiện phép cộng nhị phân trên ba bit đầu vào, bao gồm hai bit dữ liệu và một bit **carry-in** (dẫn từ phép cộng trước đó).

Đầu vào:

- + A: Bit đầu vào thứ nhất.
- + B: Bit đầu vào thứ hai.
- + C: Bit nhớ đầu vào (carry-in).

Đầu ra:

- + Y: Bit kết quả tổng (sum).
- + Cnext: Bit nhớ đầu ra (carry-out)



Hình 11.2: Thiết kế bộ Full Adder.

11.2.2. Khối Cộng 2 số 8 bit:

Thiết kế:

- + Gồm 8 khối **Full-Adder** được nối tiếp nhau.
- + Mỗi khối **Full-Adder** thực hiện phép cộng một cặp bit tương ứng từ hai số đầu vào và xử lý bit nhớ từ khối trước đó.

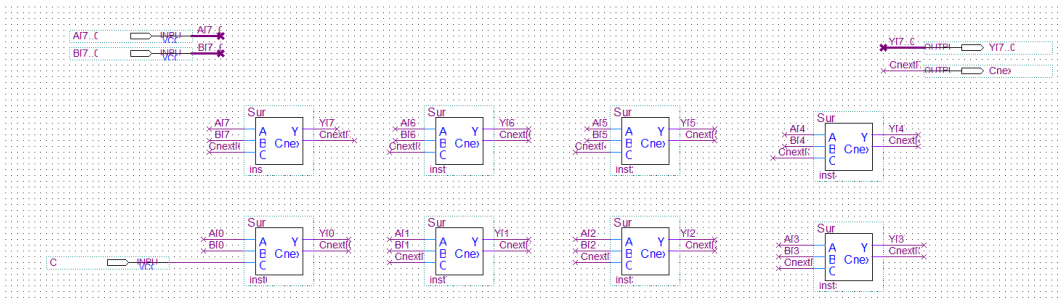
Đầu vào:

- + **A[7:0]**: Số nhị phân 8-bit đầu tiên.
- + **B[7:0]**: Số nhị phân 8-bit thứ hai.

- + **C**: Bit nhớ đầu vào (carry-in), thường được khởi tạo là 0 nếu không có phép toán trước đó.

Đầu ra:

- + **Y[7:0]**: Kết quả tổng 8-bit.
- + **Cnext**: Bit nhớ đầu ra (carry-out) sau khi cộng bit cuối cùng.



Hình 11.3: Thiết kế mạch cộng 2 số 8 bit.

11.2.3. Khối Trừ 2 số 8 bit:

Nguyên lý hoạt động:

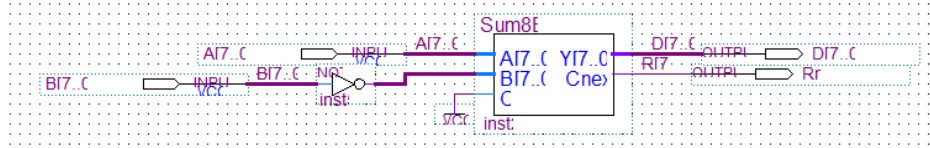
- + Được xây dựng bằng cách tận dụng khối **cộng 8-bit** với một số điều chỉnh. Điều này dựa trên nguyên tắc số học nhị phân, trong đó phép trừ $A-B$ - $BA-B$ có thể được biểu diễn thành phép cộng: $A-B=A+(-B)$

Đầu vào:

- + **A[7:0]**: Số bị trừ (minuend).
- + **B[7:0]**: Số trừ (subtrahend).

Đầu ra:

- + **Y[7:0]**: Kết quả phép trừ (difference).
- + **Cnext**: Bit nhớ ra, dùng để xác định nếu có **vay mượn** xảy ra trong phép trừ (borrow).



Hình 11.4: Thiết kế mạch trừ 2 số 8 bit.

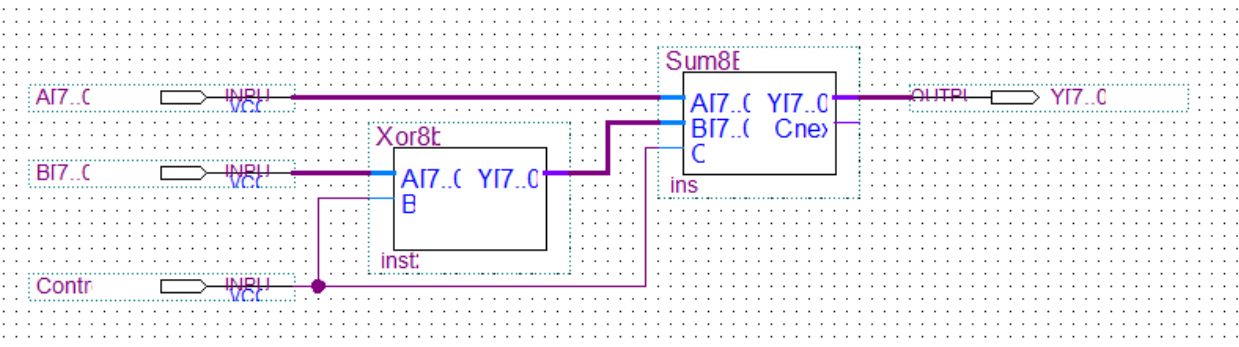
11.2.4. Khối Cộng/Trừ 2 số 8 bit:

Chức năng:

- + Control = 0: thực hiện phép cộng.
- + Control = 1: thực hiện phép trừ.

Nguyên lý hoạt động:

- + Khi **Control = 0**, các đầu vào của khối **Sum8bit** sẽ là A, B, và C=0. Khối thực hiện phép **cộng nhị phân thông thường** giữa A và B.
- + Khi **Control = 1**, các đầu vào của khối **Sum8bit** sẽ là A, NOT(B), và C=1. Khối thực hiện phép tính $A+(-B)$, tương ứng với việc cộng A với bù 2 của B ($\text{NOT}(B)+1$).



Hình 11.5: Thiết kế khối AddorSub.

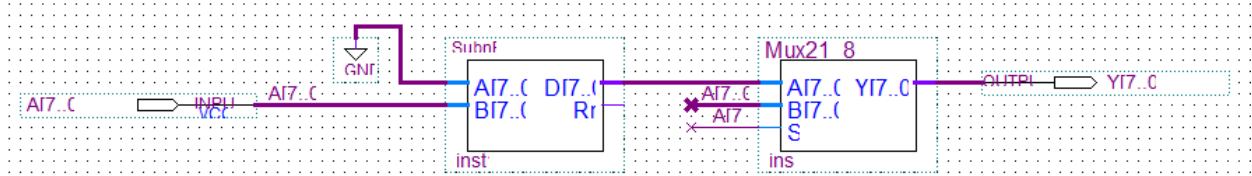
11.2.5. Khối trị tuyệt đối:

Chức năng:

- + Tính trị tuyệt đối của một số.

Nguyên lý hoạt động:

- + Khối **Selector** có chức năng chọn đầu ra dựa trên bit dấu của số đầu vào:
 - Nếu bit dấu là 0 (số dương hoặc 0): khối **Selector** xuất giá trị A.
 - Nếu bit dấu là 1 (số âm): khối **Selector** xuất giá trị 0 - A.



Hình 11.6: Thiết kế khối trị tuyệt đối.

11.2.6. Khối Min Max:

Chức năng:

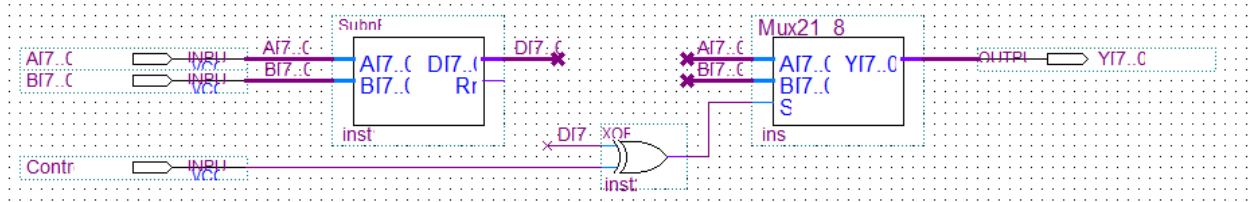
- + Tính Min, Max dựa vào tín hiệu Control.

Thiết kế:

- + Subtractor (Khối trừ)
- + Selector

Nguyên lý hoạt động:

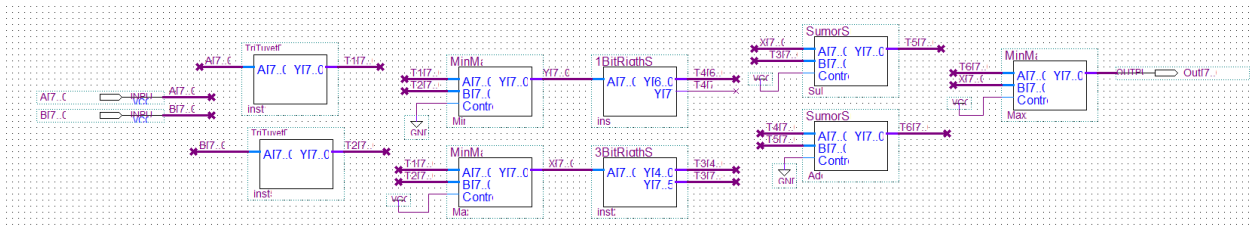
- + Subtractor (Khối trừ):
 - Thực hiện phép trừ $a - b$.
 - Kết quả của phép trừ cung cấp **bit dấu (Sign bit)**:
 - Bit dấu = 0: $a \geq b$.
 - Bit dấu = 1: $a < b$.
- + Tín hiệu điều khiển Min/Max Control:
 - Điều khiển việc chọn giá trị đầu ra là giá trị nhỏ nhất hay lớn nhất:
 - min/max control=0: Chọn giá trị nhỏ nhất ($\min(a,b)$).
 - min/max control=1: Chọn giá trị lớn nhất ($\max(a,b)$).
- + Khối Selector:
 - Nhận tín hiệu từ khối trừ (bit dấu) và tín hiệu điều khiển min/max control.
 - Tùy theo tín hiệu đầu vào, nó sẽ quyết định đầu ra:
 - Nếu cần giá trị nhỏ nhất: Dựa vào bit dấu để chọn a hoặc b.
 - Nếu cần giá trị lớn nhất: Tương tự, chọn dựa vào bit dấu.



Hình 11.7: Khối MinMax.

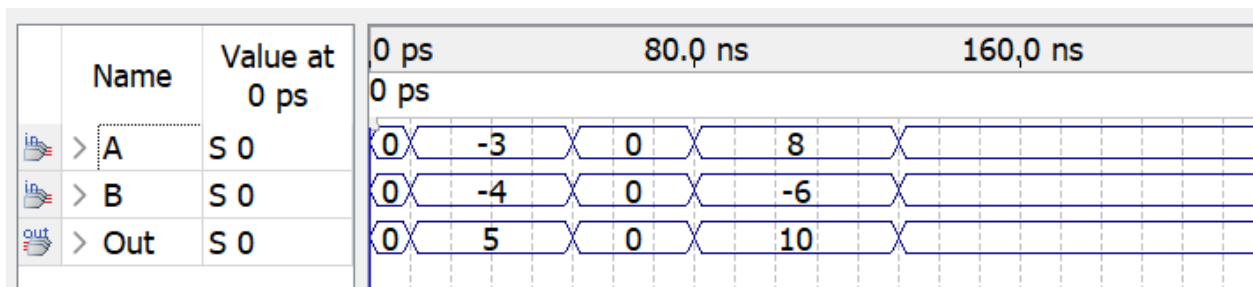
11.2.7. Thiết kế tổng:

Sử dụng các khối component để thực hiện các chức năng theo các trạng thái:



Hình 11.8: Thiết kế tổng.

11.3. Waveform:



Hình 11.9: Waveform.

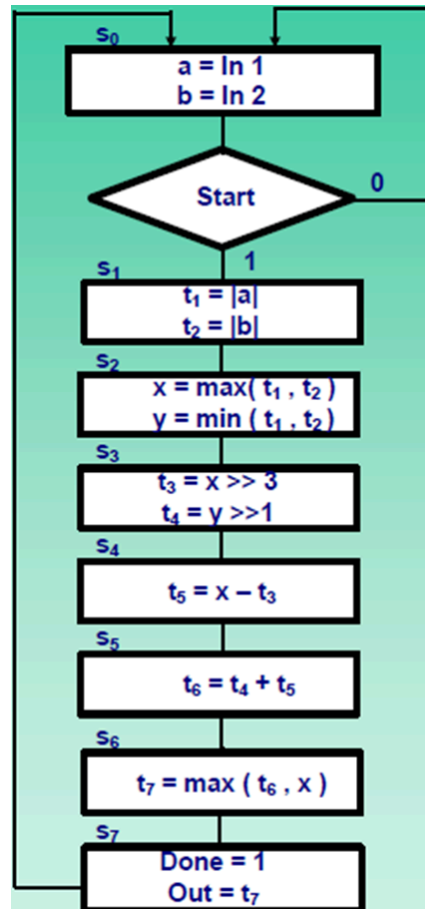
Giải thích:

- Với A = -3, B = -4 mạch tính $\sqrt{a^2 + b^2} = 5$.
- Với A = 8, B = -6 mạch tính $\sqrt{a^2 + b^2} = 10$.

Bài 12. Register assignment generated by the graph-partitioning algorithm

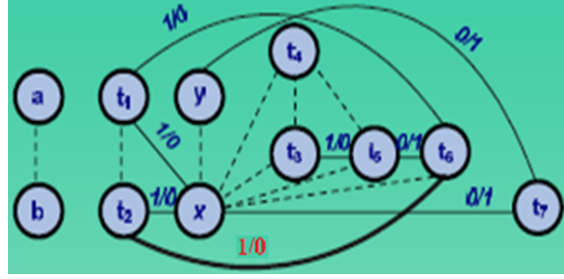
12.1. Tổng quan thiết kế:

ASM Chart:

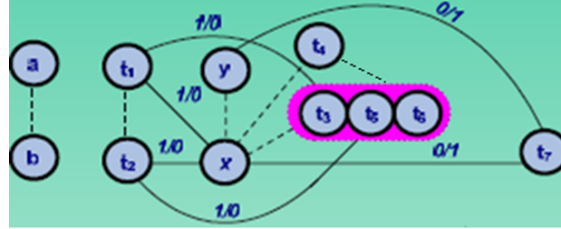


Hình 12.1: ASM Chart

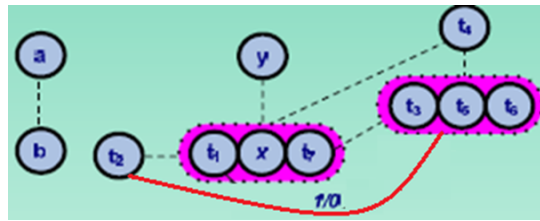
Register assignment generated by the graph-partitioning algorithm là một kỹ thuật trong lĩnh vực tối ưu hóa biên dịch (compiler optimization). Nó được sử dụng để gán các biến của chương trình (hoặc các giá trị tạm thời) vào một số lượng hữu hạn các thanh ghi của bộ xử lý. Đây là một vấn đề quan trọng để tăng hiệu suất thực thi chương trình. Tổng quát, thuật toán phân vùng đồ thị (graph-partitioning algorithm) thực hiện việc gán thanh ghi như sau:



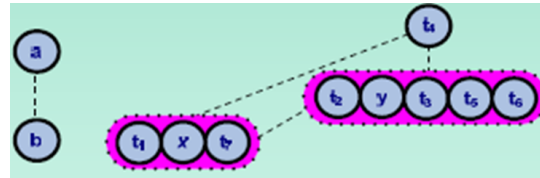
(a) Initial compatibility graph



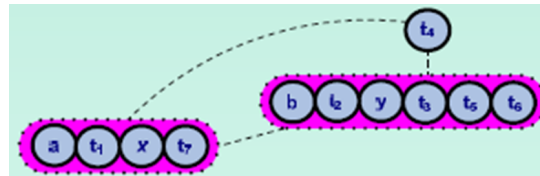
(b) Compatibility graph after merging t_3, t_5 and t_6



(c) Compatibility graph after merging t_1, x and t_7



(d) Compatibility graph after merging t_2 and y



(e) Final compatibility graph

Sau khi thực hiện giải thuật Graph Partitioning, thực hiện gom nhóm các giá trị lưu thành ghi:

$$- R1 = [a, t_1, x, t_7]$$

- R2 = [b , t2 , y , t3 , t5 , t6]
- R3 = [t4]

Đầu vào:

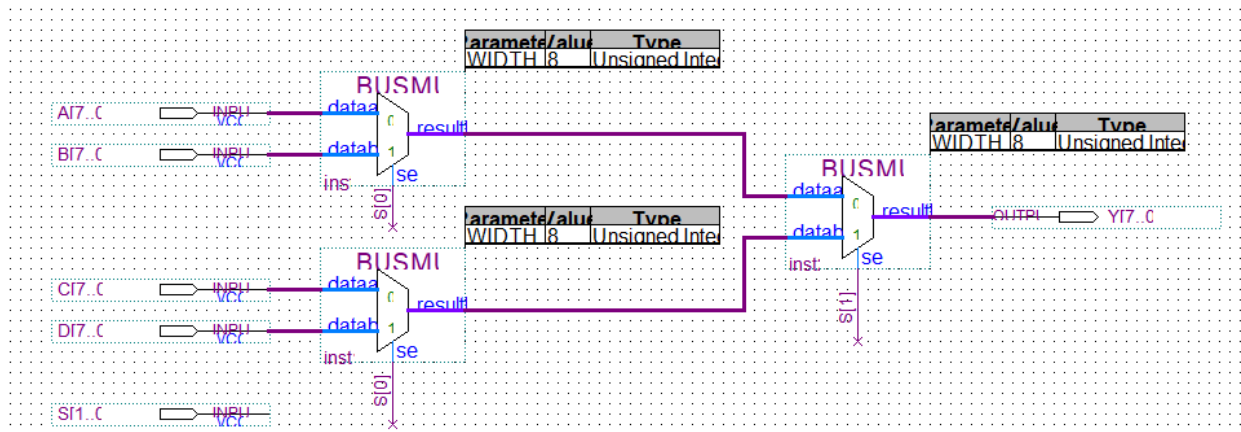
- Hai biến dữ liệu cần tính toán.
- Tín hiệu **Clock (CLK)**.
- Tín hiệu điều khiển từ **Control**.

Đầu ra:

- Kết quả của Datapath ($\sqrt{a^2 + b^2}$).

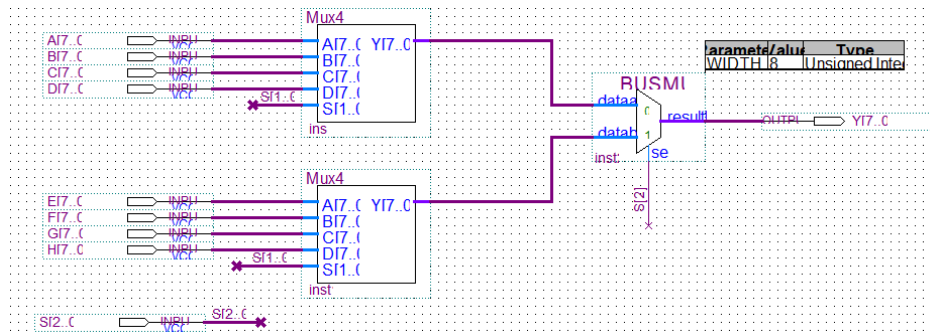
12.2. Thiết kế Datapath:

12.2.1. Khối Mux 4 to 1:



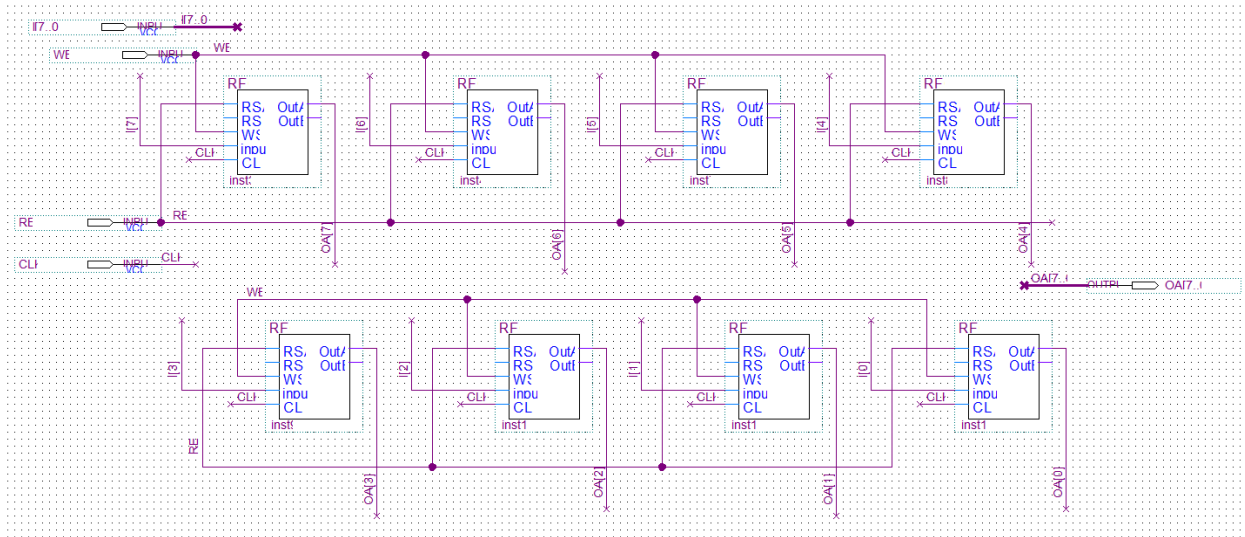
Hình 12.2: Khối Mux 4 to 1.

12.2.2. Khối Mux 8 to 1:



Hình 12.3: Khối Mux 8 to 1.

12.2.3. Khối Register 1x8:



Hình 12.4: Khối Register 1x8.

12.2.4. Hoàn thiện Datapath:

Thiết kế:

- + Gồm 3 register 8 bit để lưu các biến
- + 2 khối selector
- + Các khối tính toán (components)

Nguyên lý hoạt động

- + Các biến được gom nhóm và lưu trữ trong các **Register** thông qua thuật toán **Graph-Partitioning**, đảm bảo tối ưu hóa việc sử dụng tài nguyên.
- + Dữ liệu từ các register sau đó được truyền đến các **khối tính toán (components)**. Các khối này sẽ thực hiện các phép tính cần thiết và xuất kết quả.
- + Kết quả từ các khối tính toán được gửi trở lại các **Selector**, nơi tín hiệu điều khiển sẽ quyết định lựa chọn dữ liệu phù hợp để lưu vào **Register**. Quy trình này đảm bảo luồng dữ liệu được xử lý hiệu quả, đồng thời giảm thiểu xung đột giữa các biến khi sử dụng chung thanh ghi.

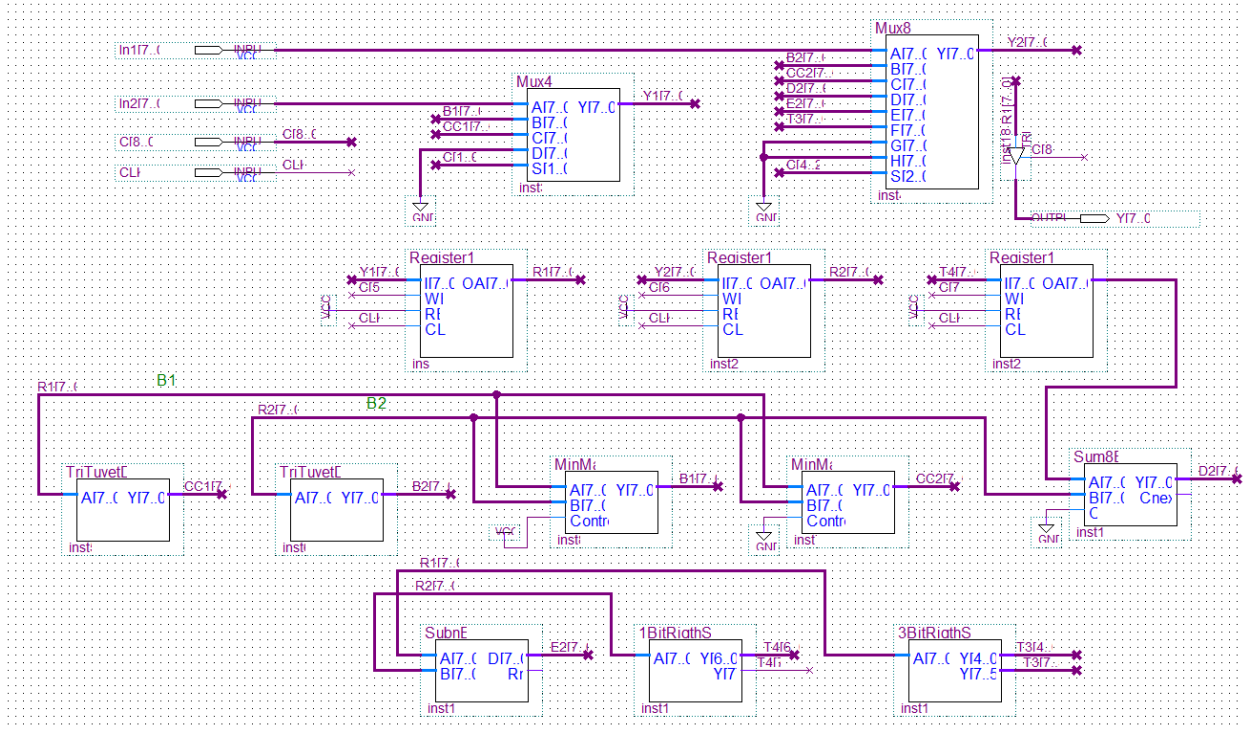
Input:

- + Hai biến dữ liệu cần tính toán.

- + Tín hiệu **Clock (CLK)**.
- + Tín hiệu điều khiển từ **Control**.

Output:

- + Kết quả của Datapath ($\sqrt{a^2 + b^2}$)



Hình 12.5: Thiết kế Datapath.

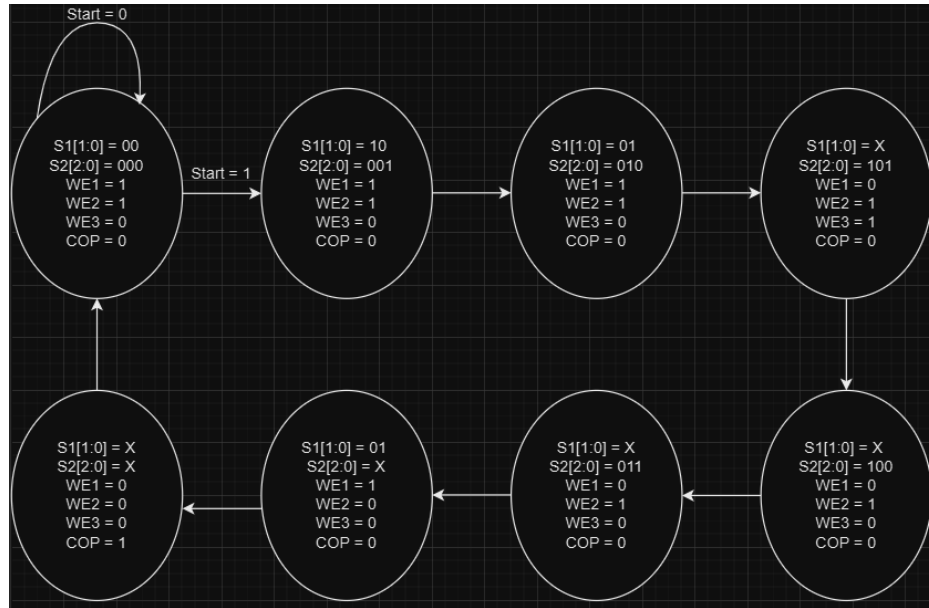
12.3. Thiết kế Controller:

12.3.1. Tín hiệu Control

State	Current State	Next State		Output					
		Start = 1	Start = 0	S1[1:0]	S2[2:0]	WE1	WE2	WE3	COP
	Q[2:0]	D[2:0]	D[2:0]						
S0	000	001	000	00	000	1	1	0	0
S1	001	010	010	10	001	1	1	0	0
S2	010	011	011	01	010	1	1	0	0
S3	011	100	100	X	101	0	1	1	0
S4	100	101	101	X	100	0	1	0	0
S5	101	110	110	X	011	0	1	0	0
S6	110	111	111	01	X	1	0	0	0
S7	111	000	000	X	X	0	0	0	1

Hình 12.6: Bảng trạng thái tiếp theo và output.

Sơ đồ chuyển trạng thái:



Hình 12.7: Sơ đồ chuyển trạng thái.

Chức năng:

- + Tín hiệu điều khiển (**control**) được tạo ra dựa trên **FSM (Finite State Machine)**
- + Tạo tín hiệu control dựa vào trạng thái hiện tại để điều khiển Datapath thực hiện nhiệm vụ phù hợp.

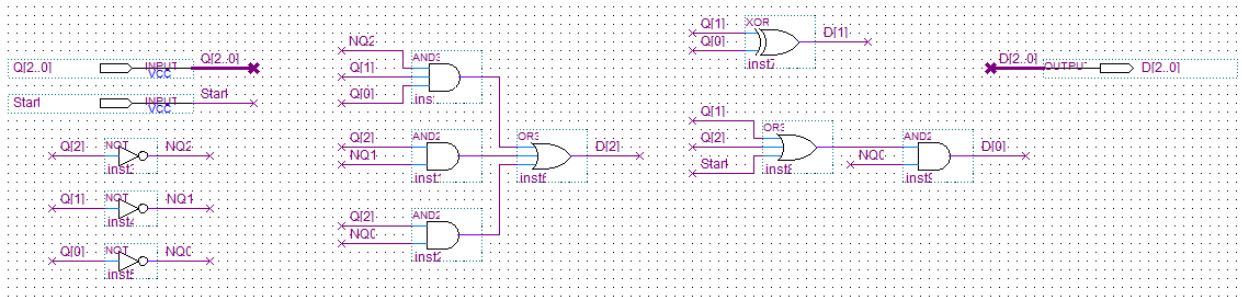
Input:

- + Tín hiệu Start.
- + Xung clock (CLK)
- + Tín hiệu Reset (CLR)

Output:

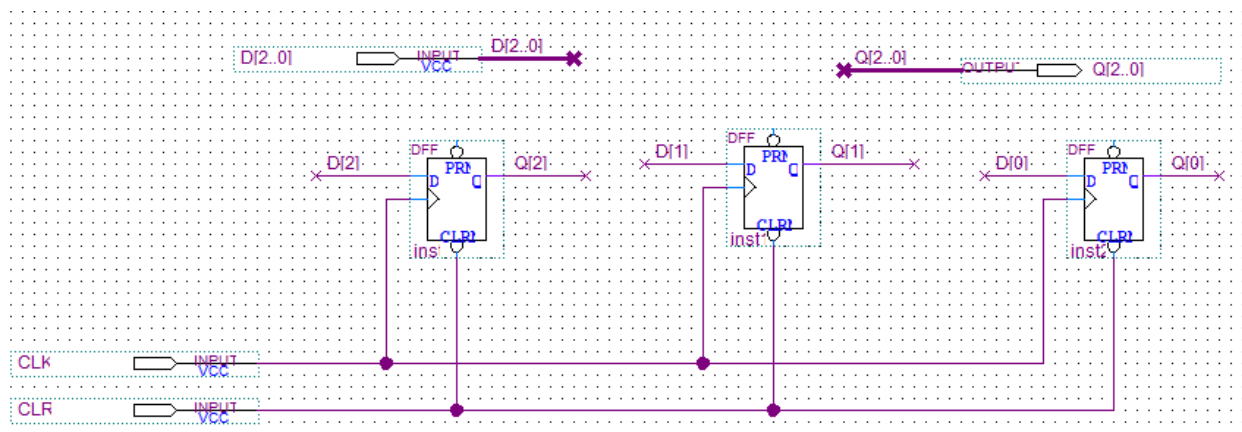
- + Tín hiệu Control.

12.3.2. Khối Next State



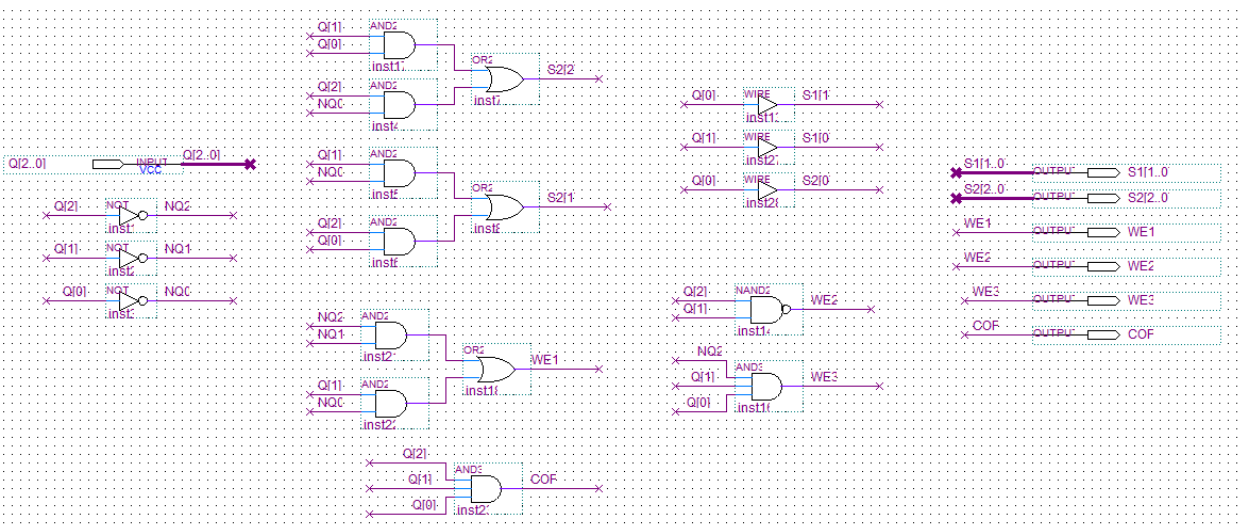
Hình 12.8: Khối Next State.

12.3.3. Khối Current State:



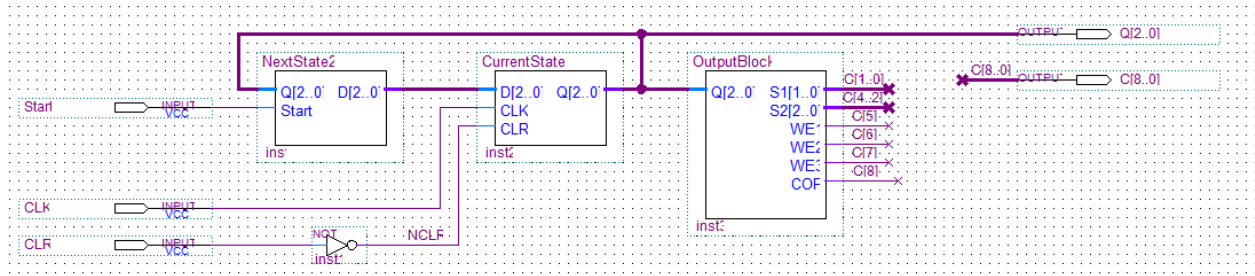
Hình 12.9: Khối Current State.

12.3.4. Khối tín hiệu Output cho thiết kế Graph partitioning:



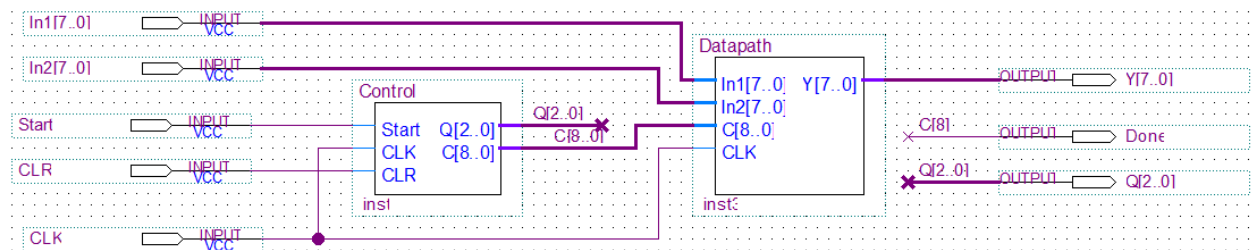
Hình 12.10: Khối Output.

12.3.5. Hoàn thiện khối Controller:



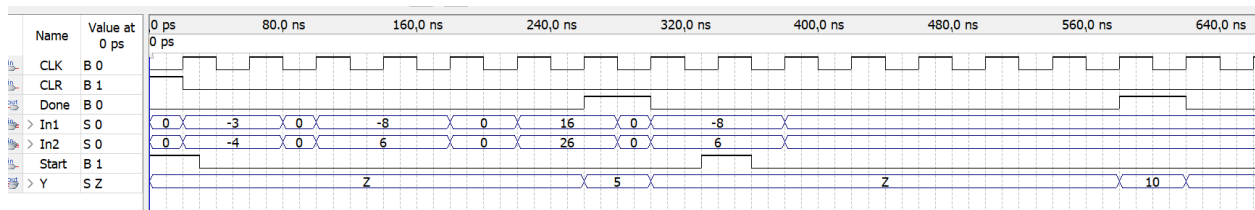
Hình 12.11: Khối Control.

12.4. Thiết kế Hoàn thiện:



Hình 12.11: Thiết kế tổng.

12.5. Waveform:



Hình 12.12: Waveform.

Giải thích:

- Chu kỳ đầu, CLR = 1 để reset mạch.
- Bật Start = 1, cho phép nhập input vào với A = -3, B = -4. Sau đó mạch thực hiện các phép tính để tính ra kết quả $\sqrt{a^2 + b^2} = 5$ khi Done = 1.

- Sau đó bật Start = 1, để nhập input khác vào với A = -8, B = 6. Mạch tính

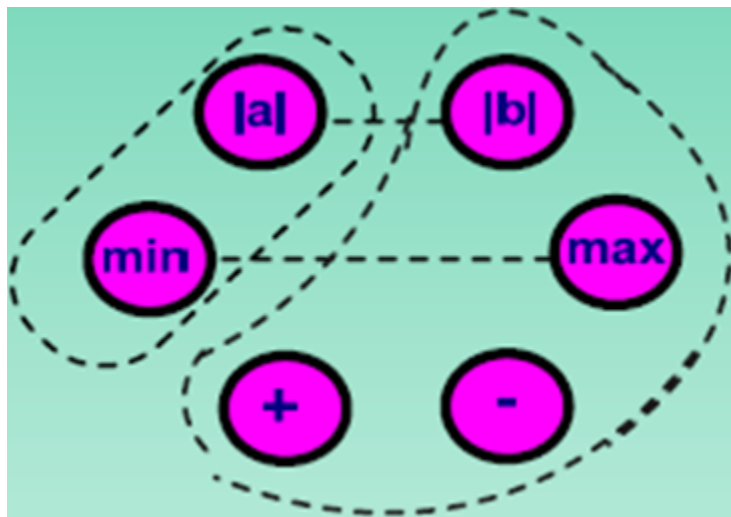
$$\sqrt{a^2 + b^2} = 10 \text{ khi Done} = 1.$$

Bài 13. Datapath connectivity

13.1. Tổng quan thiết kế:

Với mục tiêu thiết kế tối ưu mạch thực hiện bài toán tính giá trị xấp xỉ căn bậc hai tổng bình phương hai số nguyên có dấu a và b sử dụng thuật toán đã trình bày ở các mục trên.

Từ bảng sử dụng các phép toán qua từng trạng thái, ta thấy rằng có một số bộ tính toán không được thực thi đồng thời, điều đó có nghĩa rằng ta có thể gom một số khối chức năng với nhau, sử dụng một số cổng logic để điều khiển nhằm tối ưu hóa diện tích, số cổng luận lý và số khối chức năng trong đường dữ liệu. Việc hiện thực những điều trên gọi là chia sẻ khối chức năng.



Hình 13.1. Merging alternative (simple merging operator)

Sau khi thực hiện gom nhóm các giá trị lưu thành ghi và gom nhóm các khối tính toán ta có kết quả:

$$R1 = [a, t1, x, t7]$$

$$R2 = [b, t2, y, t3, t5, t6]$$

$$R3 = [t4]$$

AU1 = [Abs, Max, +, -]

AU2 = [Abs, Min]

SH1 = [$\gg 1$]

SH2 = [$\gg 3$]

Ta xây dựng mạch cho thiết kế với khối điều khiển Controller và khối đường dữ liệu Datapath bao gồm:

Đầu vào:

- + A[7..0], B[7..0]: hai đầu vào với độ rộng 8 bit.
- + CLK: đầu vào xung clock cho mạch.
- + CLR: tín hiệu clear các thanh ghi.
- + Start: tín hiệu bắt đầu hoạt động của mạch.

Đầu ra:

- + RESULT[7..0]: kết quả của mạch.
- + DONE: tín hiệu báo valid output.

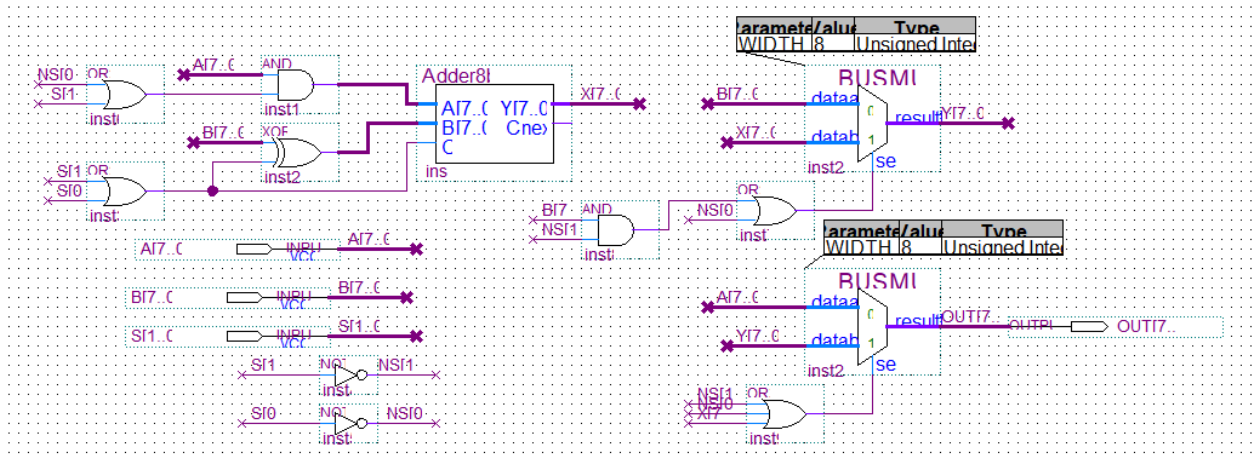
13.2. Thiết kế Datapath:

13.2.1. Khối AU1: Add/Abs/Sub/Max

Tín hiệu điều khiển:

Opcode - S[1..0]	Functions
00	Add
01	Abs
10	Sub
11	Max

Thiết kế:



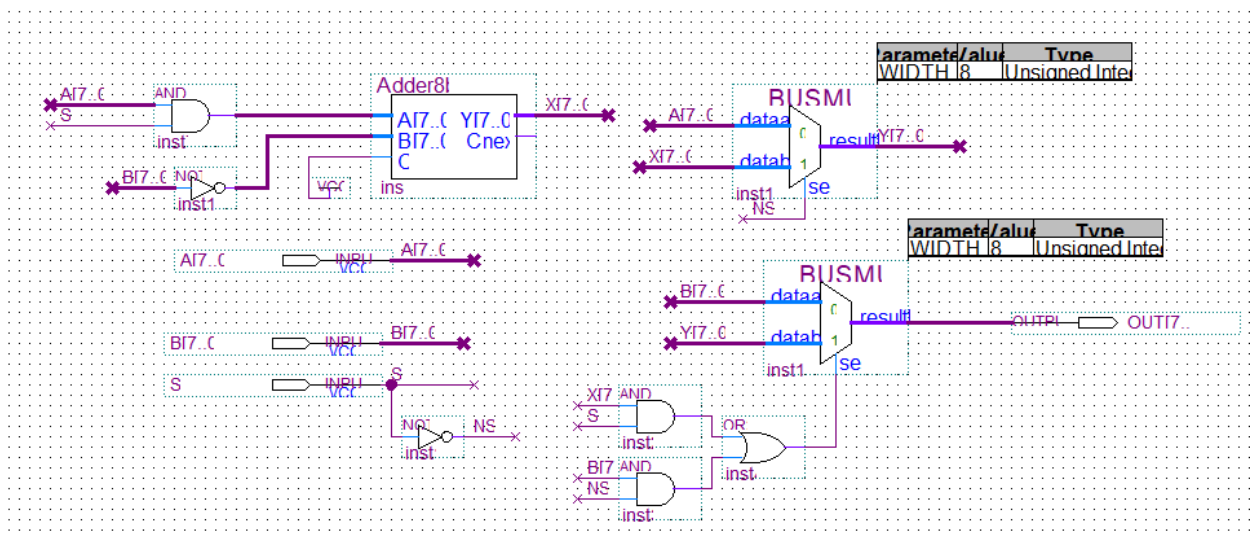
Hình 13.2: Thiết kế khối AU1.

13.2.2. Khối AU2: Abs/Min

Tín hiệu điều khiển:

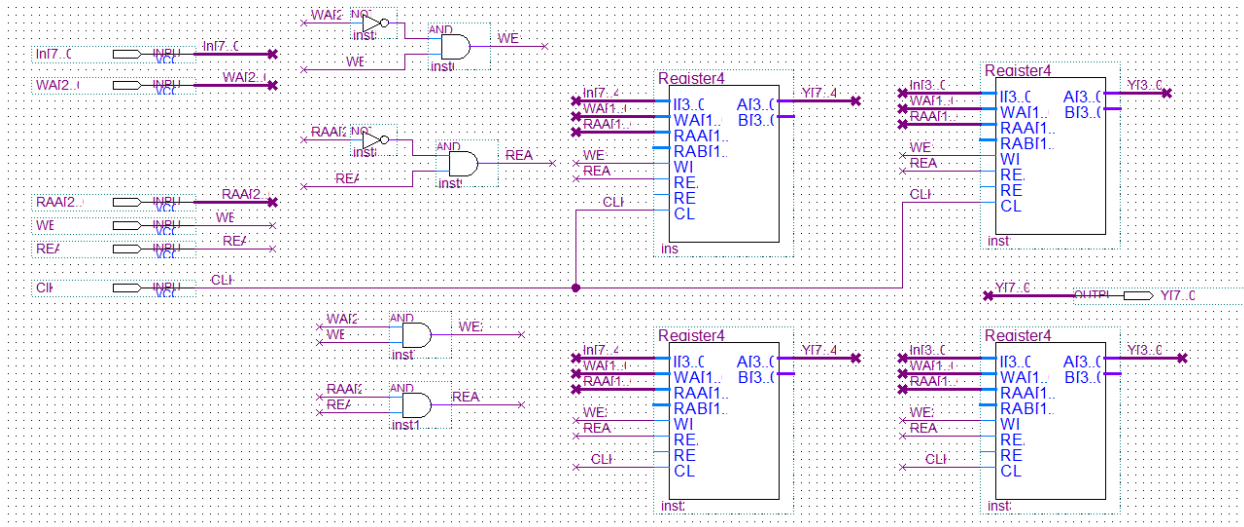
Opcode - S	Functions
0	Abs
1	Min

Thiết kế:



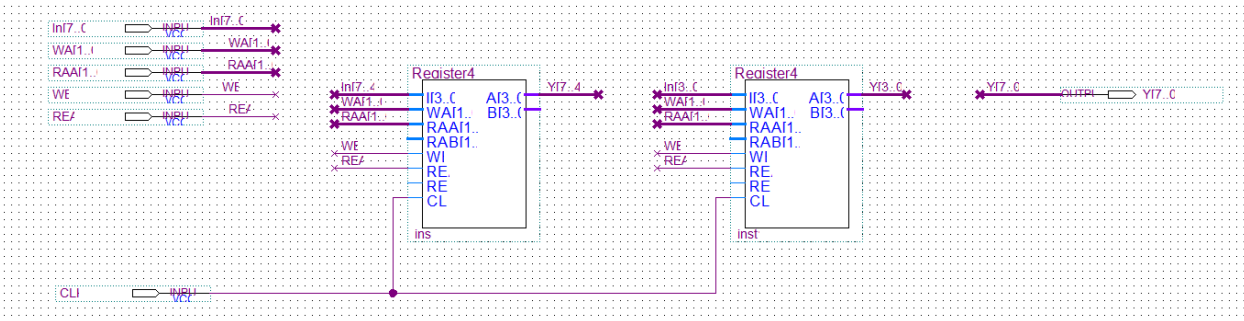
Hình 13.3: Thiết kế khối AU2.

13.2.3. Khối Register 8x8:



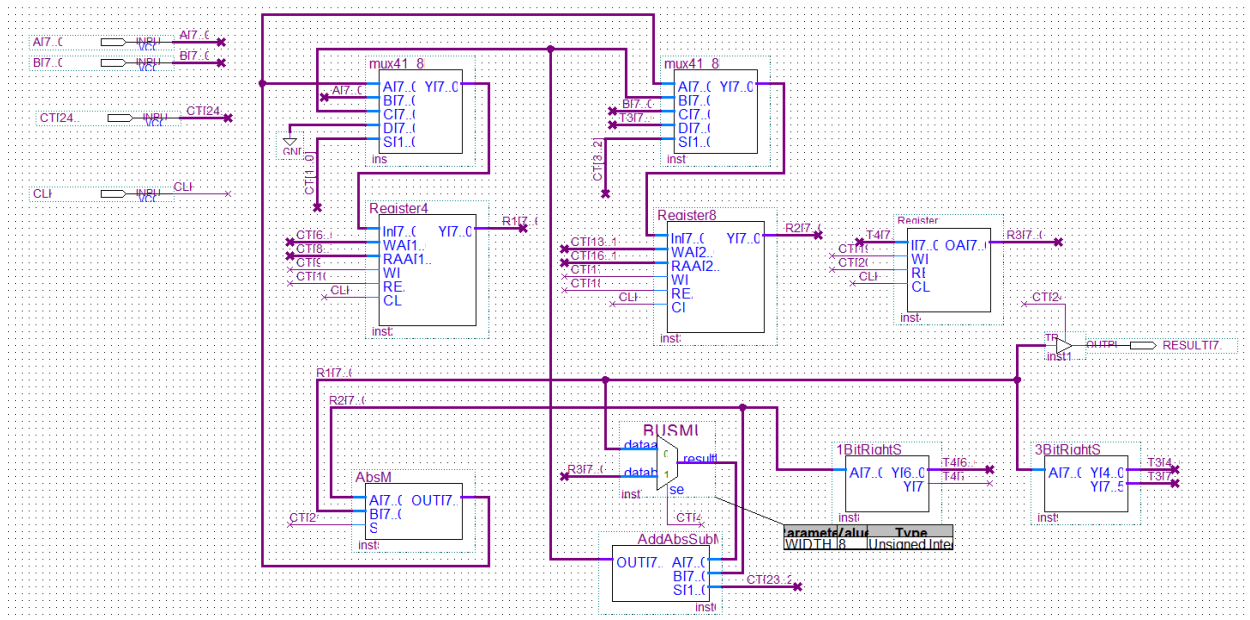
Hình 13.4: Thiết kế Register 8x8.

13.2.4. Khối Register 4x8: từ Register 4x4



Hình 13.5: Thiết kế Register 4x8.

13.2.5. Hoàn thiện Datapath:



Hình 13.6: Thiết kế Datapath.

13.3. Thiết kế Controller:

13.3.1. Bảng trạng thái và các tín hiệu điều khiển:

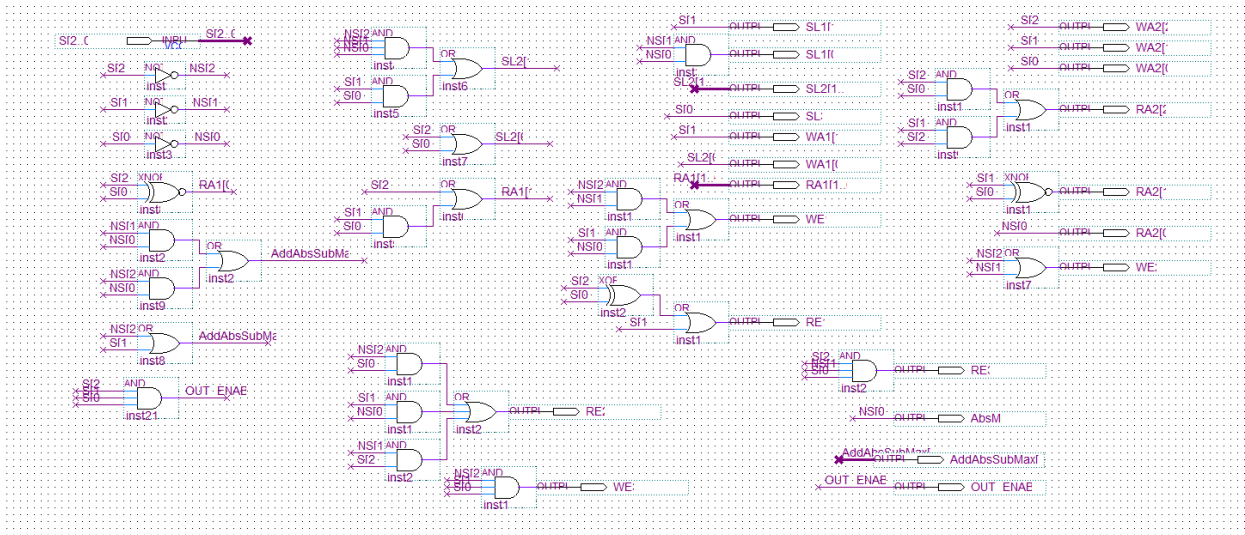
\

Present State	Next State	SL 1	SL 2	SL 3	W A1	W A2	W A3	RA 1	R A2	R A3	W E1	W E2	W E3	R E1	R E2	R E3	AU 2	AU 1	Output Enable
S0	S1	1	2	x	0	0	x	x	x	x	1	1	0	0	0	0	x	x	0
S1	S2	0	1	x	1	1	x	0	0	x	1	1	0	1	1	0	0	1	0
S2	S3	2	0	0	2	2	x	1	1	x	1	1	0	1	1	0	1	3	0
S3	S4	x	3	x	x	3	0	2	2	x	0	1	1	1	1	0	x	x	0

S4	S5	x	1	0	x	4	x	2	3	x	0	1	0	1	1	0	x	2	0
S5	S6	x	1	1	x	5	x	x	4	0	0	1	0	0	1	1	x	0	0
S6	S7	2	x	0	3	x	x	2	5	x	1	0	0	1	1	0	x	1	0
S7	S0	X	X	X	X	X	X	3	X	X	0	0	0	1	0	0	X	X	1

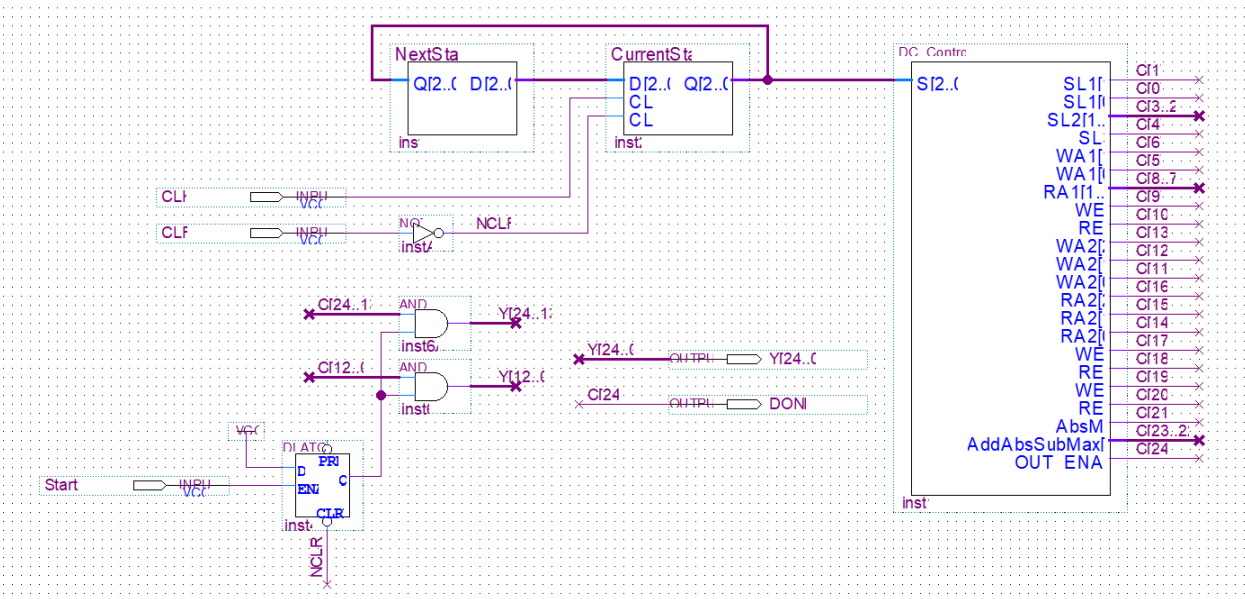
- Tại trạng thái S0: Ta cho phép ghi dữ liệu từ input vào R1 và R2 (điều khiển mux_R1 và mux_R2 trước R1 và R2 để chọn dữ liệu từ input1 và input2) và đợi giá trị Start để chuyển sang trạng thái S1.
- Tại trạng thái S1: ta đưa giá trị từ R1 vào AU2 để tính Abs và điều khiển mux_R1 để cho phép ghi dữ liệu từ AU2 vào R1. Đồng thời đưa giá trị từ R2 vào AU1 để tính Abs (vì phép tính Abs nên ta không cần quan tâm đầu vào còn lại của AU1) và điều khiển mux_R2 để cho phép ghi dữ liệu AU1 vào R2.
- Tại trạng thái S2: cho phép giá trị từ R1 và R2 vào AU2 (vì ở 2 bus khác nhau nên không sợ xung đột) và điều khiển mux_R2 để cho phép lưu giá trị Min(R1, R2) vào R2. Đồng thời điều khiển mux_AU1 để chọn giá trị từ R1 vào AU1 để tính Max(R1, R2) và điều khiển mux_R1 để cho phép lưu giá trị vào R1.
- Tại trạng thái S3: điều khiển mux_R2 để lưu giá trị SHIFT3 vào R2 và lưu giá trị SHIFT1 vào R3.
- Tại trạng thái S4: điều khiển mux_AU1 để chọn giá trị từ R1 vào AU1 để thực hiện tính Sub(R1, R2) và điều khiển mux_R2 để lưu giá trị Sub vào R2.
- Tại trạng thái S5: điều khiển mux_AU1 để chọn giá trị từ R3 vào AU1 để thực hiện tính Add(R3, R2) và điều khiển mux_R2 để lưu giá trị Add vào R2.
- Tại trạng thái S6: điều khiển mux_AU1 để chọn giá trị từ R1 vào AU1 để thực hiện tính Max(R1, R2) và điều khiển mux_R1 để lưu giá trị Max vào R1.
- Tại trạng thái S7: ta bật tín hiệu Done thông báo hoàn thành tính toán và TRI là tín hiệu cho phép xuất giá trị Output từ thanh ghi R1.

13.3.2. Khối tín hiệu Output cho thiết kế Datapath connectivity:



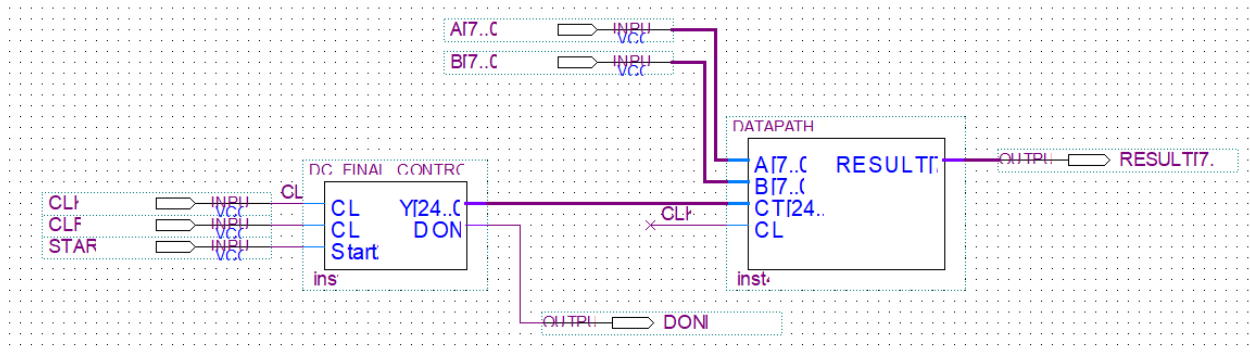
Hình 13.7: Khối Output.

13.3.3. Hoàn thiện khối Controller:



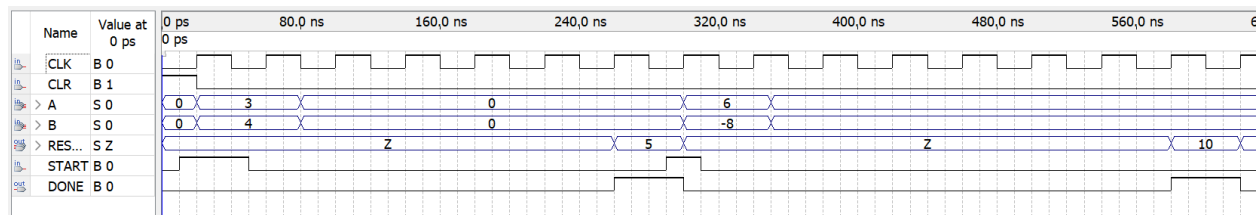
Hình 13.8: Khối Control.

13.4. Thiết kế Hoàn thiện:



Hình 13.9: Thiết kế tổng.

13.5. Waveform:



Hình 13.10: Waveform.

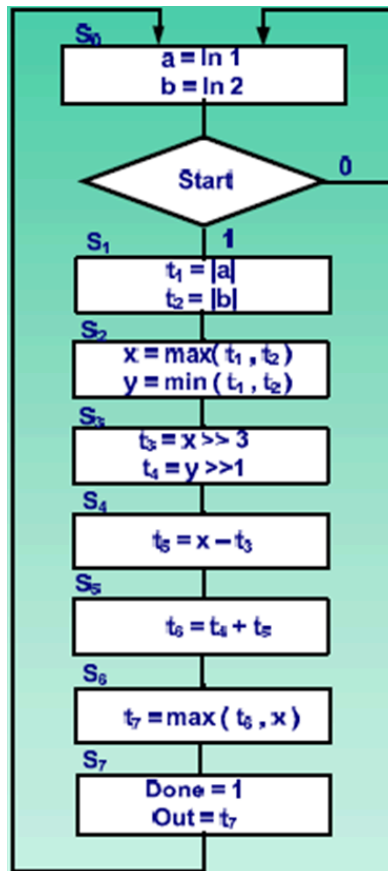
Giải thích:

- Chu kỳ đầu, CLR = 1 để reset mạch.
- Bật Start = 1, cho phép nhập input vào với A = 3, B = 4. Sau đó mạch thực hiện các phép tính để tính ra kết quả $\sqrt{a^2 + b^2} = 5$ khi Done = 1.
- Sau đó bật Start = 1, để nhập input khác vào với A = 6, B = -8. Mạch tính $\sqrt{a^2 + b^2} = 10$ khi Done = 1.

Bài 14. SRA Datapath generated by prioritized partitioning

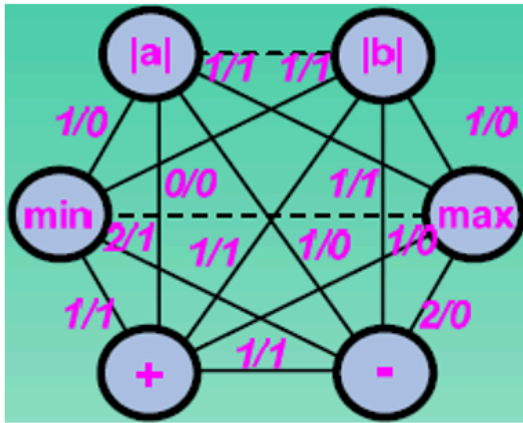
14.1. Tổng quan thiết kế:

ASM Chart:

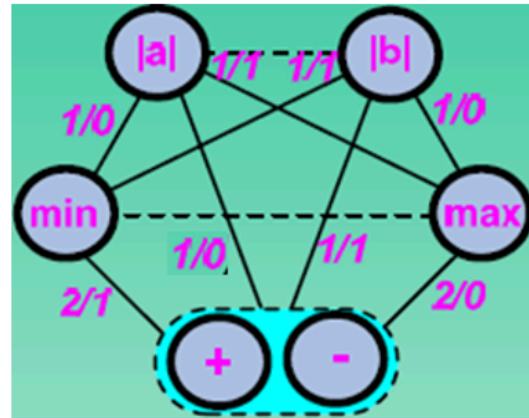


Hình 14.1: ASM Chart.

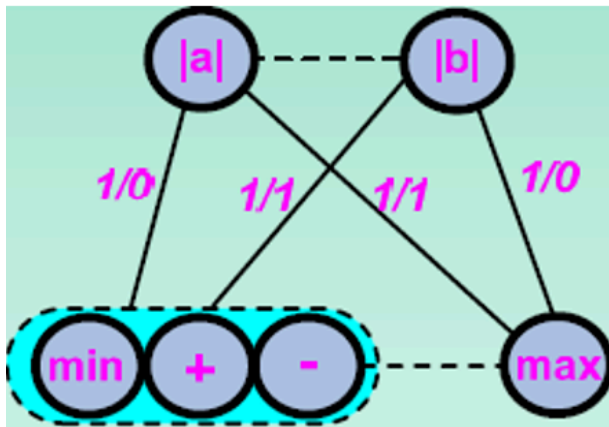
Đối với nội dung bài tập này thực hiện chia sẻ khối chức năng sử dụng giải thuật Graph Partitioning.



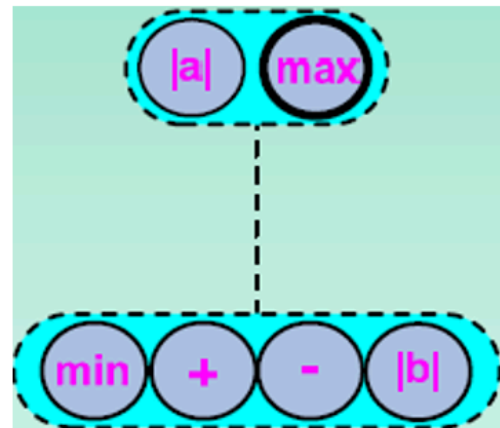
(a) Compatibility graph



(b) Compatibility graph after merging of + and -
(operator's similarity grouping)



(c) Compatibility graph after merging of min, + and -
(datapath's connection grouping)



(d) Final graph partitions
(datapath's connection grouping)

Hình 14.2: Giải thuật Graph Partitioning thiết kế Functional Unit Sharing.

Sau khi thực hiện giải thuật Graph Partitioning, thực hiện gom nhóm các giá trị lưu thành ghi và gom nhóm các khối tính toán ta có kết quả:

$$R1 = [a, t1, x, t7]$$

$$R2 = [b, t2, y, t3, t5, t6]$$

$$R3 = [t4]$$

$$AU1 = [Abs, Max]$$

$$AU2 = [Abs, +, -, Min]$$

$$SH1 = [>1]$$

SH2 = [$\gg 3$]

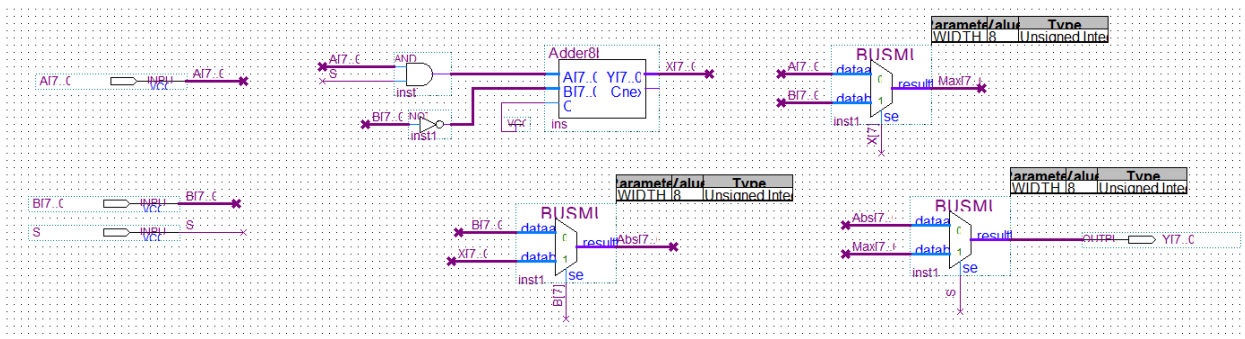
14.2. Thiết kế Datapath:

14.2.1. Khối AU1: Abs/Max

Tín hiệu điều khiển:

Opcode - S	Functions
0	Abs
1	Max

Thiết kế:



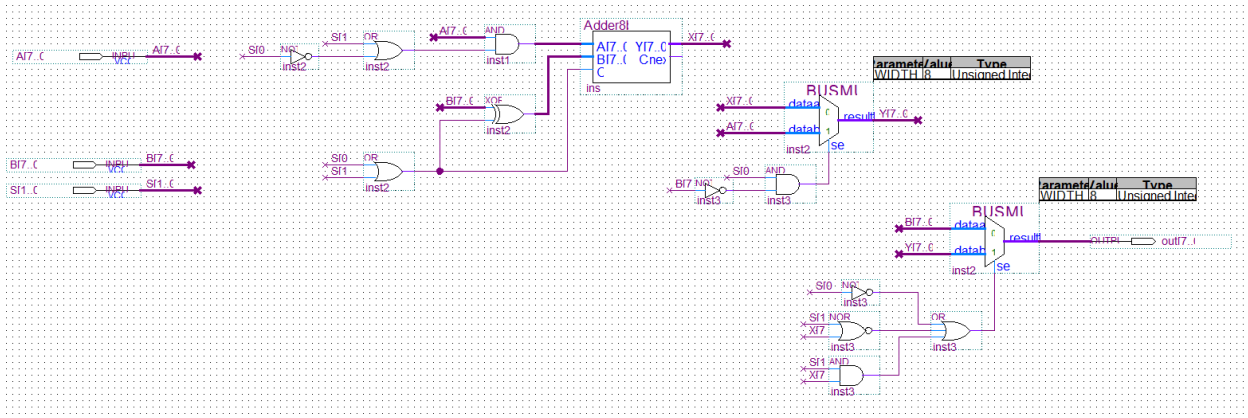
Hình 14.3: Thiết kế khối AU1.

14.2.2. Khối AU2: Add/Abs/Sub/Min

Tín hiệu điều khiển:

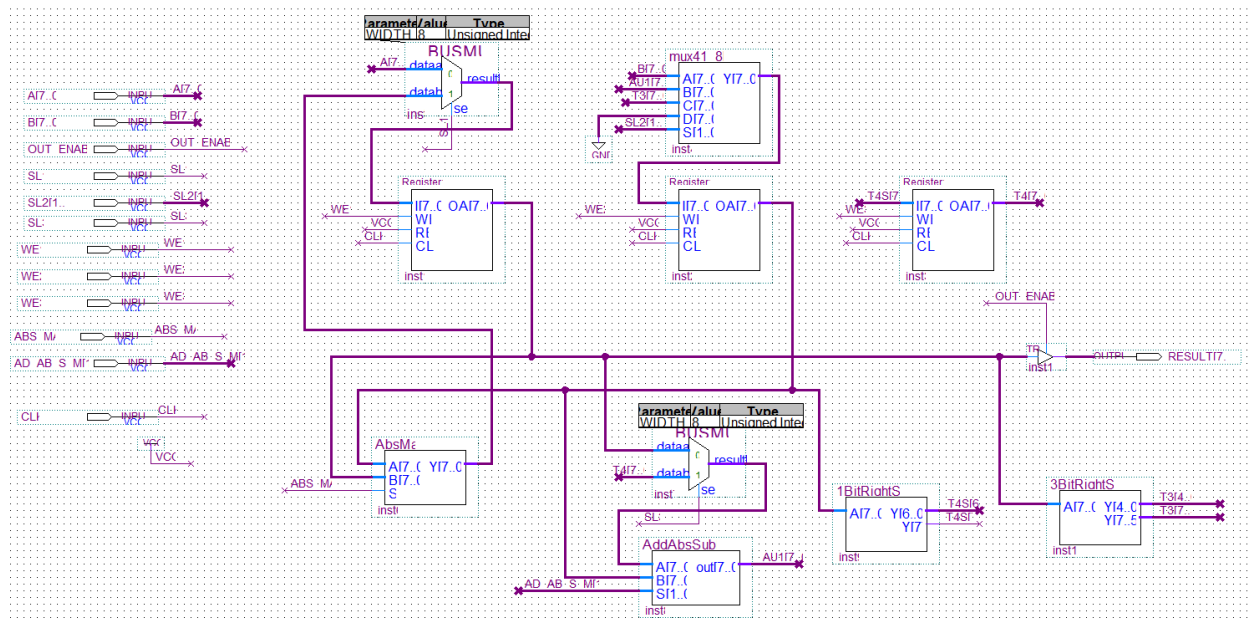
Opcode - S[1..0]	Functions
00	Add
01	Abs
10	Sub
11	Min

Thiết kế:



Hình 14.4: Thiết kế khối AU2

14.2.3. Hoàn thiện Datapath:



Hình 14.5: Thiết kế khối Datapath.

14.3. Thiết kế Controller:

Bảng trạng thái và các tín hiệu điều khiển:

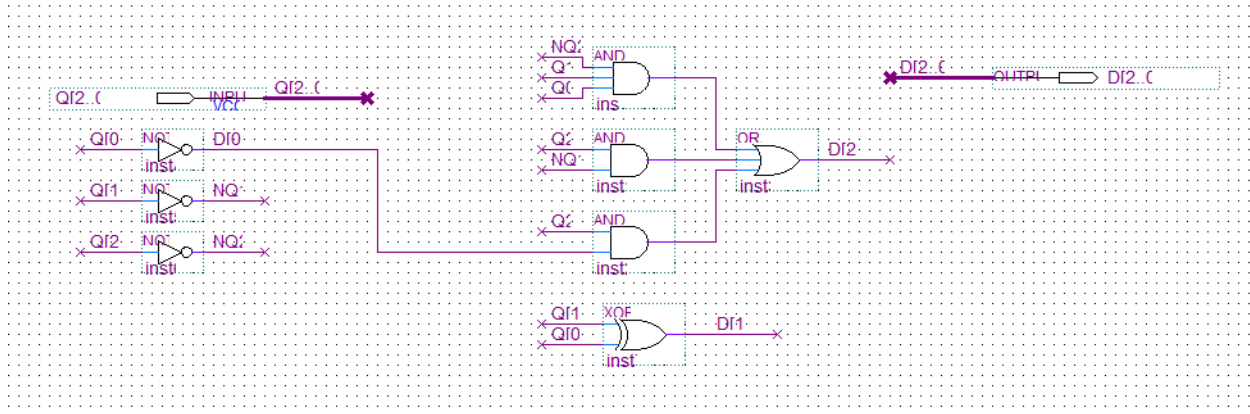
Present State	SL1	SL2[1..0]	SL3	WE[3..1]	AU1	AU2
S0	0	0	0	X	X	X

S1	1	1	0	X	X	1
S2	1	1	0	X	1	3
S3	0	2	1	0	0	0
S4	0	1	0	X	0	1
S5	0	1	1	X	0	0
S6	1	0	0	X	1	0
S7	x	X	x	X	X	X

- Tại trạng thái 0: bật tín hiệu WE1 và WE2 và điều khiển tín hiệu mux để ghi giá trị input vào R1 và R2.
- Tại trạng thái 1: bật tín hiệu WE1 và WE2 và điều khiển tín hiệu mux để ghi giá trị vào R1 và R2. Ở đây tín hiệu điều khiển $AU1 = 0$ và $AU2 = 1$.
- Tại trạng thái 2: bật tín hiệu WE1 và WE2 và điều khiển tín hiệu mux để ghi giá trị vào R1 và R2. Ở đây tín hiệu điều khiển $AU1 = 1$ và $AU2 = 3$ và điều khiển tín hiệu mux chọn đầu vào cho AU2 chọn giá trị của R1.
- Tại trạng thái 3: bật tín hiệu WE2 và WE3 và điều khiển tín hiệu mux để ghi giá trị vào R2 và R3
- Tại trạng thái 4: điều khiển tín hiệu mux chọn R1 làm đầu vào cho AU2, ở đây tín hiệu điều khiển $AU2 = 2$. Bật tín hiệu WE2 và điều khiển tín hiệu mux để ghi giá trị vào R2.
- Tại trạng thái 5: điều khiển tín hiệu mux chọn R3 làm đầu vào cho AU2, ở đây tín hiệu điều khiển $AU2 = 0$. Bật tín hiệu WE2 và điều khiển tín hiệu mux để ghi giá trị vào R2.
- Tại trạng thái 6: Bật tín hiệu WE2 và điều khiển tín hiệu mux để ghi giá trị vào R1. Ở đây tín hiệu điều khiển $AU1 = 1$.

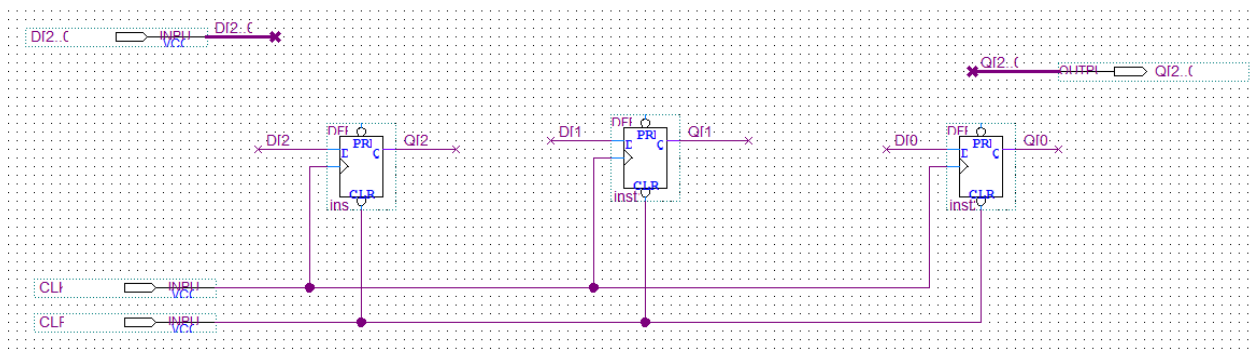
- Tại trạng thái 7: bật tín hiệu Done thông báo hoàn thành tính toán và TRI là tín hiệu cho phép xuất giá trị Output từ thanh ghi R1.

14.3.1. Thiết kế trạng thái kế tiếp:



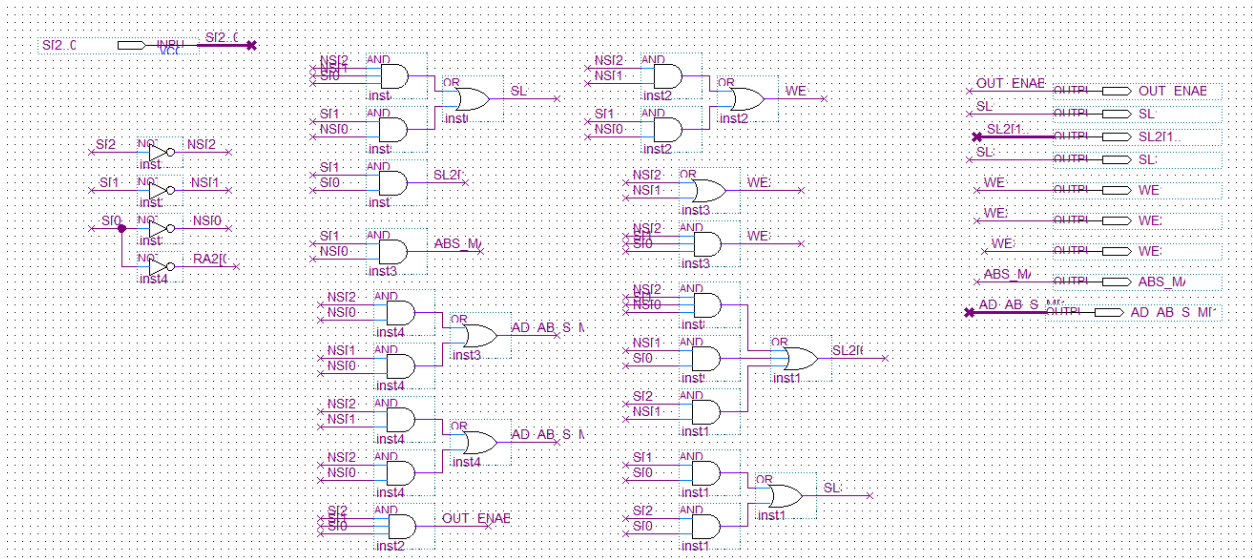
Hình 14.6: Thiết kế khối NEXTSTATE

14.3.2. Thiết kế trạng thái hiện tại:



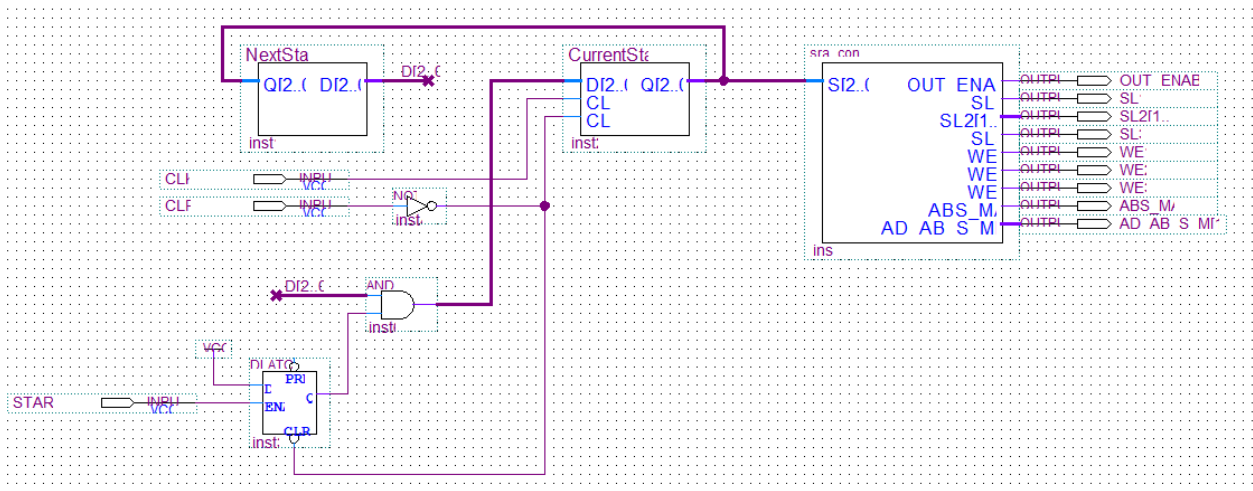
Hình 14.7: Thiết kế khối CURRENTSTATE

14.3.3. Khối tín hiệu Output cho thiết kế SRA Datapath:



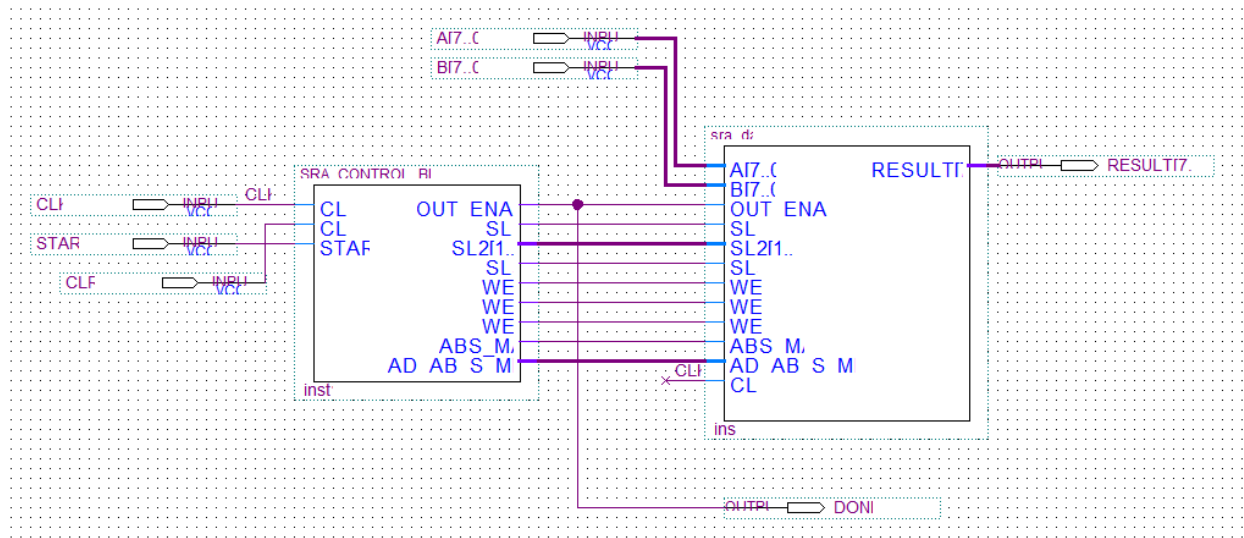
Hình 14.8: Thiết kế khối Output cho SRA Datapath.

14.3.4. Hoàn thiện Controller:



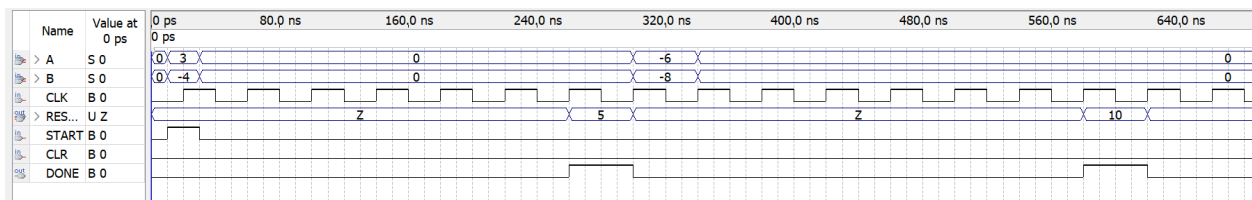
Hình 14.9: Thiết kế khối Controller.

14.4. Thiết kế Hoàn thiện:



Hình 14.10: Thiết kế hoàn chỉnh mạch SRA Datapath

14.5. Waveform:



Hình 14.11: Waveform SRA Datapath.

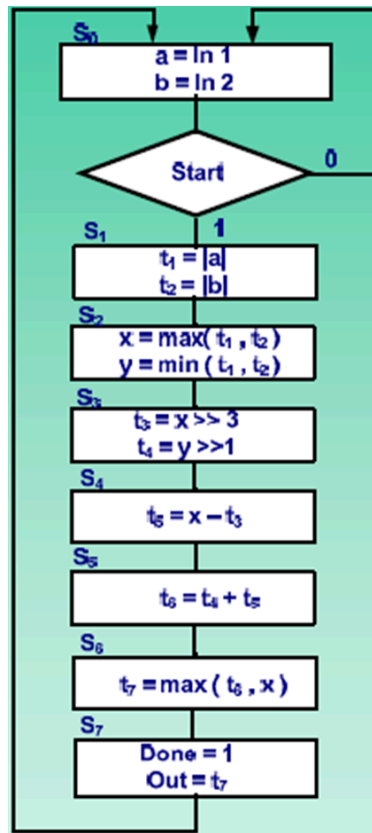
Giải thích:

- Bật Start = 1, cho phép nhập input vào với A = 3, B = -4. Sau đó mạch thực hiện các phép tính để tính ra kết quả $\sqrt{a^2 + b^2} = 5$ và Done = 1.
- Sau đó bật Start = 1, để nhập input khác vào với A = -6, B = -8. Mạch tính $\sqrt{a^2 + b^2} = 10$ khi Done = 1.

Bài 15. Register merging

15.1. Tổng quan thiết kế:

ASM Chart:



Hình 15.1: ASM Chart.

Input:

- + Hai biến dữ liệu cần tính toán.
- + Tín hiệu **Clock (CLK)**.
- + Tín hiệu điều khiển từ **Control**.

Output:

- + Kết quả của Datapath ($\sqrt{a^2 + b^2}$).

Register Merging là kỹ thuật tối ưu hóa trong thiết kế phần cứng nhằm giảm số lượng **register** (thanh ghi) được sử dụng bằng cách kết hợp nhiều biến hoặc giá trị vào cùng một thanh ghi khi chúng không xung đột về thời gian sử dụng. Điều này giúp tiết

$$R3 = [t4]$$

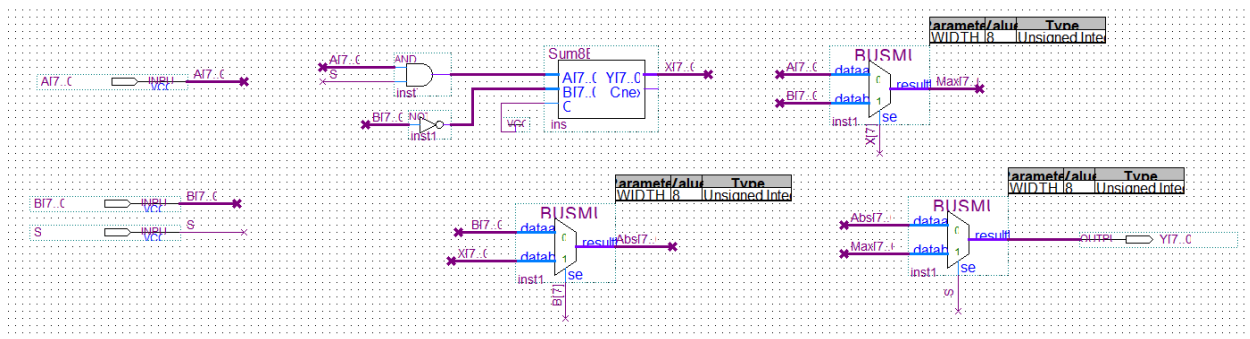
	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
R_1	▶▶	▶▶	▶▶		▶▶		▶▶	▶▶
R_2	▶▶	▶▶	▶▶	▶▶	▶▶	▶▶		
R_3				▶▶		▶▶		

88

15.2. Thiết kế Datapath:

15.2.1. Khối Abs_Max:

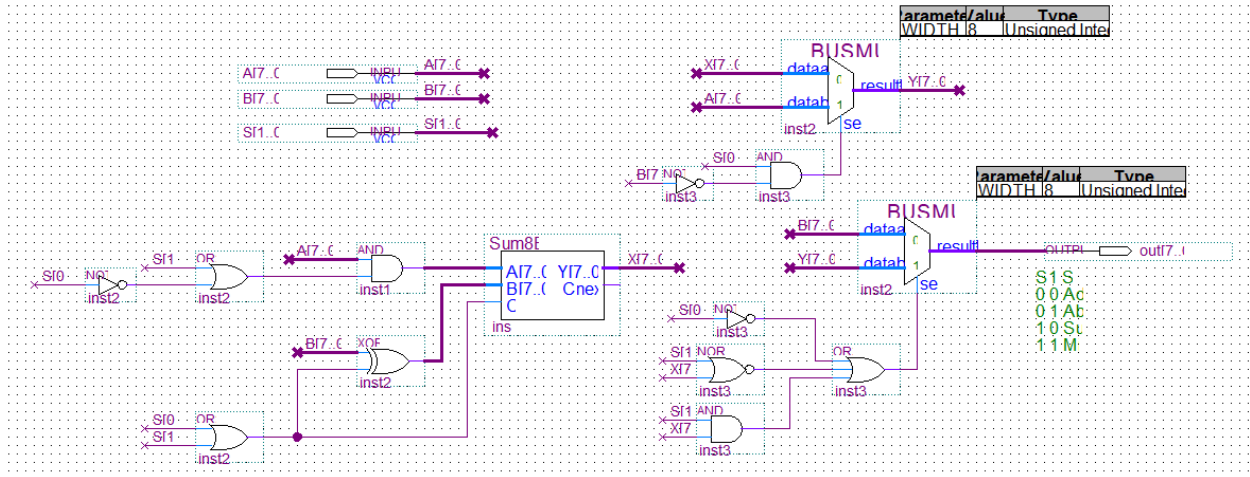
Select (S)	Functions
1	Max
0	Absolute



Hình 15.2: Khối Abs_Max.

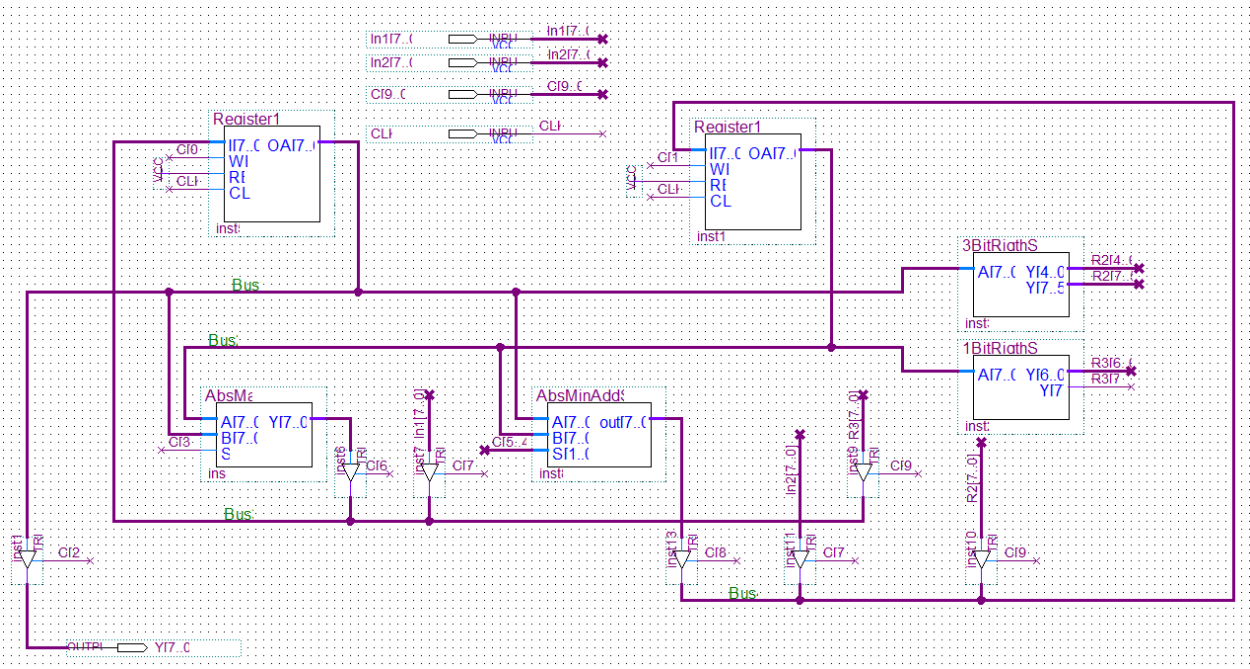
15.2.2. Khối Abs_Min_Add_Sub:

S[1:0]	Functions
00	Add
01	Absolute
10	Sub
11	Min



Hình 15.3: Khối Abs_Min_Add_Sub.

15.2.3. Hoàn thiện Datapath:



Hình 15.4: Datapath.

- **C[0]**: Điều khiển tín hiệu **WE** của **register1**, cho phép hoặc ngừng ghi dữ liệu vào thanh ghi này.
- **C[1]**: Điều khiển tín hiệu **WE** của **register2**, cho phép hoặc ngừng ghi dữ liệu vào thanh ghi này.

- **C[2]:** Điều khiển cổng **Tri-State** để quyết định liệu dữ liệu có thể xuất ra từ hệ thống hay không.
- **C[3]:** Điều khiển chức năng của khối **AbsMax**, xác định cách thức khối này xử lý dữ liệu.
- **C[5:4]:** Điều khiển chức năng của khối **AbsMinAddSubđi**, xác định cách thức khối này xử lý dữ liệu.
- **C[6]:** Điều khiển cổng **Tri-State** để cho phép hoặc ngừng xuất kết quả từ khối **AbsMax** ra ngoài.
- **C[7]:** Điều khiển cổng **Tri-State** của **input1**, cho phép hoặc ngừng việc truyền dữ liệu vào datapath từ input1.
- **C[8]:** Điều khiển cổng **Tri-State** để cho phép hoặc ngừng xuất kết quả từ khối **AbsMinAddSub** đi ra ngoài.
- **C[9]:** Điều khiển cổng **Tri-State** của **input2**, cho phép hoặc ngừng việc truyền dữ liệu vào datapath từ input1.

15.3. Thiết kế Controller:

Input:

- + Tín hiệu Start.
- + Xung clock (CLK)
- + Tín hiệu Reset (CLR)

Output:

- + Tín hiệu Control.

15.3.1. Bảng trạng thái:

State	Current State	Next State	
		Start = 1	Start = 0
	Q[2:0]	D[2:0]	D[2:0]
S0	000	001	000
S1	001	010	010
S2	010	011	011
S3	011	100	100
S4	100	101	101
S5	101	110	110
S6	110	111	111
S7	111	000	000

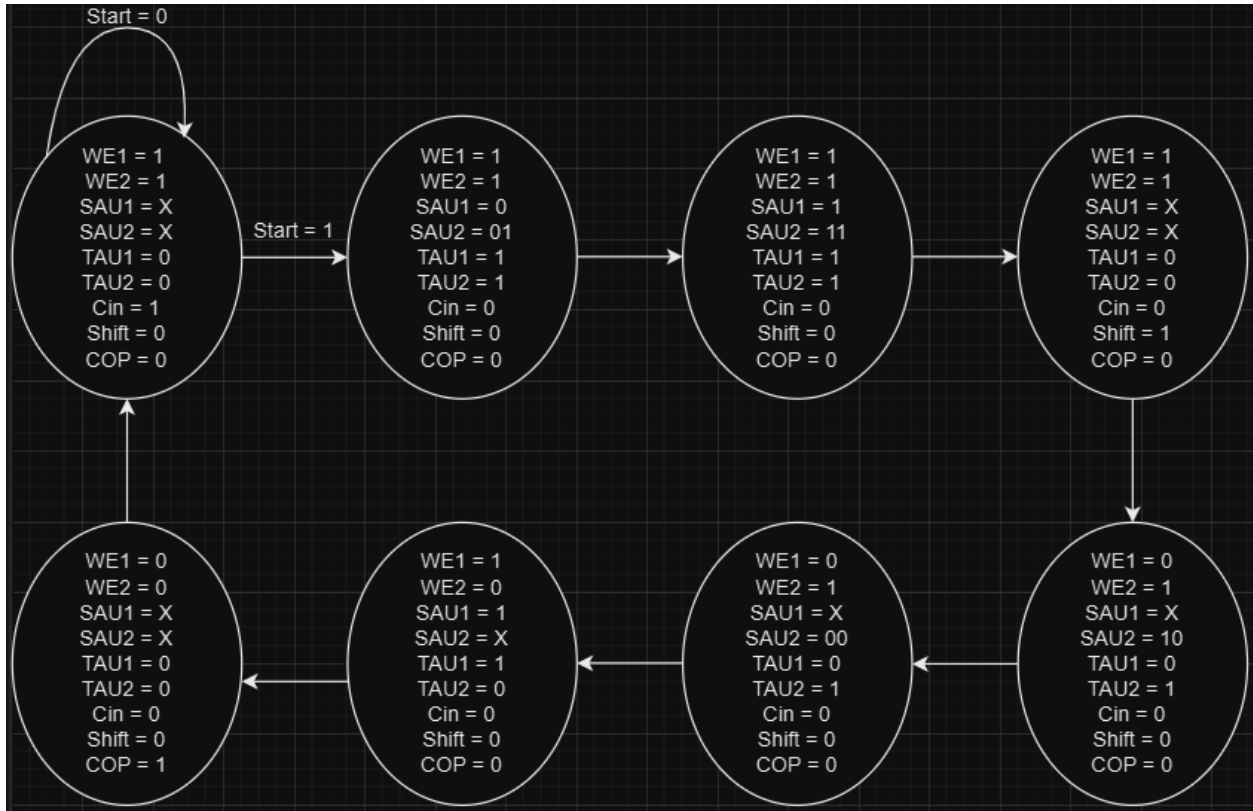
Hình 15.5: Bảng trạng thái kế tiếp

15.3.2. Tín hiệu Control:

State	WE1	WE2	SAU1	SAU2	TAU1	TAU2	Cin	Shift	COP
S0	1	1	X	X	0	0	1	0	0
S1	1	1	0	01	1	1	0	0	0
S2	1	1	1	11	1	1	0	0	0
S3	1	1	X	X	0	0	0	1	0
S4	0	1	X	10	0	1	0	0	0
S5	0	1	X	00	0	1	0	0	0
S6	1	0	1	X	1	0	0	0	0
S7	0	0	X	X	0	0	0	0	1

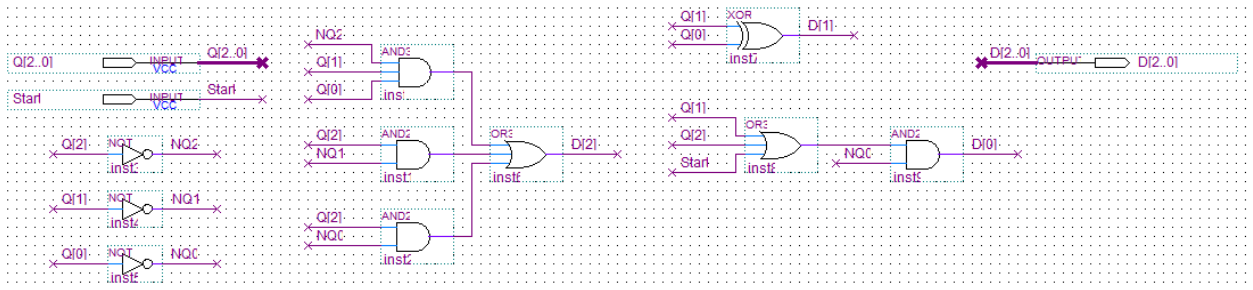
Hình 15.6: bảng tín hiệu Control ứng với mỗi trạng thái.

Sơ đồ chuyển trạng thái:



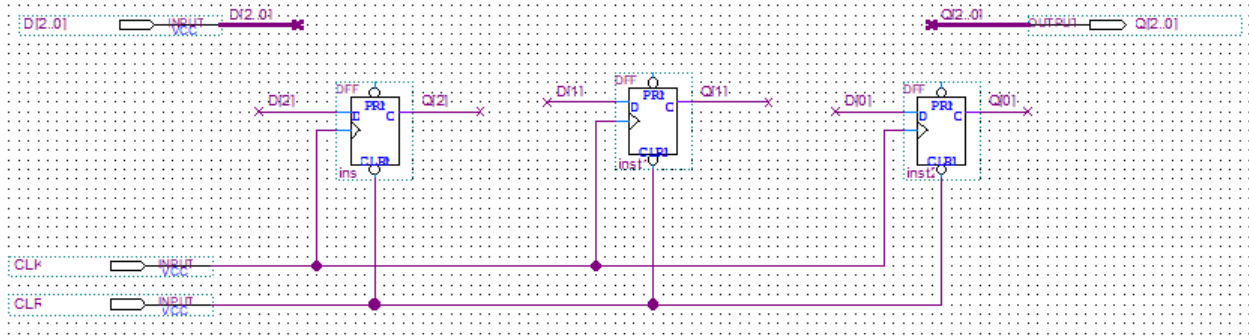
Hình 15.7: Sơ đồ chuyển trạng thái.

15.3.3. Khối NextState:



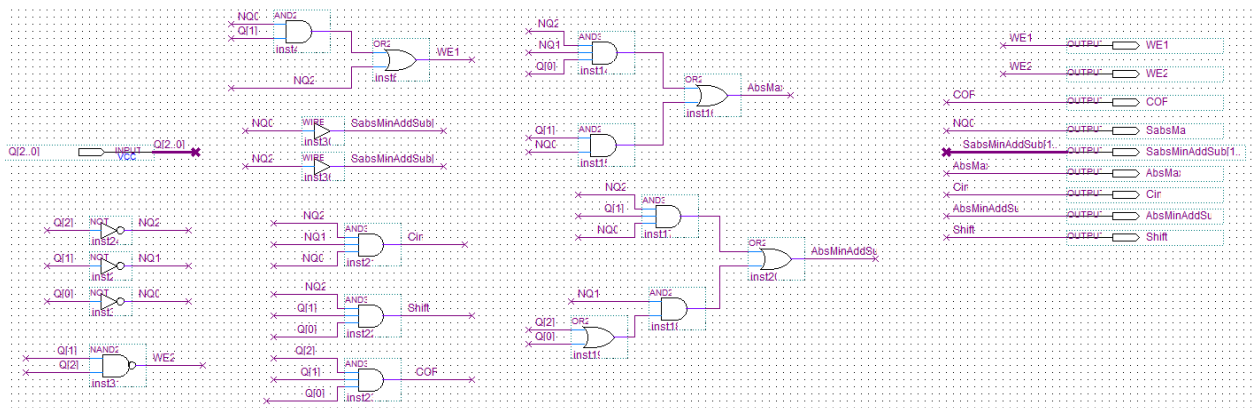
Hình 15.8: Khối Next State.

15.3.4. Khối CurrentState:



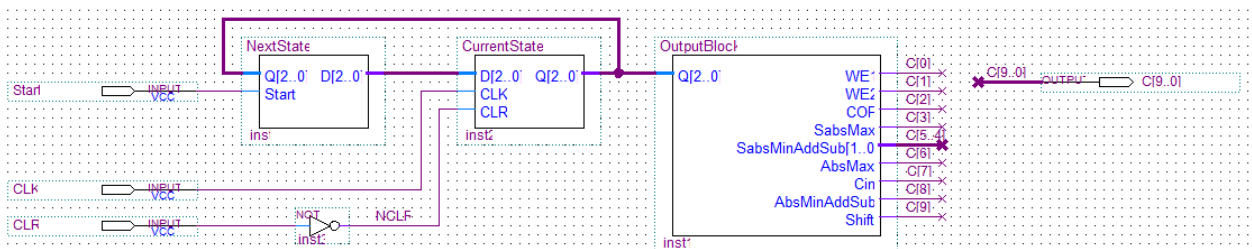
Hình 15.9: Khối Current State.

15.3.5. Khối Output:



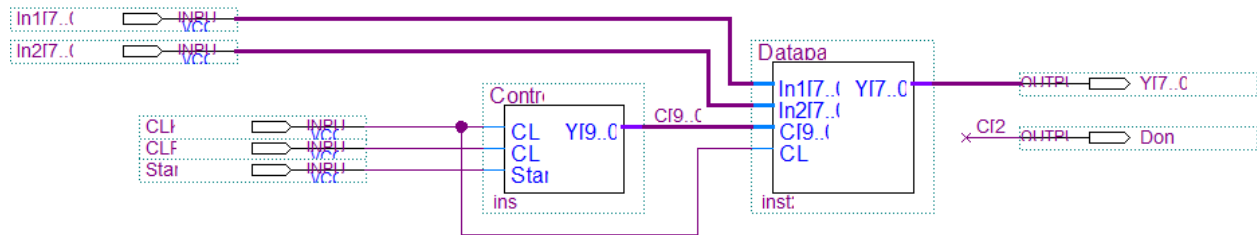
Hình 15.10: Khối Output.

15.3.6. Hoàn thiện Controller:



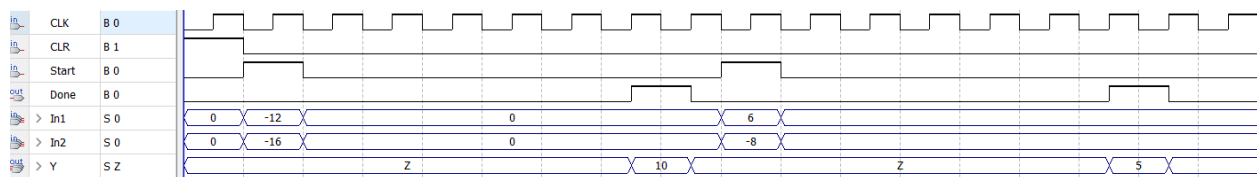
Hình 15.11: Khối Control.

15.4. Thiết kế hoàn thiện:



Hình 15.12: Mạch tổng.

15.5. Waveform:



Hình 15.13: Waveform Register Merging

Giải thích:

- Chu kỳ đầu, CLR = 1 để reset mạch.
- Bật Start = 1, cho phép nhập input vào với A = -12, B = -16. Sau đó mạch thực hiện các phép tính để tính ra kết quả 10 khi Done = 1.
- Sau đó bật Start = 1, để nhập input khác vào với A = 6, B = 8. Mạch tính

$$\sqrt{a^2 + b^2} = 5 \text{ khi Done} = 1.$$

Bài 16. Datapath with pipelined functional unit

16.1. Tổng quan mạch

Datapath được chia thành 2 state bằng các D-flipflop.

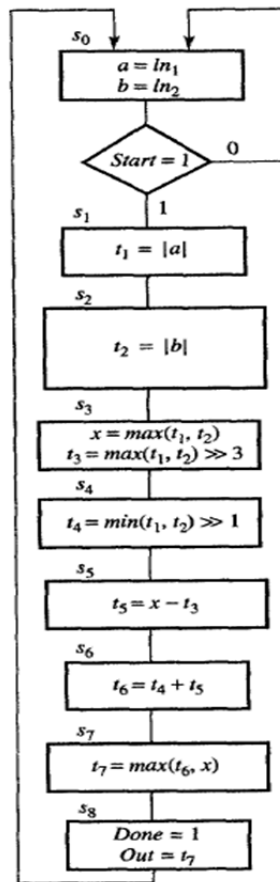
Giữa hai state có các **D-Flip Flop** đóng vai trò là **thanh ghi pipeline**:

- + **Thanh ghi pipeline** lưu trữ dữ liệu trung gian và đồng bộ hóa hoạt động giữa các giai đoạn.
- + Đảm bảo dữ liệu được truyền từ giai đoạn đầu sang giai đoạn sau một cách chính xác theo xung nhịp.

Input: A, B

Output: $\sqrt{(A^2 + B^2)}$

ASM Chart:



Hình 16.1: ASM Chart.

Timing diagram:

	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}
Read R_1		a			t_1	t_1	x				x		t_7
Read R_2			b		t_2	t_2	t_3		t_5		t_6		
Read R_3									t_4				
AU stage 1		a	b		max	min	-		+		max		
AU stage 2			a	b		max	min	-		+		max	
shifters						>>3	>>1						
Write R_1	a		t_1			x						t_7	
Write R_2	b			t_2		t_3		t_5		t_6			
Write R_3							t_4						
Output													t_7

Hình 16.2: Timing diagram.

16.2. Thiết kế Datapath:

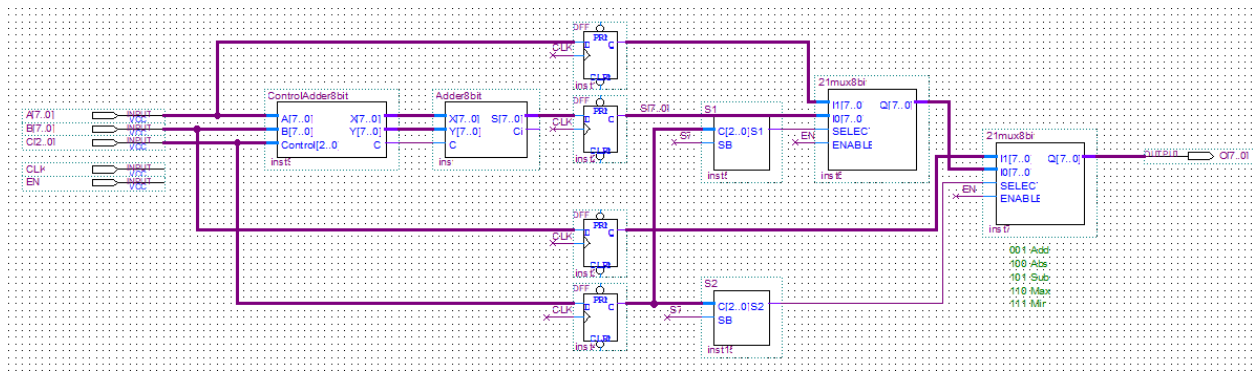
Datapath:

Thiết kế:

- + Gồm 4 khối chính: AU, SR3bit, SR1bit, Register.
- + Các D-flipflop lưu trữ dữ liệu vào và ra của các thao tác.
- + SR3bit và SR1bit thực hiện phép dịch phải 3bit và 1bit.
- + Khối AU thực hiện các phép toán số học và logic.

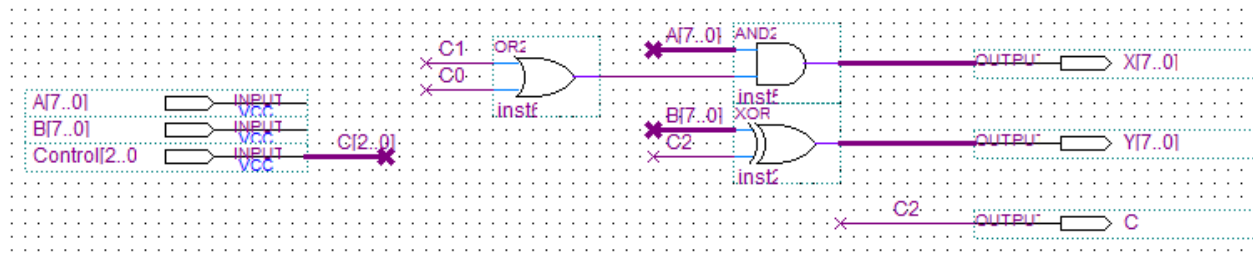
Nguyên lý hoạt động:

- + Bật Enable (EN) của Register nguồn để đưa dữ liệu lên BUS.
- + Dữ liệu trên BUS được truyền đến AU (Dựa vào Opcode) hoặc các khối SR (để dịch bit).
- + Kết quả từ AU hoặc SR được đưa trở lại BUS qua TRI-state buffer.
- + Bật Write Register (WR) của Register đích để ghi kết quả từ BUS vào.
- + Sau khi hoàn tất, tín hiệu Done được bật để sẵn sàng nhận dữ liệu mới.



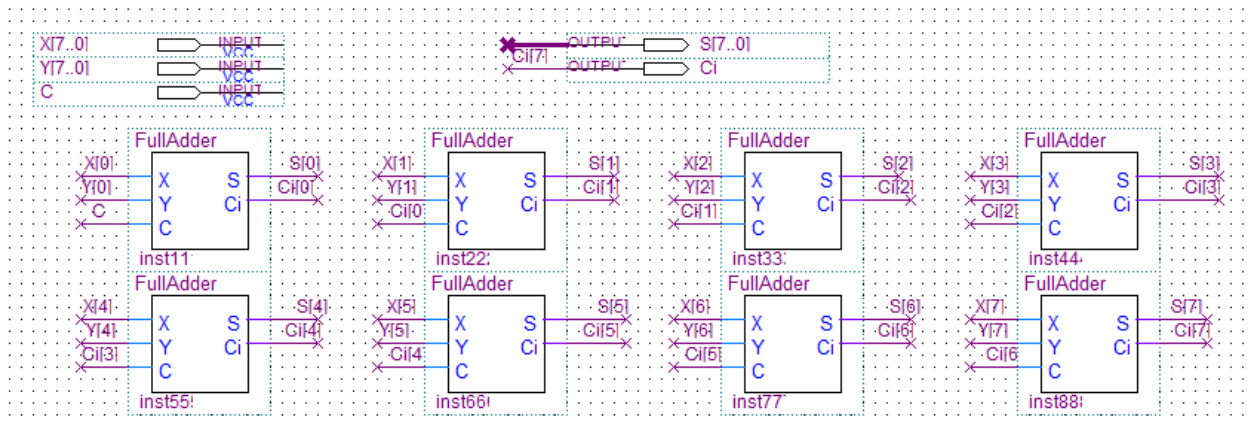
Hình 16.4: Thiết kế khối AU.

16.2.2. Control Adder8bit



Hình 16.5: Thiết kế khối Control Adder8bit.

16.2.3. Adder8bit

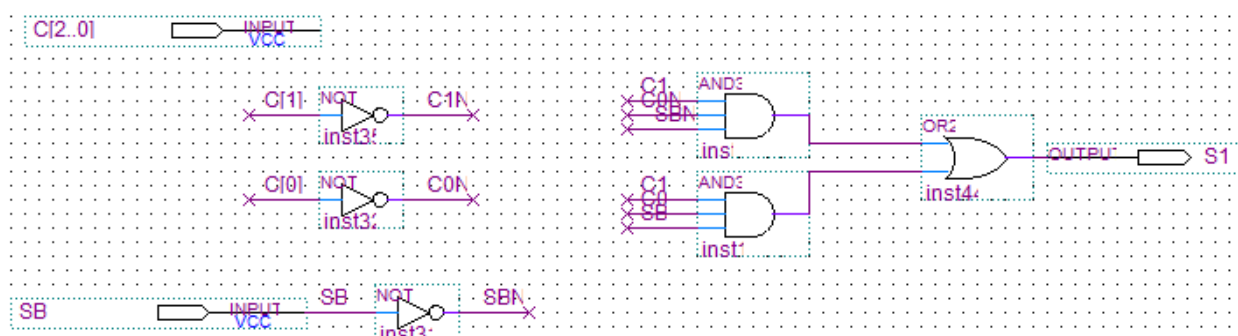


Hình 16.6: Thiết kế khối Adder8bit

16.3. Thiết kế Controller:

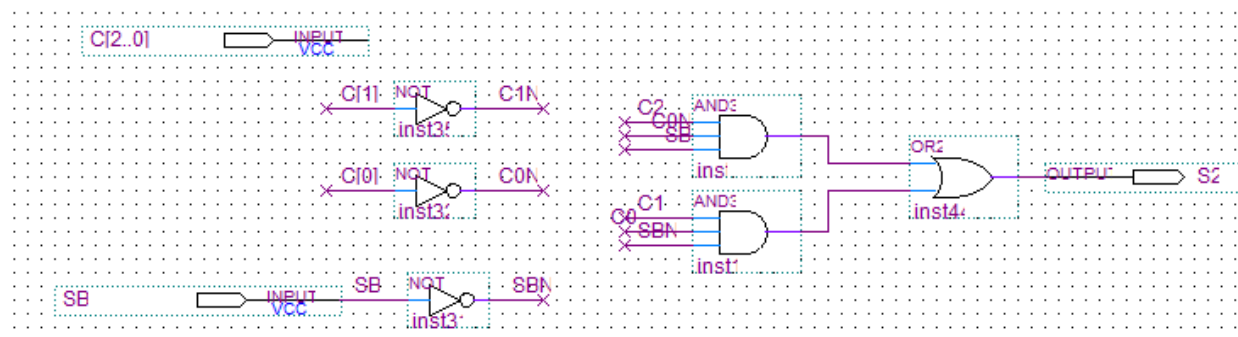
MUX 1			MUX 2		
S1	0	1	S2	0	1
Out	Adder	A	Out	Mux 1	B

16.3.1. Các khối điều khiển MUXS1 (Điều khiển MUX 1)



Hình 16.7: Thiết kế khối S1.

S2 (Điều khiển MUX 2)



Hình 16.8: Thiết kế khối S2.

Controller

Input:

- + Tín hiệu Start
- + Xung clock (CLK)
- + Tín hiệu Reset (CLR)

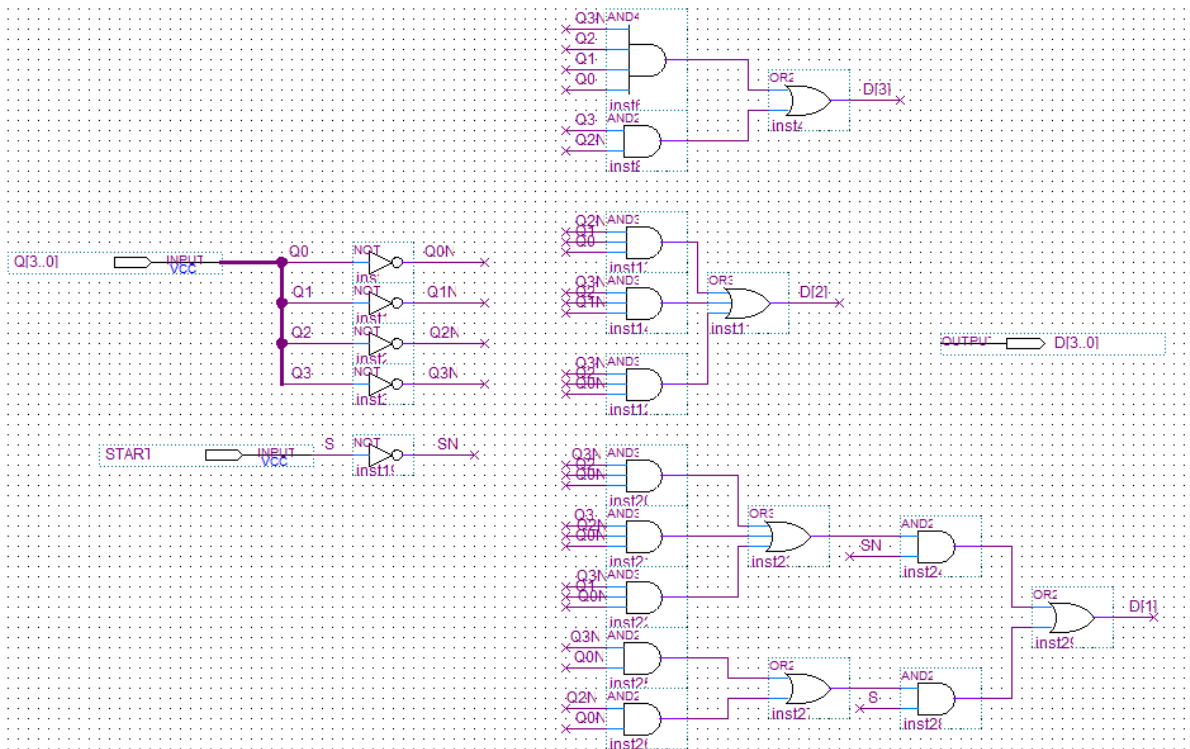
Output: Tín hiệu Control các tín hiệu TRI

16.3.2. Bảng trạng thái Control:

STATE	STATE TABLE											
	CurrentState				NextState							
					Start = 0				Start = 1			
	Q3	Q2	Q1	Q0	D3	D2	D1	D0	D3	D2	D1	D0
0	0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	1	0	0	0	1	0
2	0	0	1	0	0	0	1	1	0	0	1	1
3	0	0	1	1	0	1	0	0	0	1	0	0
4	0	1	0	0	0	1	0	1	0	1	0	1
5	0	1	0	1	0	1	1	0	0	1	1	0
6	0	1	1	0	0	1	1	1	0	1	1	1
7	0	1	1	1	1	0	0	0	1	0	0	0
8	1	0	0	0	1	0	0	1	1	0	0	1
9	1	0	0	1	1	0	1	0	1	0	1	0
10	1	0	1	0	1	0	1	1	1	0	1	1
11	1	0	1	1	1	1	0	0	1	1	0	0
12	1	1	0	0	0	0	0	0	0	0	0	0

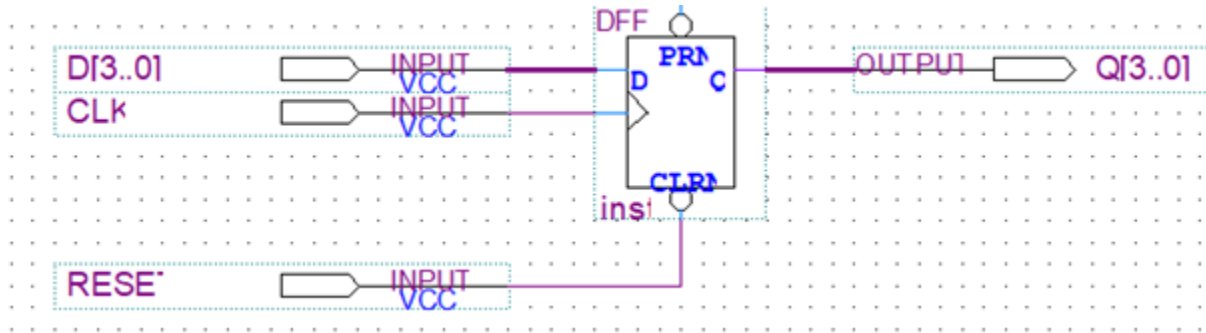
Hình 16.9: Bảng trạng thái kế tiếp và output.

16.3.3. Khối NextState



Hình 16.10: Khối NextState.

16.3.4. Khối CurrentState

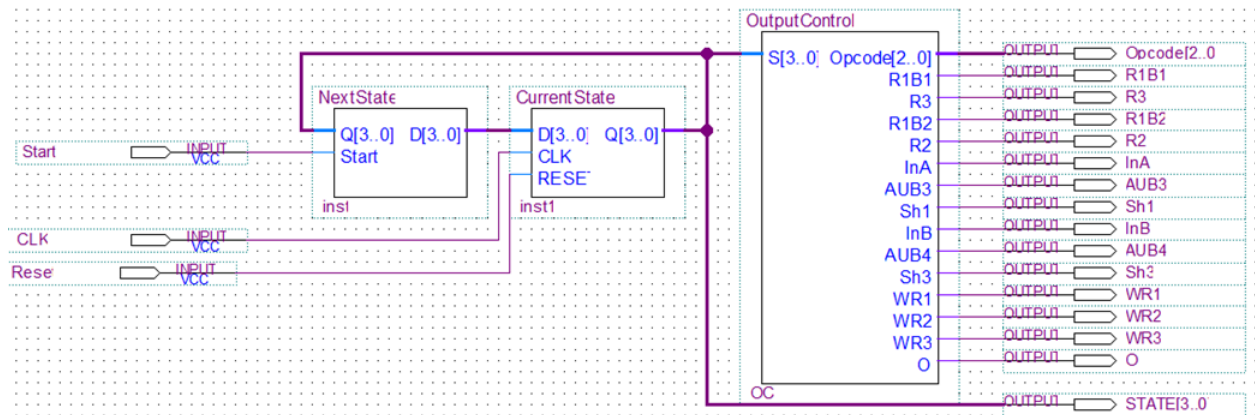


Hình 16.11: Khối Current State.

OutputControl:

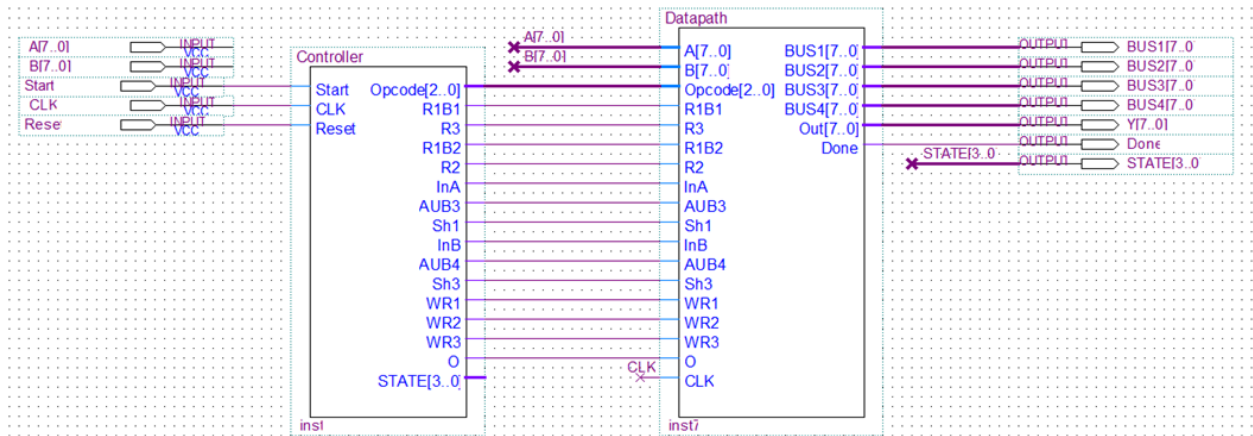
CONTROLLER TABLE																		
STATE	Opcode			BUS1		BUS2		BUS3			BUS4			WR			Out	
	Op2	Op1	Op0	R1B1	R3	R1B2	R2	InA	AUB3	Sh1	InB	AUB4	Sh3	WR1	WR2	WR3	O	
0	X	X	X					1			1			1	1			
1	1	0	0			1												
2	1	0	0				1		1					1				
3	X	X	X									1			1			
4	1	1	0	1			1											
5	1	1	1	1			1		1				1	1	1			
6	1	0	1	1			1			1							1	
7	X	X	X									1			1			
8	0	0	1		1		1									1		
9	X	X	X									1			1			
10	1	1	0	1			1											
11	X	X	X						1					1				
12	X	X	X	1														1

16.3.5. Hoàn thiện Controller:



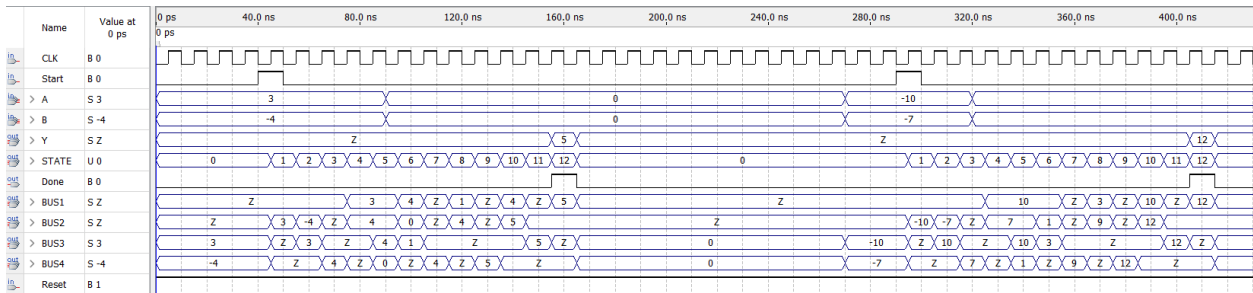
Hình 16.12: Khối Control.

16.4. Thiết kế Hoàn thiện



Hình 16.13: Mạch tổng.

16.5. Waveform



Hình 16.14: Waveform Datapath with pipelined functional

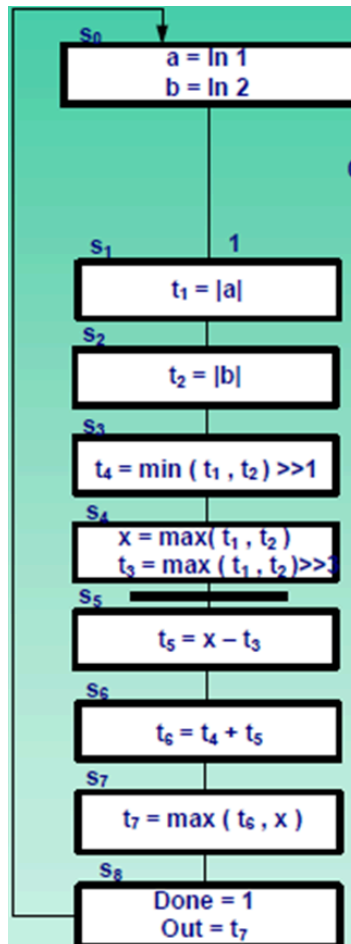
Mô tả: Nạp giá trị input a và b ở State 0, khi có cạnh lên xung clock thì Start = 1, thì mạch sẽ hoạt động và chuyển sang Stage 1, sau khi chạy xong các Stage (0 -> 12) thì tín hiệu Done sẽ bật và cho Output Y.

- Bài toán 1: $a = 3, b = -4 \Rightarrow 3^2 + (-4)^2 = 9 + 16 = 25 \Rightarrow \sqrt{25} = 5$
- Bài toán 2: $a = -10, b = -7 \Rightarrow (-10)^2 + (-7)^2 = 100 + 49 = 149 \Rightarrow \sqrt{149} \approx 12.206$

Bài 17. Datapath pipelining

17.1. Tổng quan thiết kế:

Trong bài tập trên, chúng ta đã cải thiện tốc độ khối Datapath bằng cách pipeline các khối chức năng. Khi Datapath xử lý cùng một chức năng cho nhiều tín hiệu đầu vào, tốc độ có thể tăng hơn nữa bằng cách pipeline toàn bộ biểu đồ ASM. Biểu đồ được chia thành các phần bằng nhau, mỗi phần xử lý qua một tầng pipeline riêng, như hình minh họa dưới đây.



Hình 17.1. Biểu đồ ASM cho thiết kế Datapath pipelining

Tại đây, quá trình tính toán được chia thành hai tầng. Trong khi tầng đầu tiên xử lý Input thứ n , tầng thứ hai đồng thời tính toán Input $n+1$, giúp rút ngắn đáng kể thời gian xử lý.



Hình 17.2 Giản đồ thời gian dựa trên biểu đồ ASM

Ta xây dựng mạch cho thiết kế với các đặc điểm:

Chức năng: tính căn bậc hai tổng hai bình phương

Ngõ vào:

- + In1[7..0]: input thứ 1 với độ rộng 8 bit.
- + In2[7..0]: input thứ 2 với độ rộng 8 bit.
- + CLK: tạo xung clock cho mạch.
- + Start: làm tín hiệu bắt đầu.
- + CLR: làm tín hiệu clear dữ liệu cho mạch.

Ngõ ra:

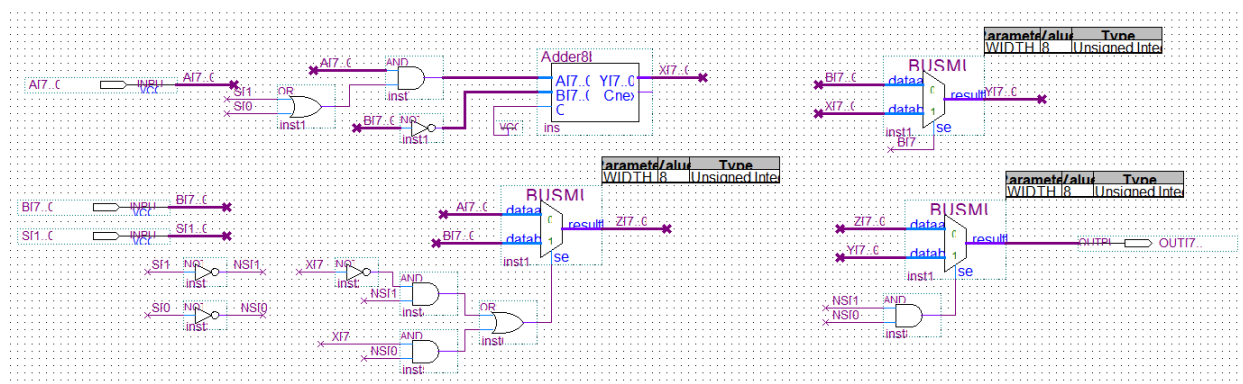
- + RESULTS[7..0]: kết quả tính toán với độ rộng 8 bit.
- + DONE: Thông báo kết thúc tính toán.

17.2. Thiết kế Datapath:

17.2.1. Khối AU1 để tính Abs/Min/Max:

S1S0	Chức năng
00	Abs
01	Min
10	Max

Bảng a.b. Opcode của khối AU1

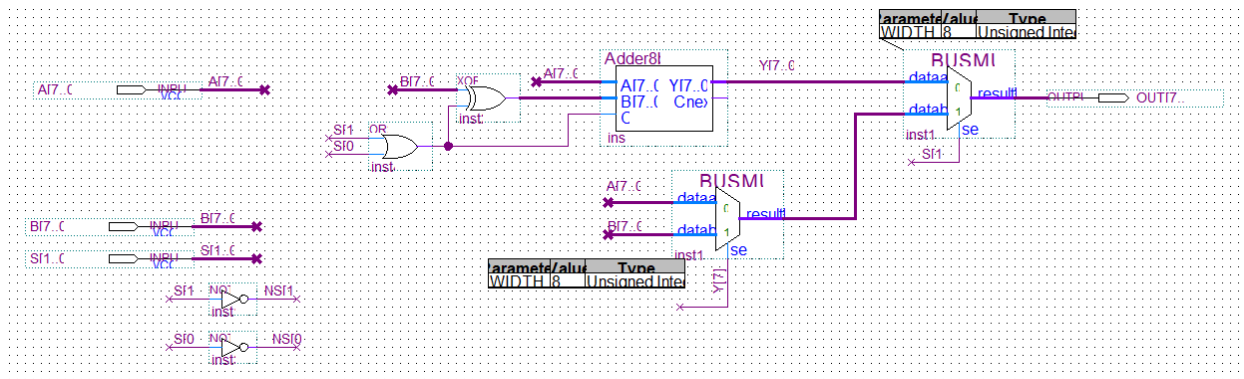


Hình 17.3: Khối AU1 (Abs/Min/Max).

17.2.2. Khối AU2 để tính Abs/Sub/Max:

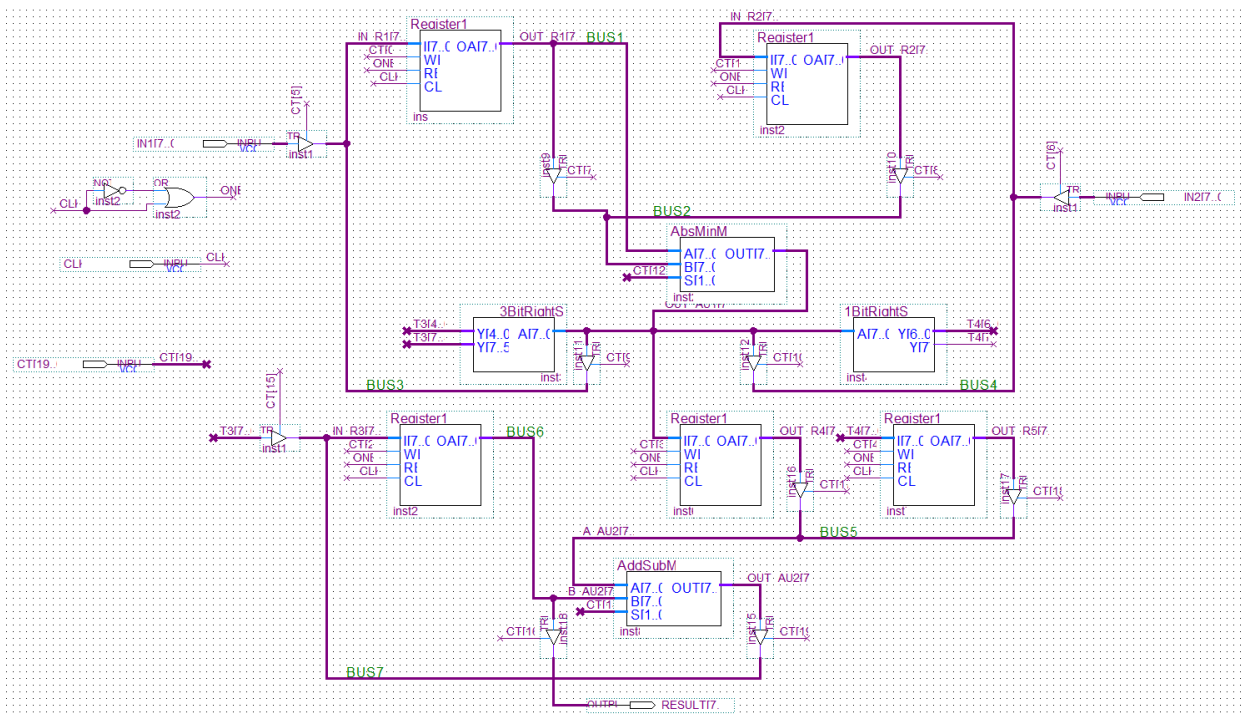
S1S0	Chức năng
00	Abs
01	Sub
10	Max

Bảng a.b. Opcode của khối AU2



Hình 17.4. Thiết kế khối AU2 để tính Abs/Sub/Max

17.2.3. Hoàn thiện Datapath:



Hình 17.5. Thiết kế Datapath hoàn chỉnh cho thiết kế Datapath pipelining

Datapath được chia làm hai tầng:

- Tầng thứ nhất: thực hiện phép tính từ S0 đến S4 với hai thanh ghi lưu giá trị là R1 (với các biến a, t1) và R2 (với các biến b, t2).
- Tầng thứ hai: thực hiện các tính toán từ S5 tới S9 với ba thanh ghi lưu giá trị là R3 (với các biến t3, t5, t6, t7), R4 (với biến x) và R5 (với biến t4).

Hoạt động của các tín hiệu:

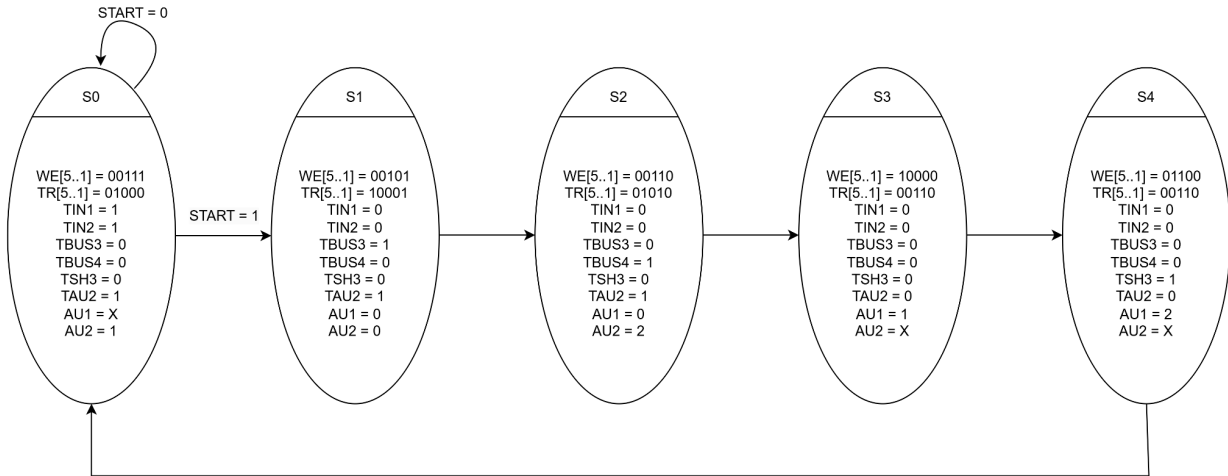
- Tín hiệu đầu vào **CT[4..0]** vào các chân WE1, WE2, WE3, WE4, WE5 ứng với Register 1, Register 2, Register 3, Register 4, Register 5 để cho phép ghi dữ liệu vào các Registers.
- Tín hiệu mức cao vào các chân RE1, RE2, RE3, RE4, RE5 để cho phép đọc dữ liệu từ các register.
- Tín hiệu đầu vào **CT[5]** và **CT[9]** lần lượt dùng để chọn dữ liệu từ đầu vào In1[7..0] hay dữ liệu từ khối AU1 để đưa vào BUS3 để ghi vào Register 1.
- Tín hiệu đầu vào **CT[6]** và **CT[10]** lần lượt dùng để chọn dữ liệu từ đầu vào In2[7..0] hay dữ liệu từ khối AU1 để đưa vào BUS4 để ghi vào Register 2.
- Tín hiệu đầu vào **CT[7]** và **CT[8]** lần lượt dùng để chọn dữ liệu từ Register 1 hay Register 2 để đưa vào BUS2.
- Tín hiệu đầu vào **CT[15]** và **CT[19]** lần lượt dùng để chọn dữ liệu từ đầu vào từ khối Shift-right-3 hay dữ liệu từ khối AU2 để đưa vào BUS7 để ghi vào Register 3.
- Tín hiệu đầu vào **CT[16]** dùng để cho phép xuất kết quả OUTPUT cuối cùng.
- Tín hiệu đầu vào CT[12..11] và CT[14..13] lần lượt dùng để cung cấp 2 bit tín hiệu điều khiển để khối AU1 và AU2 thực hiện đúng chức năng.

17.3. Thiết kế Controller:

17.3.1. Bảng trạng thái và các tín hiệu điều khiển:

Present State	WE[5..1]	TR[5..1]	TIN1	TIN2	TBUS3	TBUS4	TSH3	TAU2	AU1	AU2
S0	00111	01000	1	1	0	0	0	1	X	1
S1	00101	10001	0	0	1	0	0	1	0	0
S2	00110	01010	0	0	0	1	0	1	0	2

S3	10000	00110	0	0	0	0	0	0	1	X
S4	01100	00110	0	0	0	0	1	0	2	X



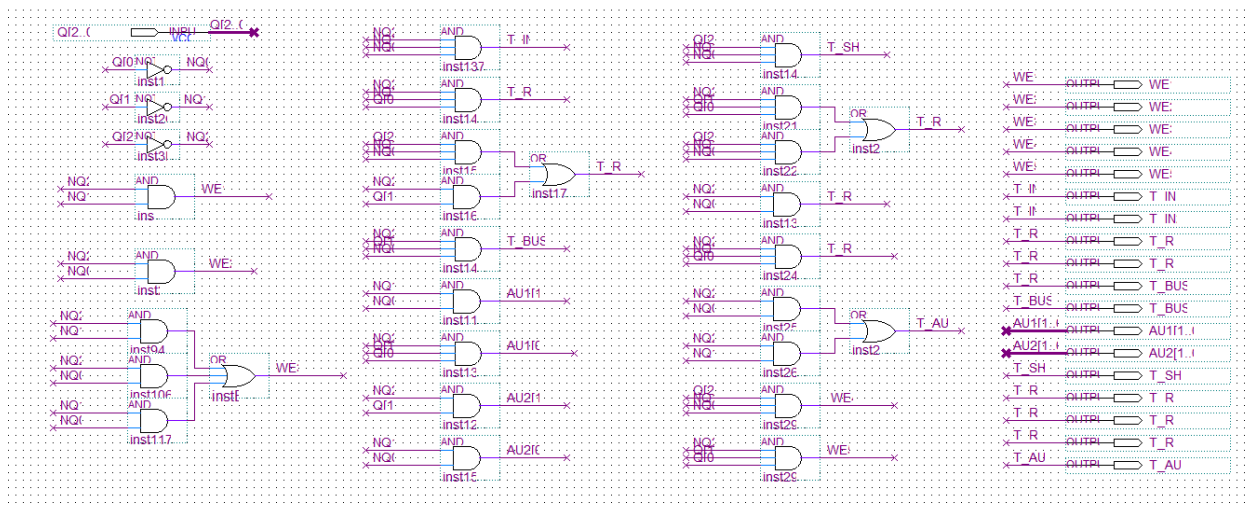
Hình 17.6. Sơ đồ chuyển trạng thái cho thiết kế Datapath pipelining

- Trạng thái S0: Khi tín hiệu Start = 1 mới cho phép chuyển trạng thái sang tiếp theo
- + Tầng 1: bật tín hiệu TIN1 và TIN2 cùng với WE1 và WE2 để cho phép ghi hai giá trị đầu vào vào R1 và R2.
- + Tầng 2: bật tín hiệu TR4 và tín hiệu điều khiển AU2 = 1 để tính $x - t3$, bật tín hiệu TAU2 và WE3 để lưu kết quả vào R3.
- Trạng thái S1: Khi tín hiệu Start = 1 mới cho phép chuyển trạng thái sang tiếp theo
- + Tầng 1: Bật tín hiệu TR1 và tín hiệu điều khiển AU1 = 0 để tính $|a|$, bật TBUS3 cùng với WE1 để lưu kết quả vào R1.
- + Tầng 2: bật tín hiệu TR5 và tín hiệu điều khiển AU2 = 0 để tính $t4 + t5$, bật tín hiệu TAU2 và WE3 để lưu kết quả vào R3.
- Trạng thái S2: Khi tín hiệu Start = 1 mới cho phép chuyển trạng thái sang tiếp theo.
- + Tầng 1: Bật tín hiệu TR2 và tín hiệu điều khiển AU1 = 0 để tính $|b|$, bật TBUS4 cùng với WE2 để lưu kết quả vào R2.
- + Tầng 2: bật tín hiệu TR4 và tín hiệu điều khiển AU2 = 2 để tính $\max(x, t6)$, bật tín hiệu TAU2 và WE3 để lưu kết quả vào R3.
- Trạng thái S3: Khi tín hiệu Start = 1 mới cho phép chuyển trạng thái sang tiếp theo. Dựa

- + Tầng 1: Bật tín hiệu TR2 và tín hiệu điều khiển AU1 = 1 để tính $\min(t1, t2)$ và bật tín hiệu WE5 để lưu kết quả vào R5.
- + Tầng 2: Kiểm tra tín hiệu Q của khối kiểm soát Output nếu Q = 1 thì cho phép TRI được bật và xuất Output. Đồng thời tín hiệu DONE được bật
- Trạng thái S4: Khi tín hiệu Start = 1 mới cho phép chuyển trạng thái sang tiếp theo
- + Tầng 1: Bật tín hiệu TR2 và tín hiệu điều khiển AU1 = 2 để tính $\max(t1, t2)$ và bật các tín hiệu TSH3, WE3, WE4 để lưu các giá trị vào R3 và R4.
- + Tầng 2: Kiểm tra tín hiệu Q của khối kiểm soát Output nếu Q = 1 thì cho phép TRI được bật và xuất Output. Đồng thời tín hiệu DONE được bật.

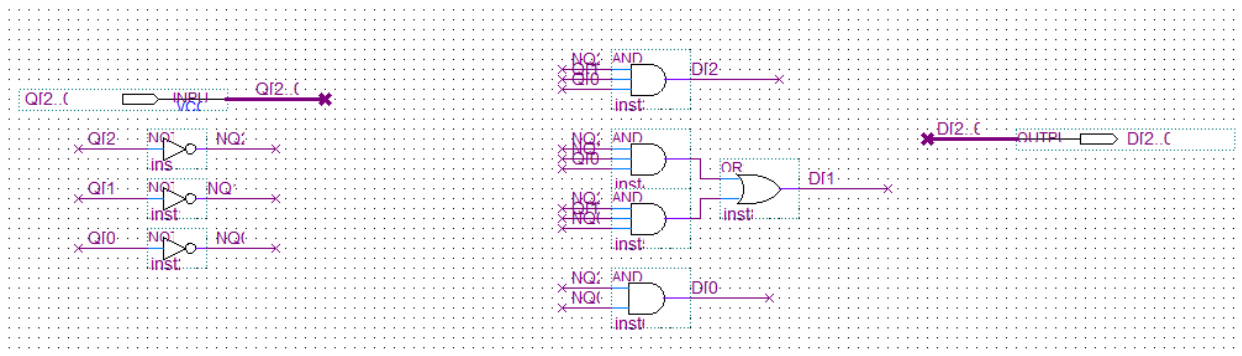
17.3.2. Hoàn thiện khối Controller:

Khối tín hiệu Output cho thiết kế Datapath Pipeline:



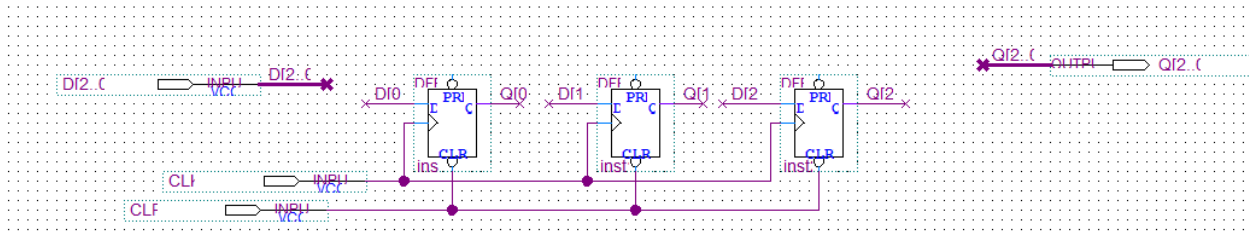
Hình 17.7. Khối tín hiệu Output cho thiết kế Datapath Pipeline

Mạch trạng thái kế tiếp:



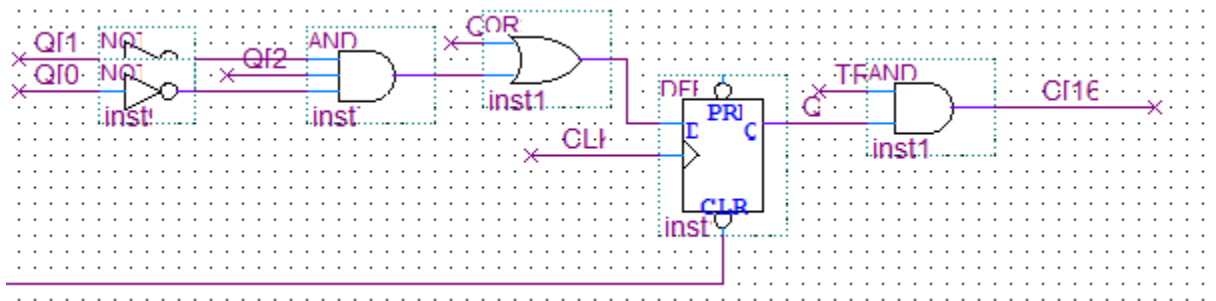
Hình 17.8. Mạch trạng thái kế tiếp

Mạch trạng thái hiện tại:



Hình 17.9. Mạch trạng thái hiện tại

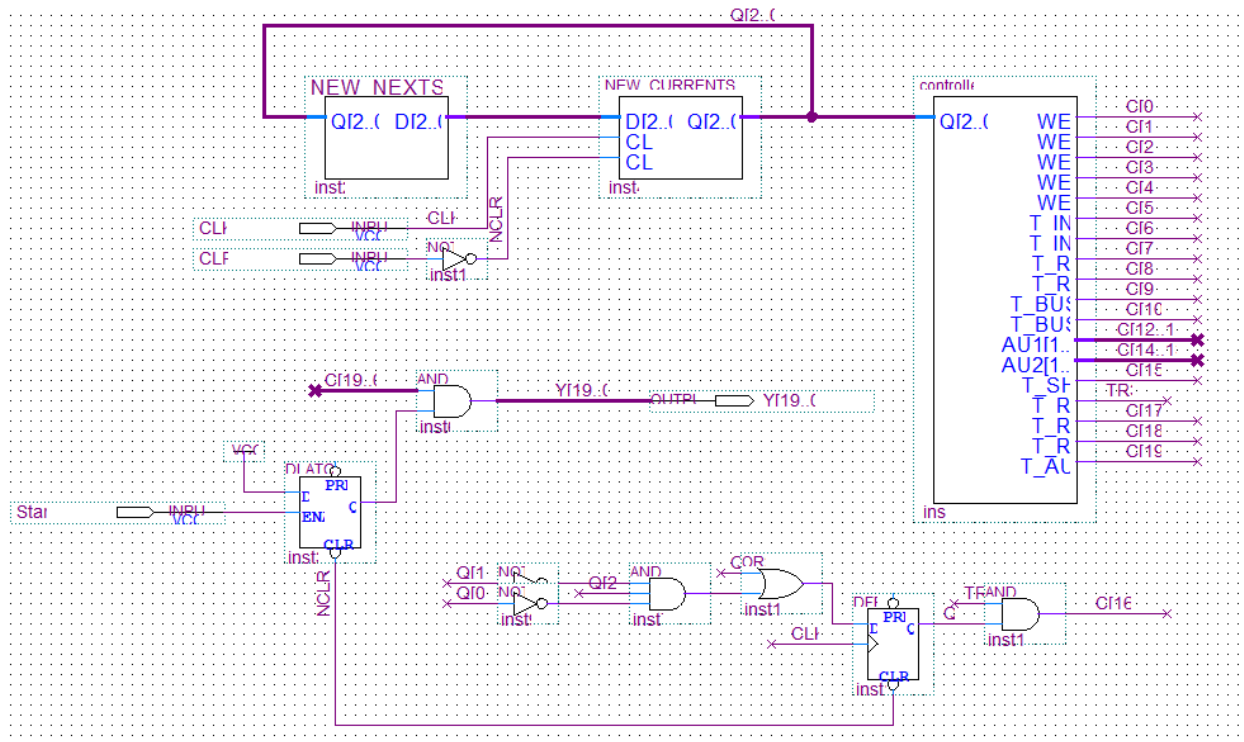
Mạch kiểm soát đầu ra:



Hình 17.10. Mạch kiểm soát đầu ra

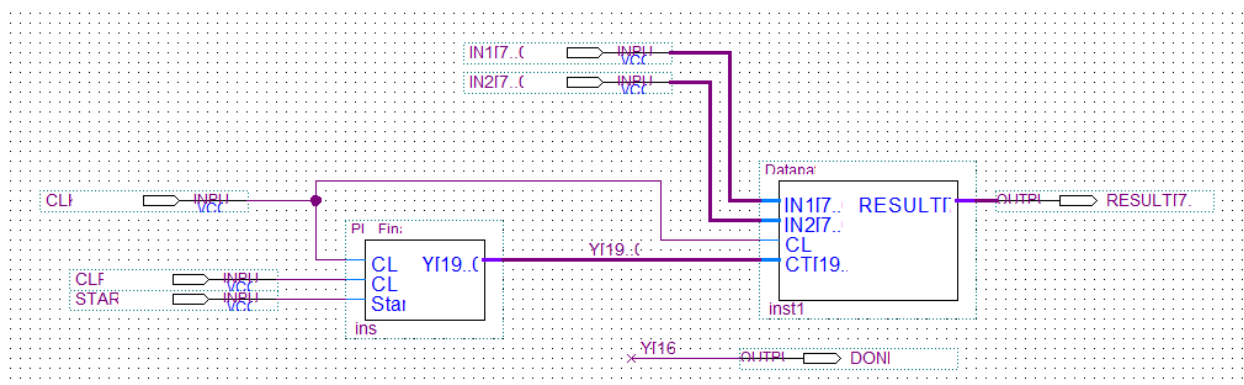
Ban đầu mạch được Clear để $Q = 0$, khi qua trạng thái S4 đầu tiên thì giá trị Q mới được lên 1 (tức bỏ qua lần xuất kết quả của 5 chu kì đầu tiên - lúc kết quả chưa được tính toán hoàn thành) và thực hiện AND với TR3 để điều khiển TRI kiểm soát đầu ra và tín hiệu DONE.

17.3.3. Hoàn thiện Controller:



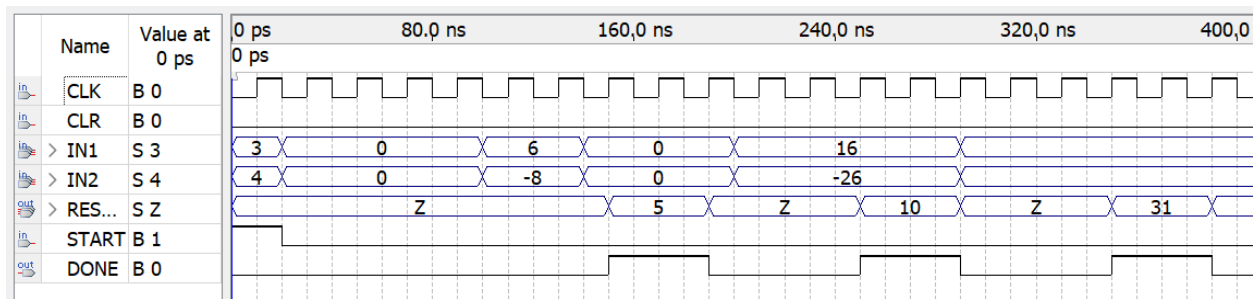
Hình 17.11. Controller hoàn thiện cho thiết kế Datapath pipelining

17.4. Thiết kế Hoàn thiện:



Hình 17.12. Thiết kế hoàn thiện của Datapath pipelining

17.5. Waveform:



Hình 17.13. Waveform của thiết kế Datapath pipelining

Khi tín hiệu Start chuyển lên mức 1, ở xung Clock tiếp theo, mạch chuyển sang trạng thái S1 và lưu hai giá trị Input đầu tiên. Sau 5 chu kỳ Clock kể từ lúc tín hiệu Start kích hoạt, mạch chuyển sang trạng thái S5 và lưu hai giá trị Input thứ hai. Đến chu kỳ thứ 8, mạch xuất Output cho cặp Input đầu tiên là 5. Tiếp theo, tại chu kỳ thứ 13, mạch xuất Output cho cặp Input thứ hai là 10. Tương tự, với cặp Input thứ ba, các giá trị Input được nhận tại chu kỳ thứ 10 và Output là 31 được xuất tại chu kỳ thứ 18. Như vậy, để xử lý 3 tập Input, mạch cần tổng cộng 19 chu kỳ (bao gồm cả chu kỳ 0). Kết quả Waveform trên phù hợp với yêu cầu bài toán, với số chu kỳ cần thiết để xử lý n tập Input được tính theo công thức $5n+4$.

Bài 18. Datapath pipeline with pipelined functional units

18.1. Tổng quan thiết kế:

Pipeline hóa là một kỹ thuật được sử dụng rộng rãi trong thiết kế hệ thống vi xử lý và các mạch kỹ thuật số, nhằm tăng hiệu suất bằng cách chia nhỏ quá trình xử lý thành các giai đoạn (stages) và thực hiện song song nhiều tác vụ. Trong trường hợp các đơn vị chức năng được pipeline hóa, mỗi đơn vị xử lý một phần dữ liệu trong từng chu kỳ đồng hồ, cải thiện throughput của toàn hệ thống.

- Datapath pipeline:

Trong thiết kế **Datapath Pipeline**, việc tách khối **Arithmetic Unit (AU)** thành hai đơn vị xử lý riêng biệt **AU1** và **AU2** giúp cải thiện khả năng xử lý song song, tối ưu hóa hiệu suất và giảm thời gian trễ.

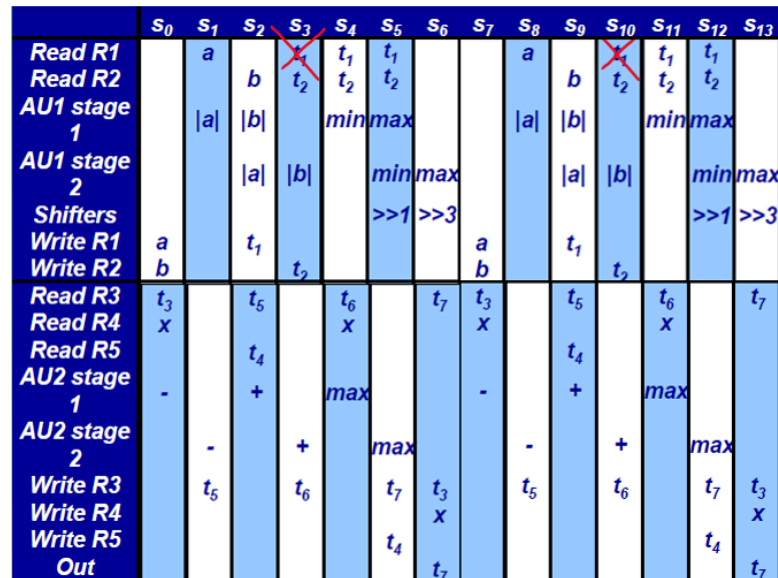
Khi **AU1** hoàn thành một phần phép toán và truyền kết quả xuống **AU2** để tiếp tục xử lý, trong khi đó, **AU1** có thể nhận dữ liệu mới và bắt đầu thực hiện một phép toán khác. Điều này giúp **pipeline hóa** quá trình xử lý, tối đa hóa hiệu suất bằng cách đảm bảo rằng các đơn vị chức năng không bao giờ bị "nhàn rỗi".

- Pipelined functional units:

Khi **AU** được chia thành hai trạng thái (state1 và state2), ta có thể sử dụng **D flip-flop** để phân tách khối **AU**, tạo ra cơ chế **lưu trữ và chặn dữ liệu** giữa các trạng thái. Cụ thể, khi **state1** hoàn thành công việc và truyền kết quả cho **state2**, **D flip-flop** sẽ giúp **chuyển dữ liệu** từ **state1** sang **state2** một cách chính xác và đồng bộ, đảm bảo không có mất mát thông tin.

Lúc này, khi **state1** đã hoàn tất và kết quả được truyền cho **state2**, **D flip-flop** cũng đồng thời cho phép **state1** nhận dữ liệu mới và bắt đầu thực hiện một phép toán khác mà không bị gián đoạn. Điều này tạo ra một dòng chảy dữ liệu liên tục, giúp tối ưu hóa hiệu suất và tăng khả năng xử lý song song trong hệ thống.

- Timing diagram for datapath pipeline with pipelined functional units:



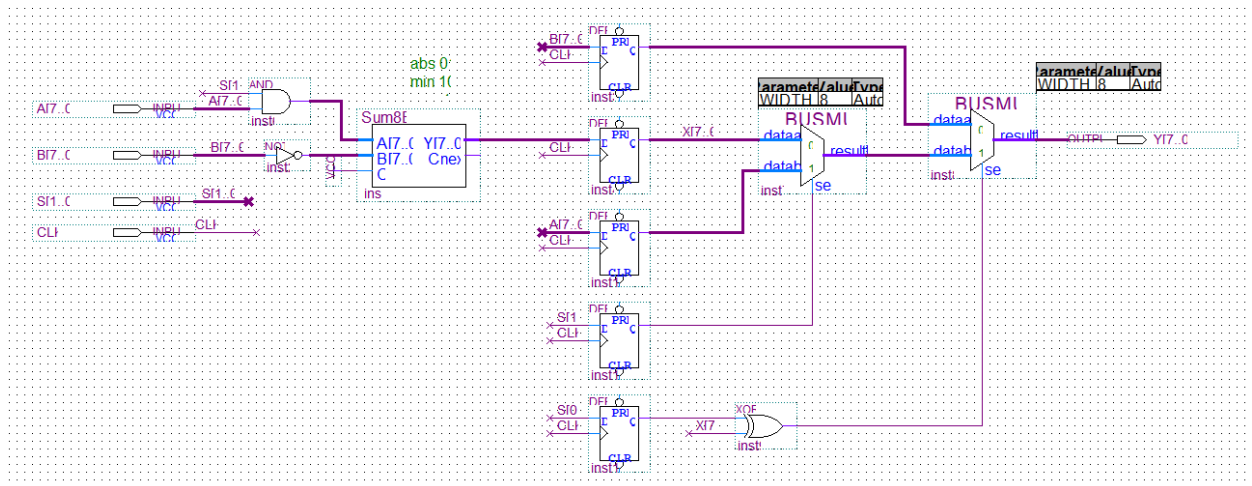
Hình 18.1. Timing diagram for datapath pipeline with pipelined functional.

18.2. Thiết kế Datapath:

18.2.1. Khối AU1:

Tín hiệu điều khiển:

S[1:0]	Functions
01	Absolute
10	Min
11	Max

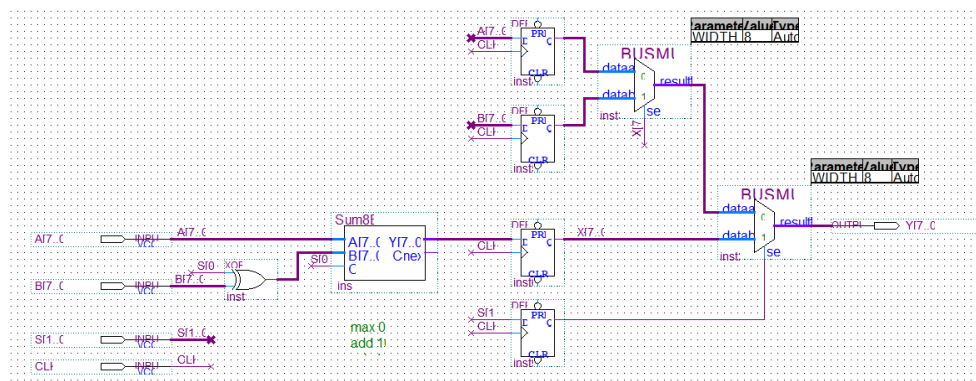


Hình 18.2. Khối AU1 (Abs/Min/Max)

18.2.2. Khối AU2:

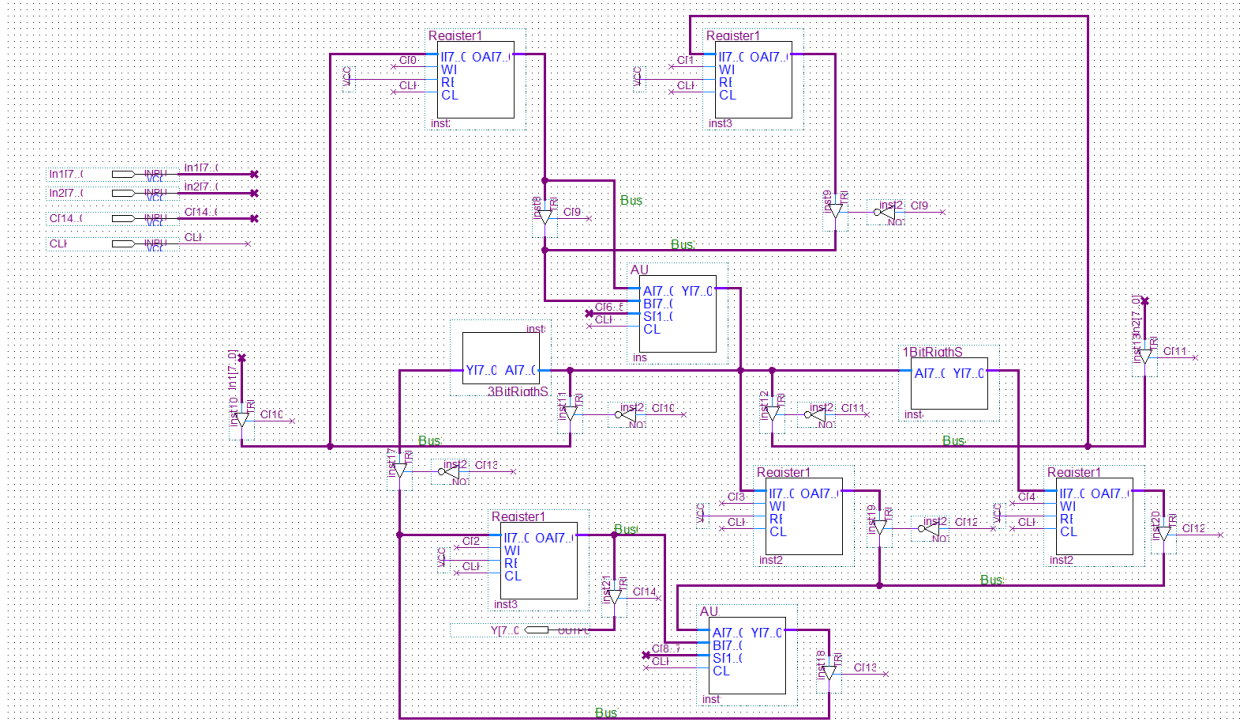
Tín hiệu điều khiển:

S[1:0]	Functions
01	Max
10	Add
11	Sub



Hình 18.3. Khối AU2 (Max/Add/Sub).

18.2.3. Hoàn thiện Datapath:



Hình 18.3. Datapath.

- **C[0]:** Điều khiển tín hiệu **WE** của **register 1**, xác định liệu dữ liệu có được ghi vào thanh ghi này hay không.
- **C[1]:** Điều khiển tín hiệu **WE** của **register 2**, xác định liệu dữ liệu có được ghi vào thanh ghi này hay không.
- **C[2]:** Điều khiển tín hiệu **WE** của **register 3**, xác định liệu dữ liệu có được ghi vào thanh ghi này hay không.
- **C[3]:** Điều khiển tín hiệu **WE** của **register 4**, xác định liệu dữ liệu có được ghi vào thanh ghi này hay không.
- **C[4]:** Điều khiển tín hiệu **WE** của **register 5**, xác định liệu dữ liệu có được ghi vào thanh ghi này hay không.
- **C[6:5]:** Điều khiển chức năng của **khối AU1**, giúp chọn chức năng phù hợp để thực hiện phép toán.
- **C[8:7]:** Điều khiển chức năng của **khối AU2**, giúp chọn chức năng phù hợp để thực hiện phép toán.

- **C[9]:** Điều khiển hai cổng **Tri-State** của **Bus2**, giúp điều chỉnh dòng dữ liệu trong Bus2 (hai cổng Tri-State hoạt động đối kháng với nhau).
- **C[10]:** Điều khiển hai cổng **Tri-State** của **Bus3**, giúp điều chỉnh dòng dữ liệu trong Bus2 (hai cổng Tri-State hoạt động đối kháng với nhau).
- **C[11]:** Điều khiển hai cổng **Tri-State** của **Bus4**, giúp điều chỉnh dòng dữ liệu trong Bus2 (hai cổng Tri-State hoạt động đối kháng với nhau).
- **C[12]:** Điều khiển hai cổng **Tri-State** của **Bus5**, giúp điều chỉnh dòng dữ liệu trong Bus2 (hai cổng Tri-State hoạt động đối kháng với nhau).
- **C[13]:** Điều khiển hai cổng **Tri-State** của **Bus7**, giúp điều chỉnh dòng dữ liệu trong Bus2 (hai cổng Tri-State hoạt động đối kháng với nhau).
- **C[14]:** Điều khiển cổng **Tri-State** để quyết định liệu kết quả có thể xuất ra ngoài hay không.

Các giá trị được lưu trong thanh ghi:

- $R1 = [a, t1]$
- $R2 = [b, t2]$
- $R3 = [t3, t5, t6, t7]$
- $R4 = [t4]$
- $R5 = [x]$

Các chức năng của các khối AU:

- $AU1 = [\text{abs/min/max}]$
- $AU2 = [+/-/max]$

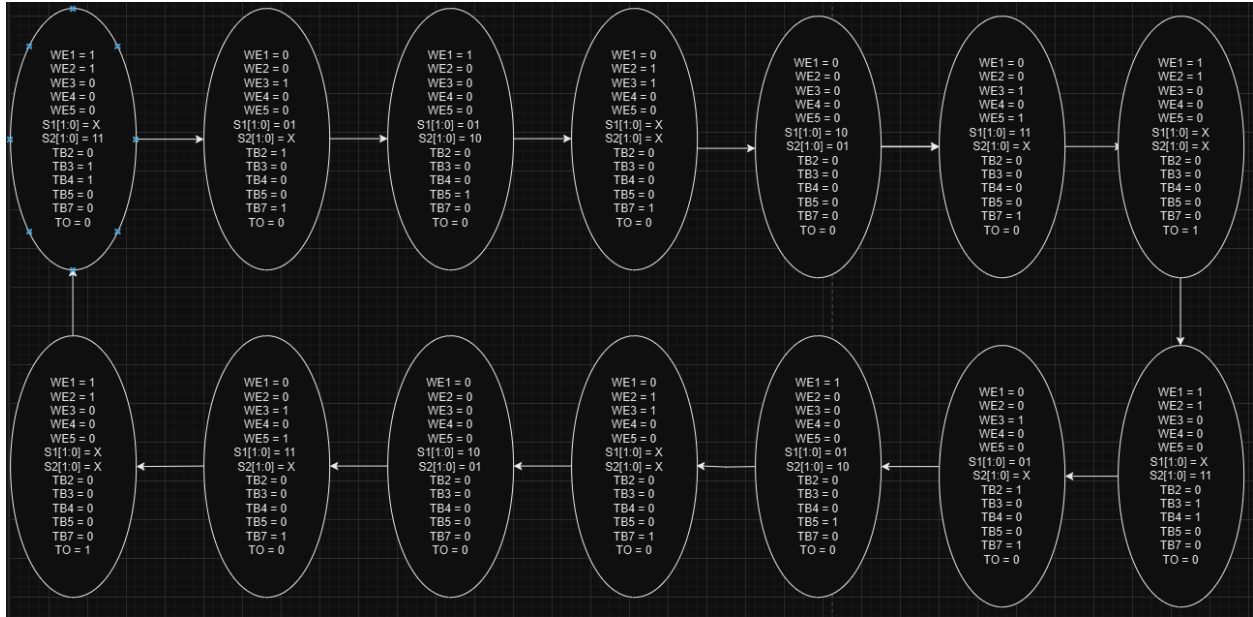
18.3. Thiết kế Controller:

18.3.1. Tín hiệu control:

State	WE1	WE2	WE3	WE4	WE5	S1[1:0]	S2[1:0]	TB2	TB3	TB4	TB5	TB7	TO
S0	1	1	0	0	0	X	11	0	1	1	0	0	0
S1	0	0	1	0	0	01	X	1	0	0	0	1	0
S2	1	0	0	0	0	01	10	0	0	0	1	0	0
S3	0	1	1	0	0	X	X	0	0	0	0	1	0
S4	0	0	0	0	0	10	01	0	0	0	0	0	0
S5	0	0	1	0	1	11	X	0	0	0	0	1	0
S6	0	0	1	1	0	X	X	0	0	0	0	0	1
S7	1	1	0	0	0	X	11	0	1	1	0	0	0
S8	0	0	1	0	0	01	X	1	0	0	0	1	0
S9	1	0	0	0	0	01	10	0	0	0	1	0	0
S10	0	1	1	0	0	X	X	0	0	0	0	1	0
S11	0	0	0	0	0	10	01	0	0	0	0	0	0
S12	0	0	1	0	1	11	X	0	0	0	0	1	0
S13	0	0	1	1	0	X	X	0	0	0	0	0	1

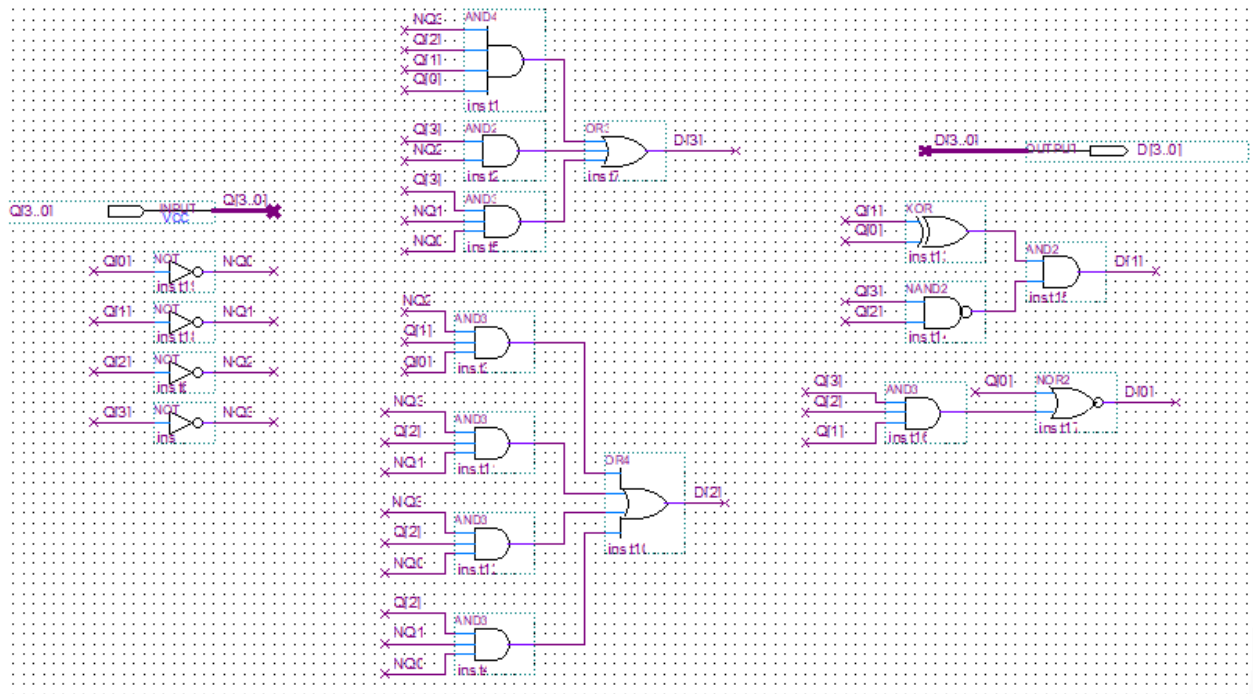
Hình 18.4. Tín hiệu control ứng với từng trạng thái.

Sơ đồ chuyển trạng thái:



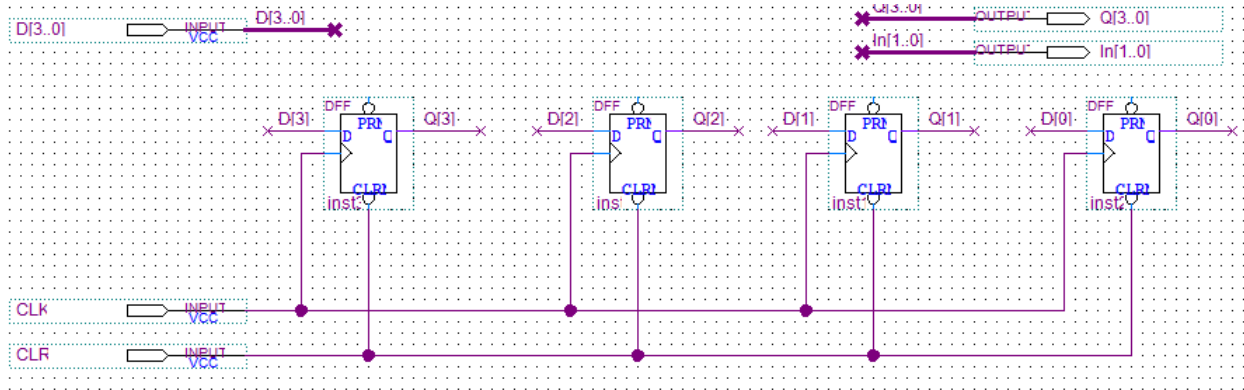
Hình 18.5: Sơ đồ chuyển trạng thái.

18.3.2. Khối NextState:



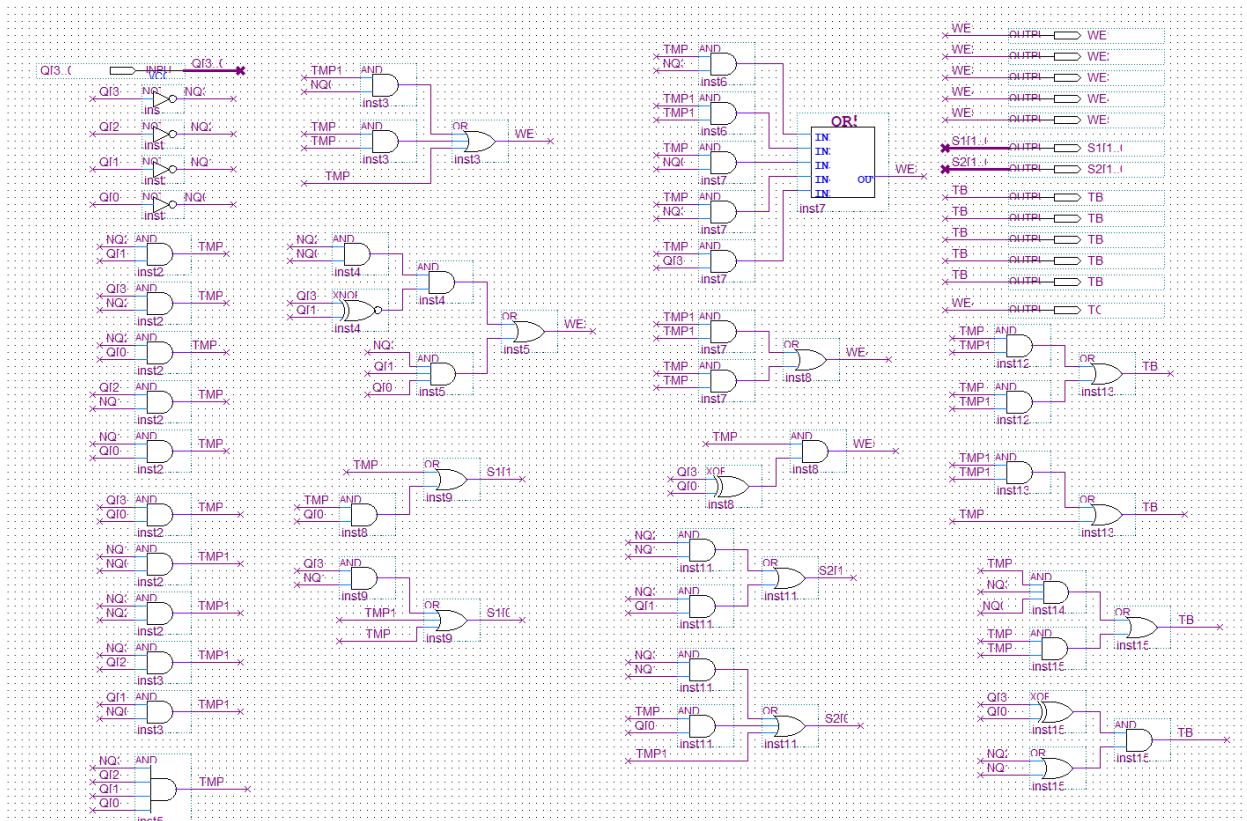
Hình 18.6. Khối Next State.

18.3.3. Khối Current State:



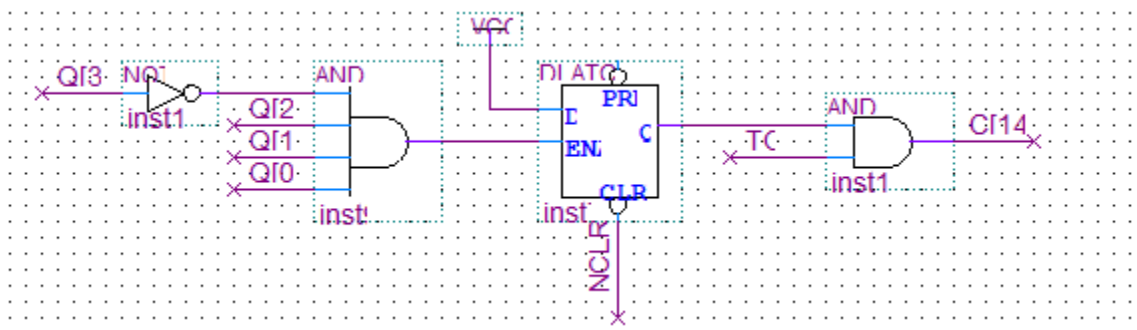
Hình 18.7. Khối Current State.

18.3.4. Khối Output:



Hình 18.8. Khối Output.

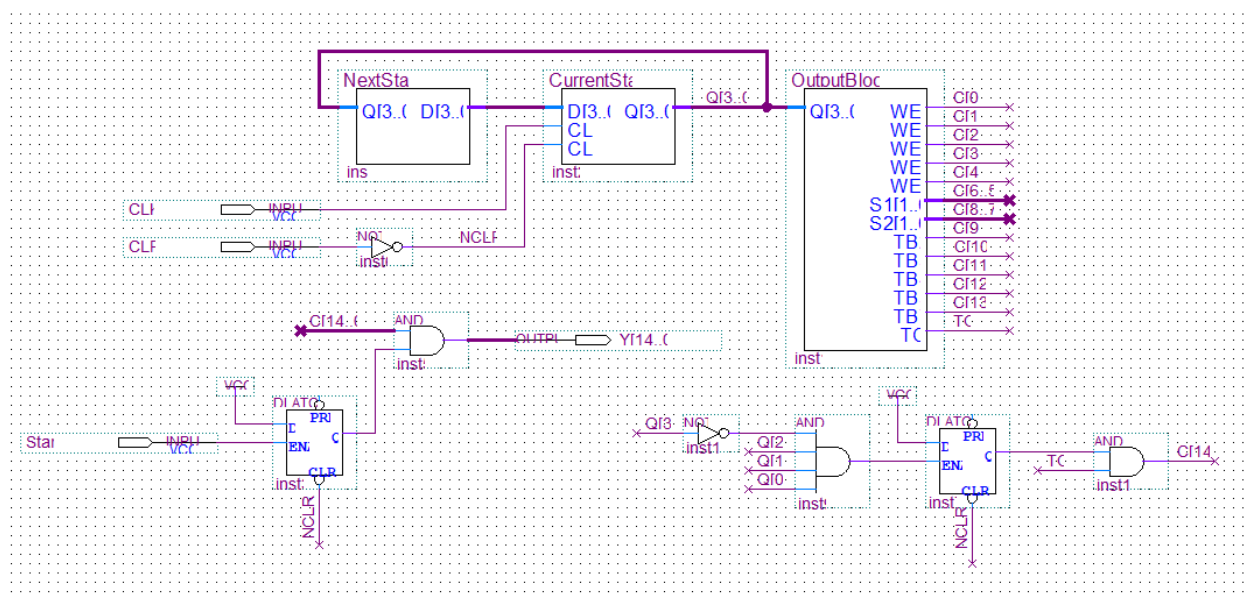
18.3.5. Khối tín hiệu hoàn thành:



Hình 18.9. Khối tín hiệu hoàn thành.

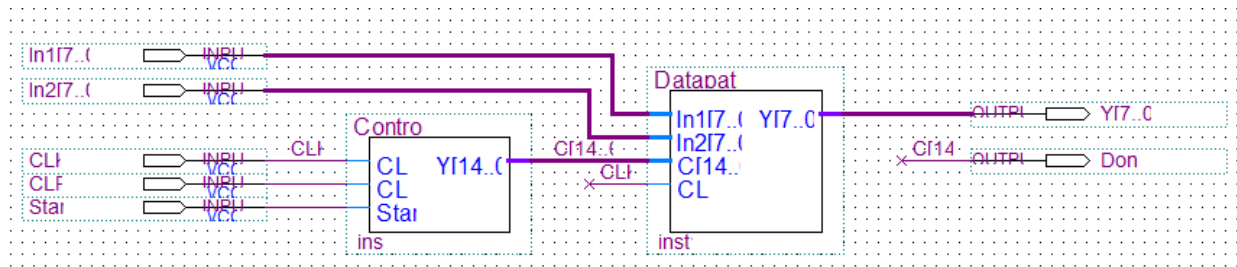
Khi ở **trạng thái 6 đầu tiên**, tín hiệu **C[14]**, điều khiển việc xuất dữ liệu ra output trong **Datapath**, sẽ không được kích hoạt. Điều này là do **input đầu tiên** yêu cầu một quá trình xử lý kéo dài **13 chu kỳ** trước khi có thể xuất kết quả ra ngoài. Vì vậy, trong giai đoạn này, tín hiệu **C[14]** sẽ bị **tắt** để đảm bảo rằng quá trình xử lý hoàn tất trước khi bất kỳ dữ liệu nào được xuất ra output.

18.3.6. Hoàn thiện Controller:



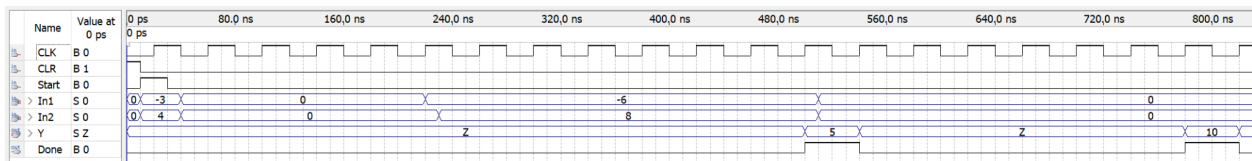
Hình 18.10. Khối Control.

18.4. Thiết kế Hoàn thiện:



Hình 18.11. Thiết kế tổng.

18.5. Waveform:



Hình 18.12. Waveform.

Giải thích:

- Bật CLR = 1 ở chu kỳ đầu để reset mạch.
- Tín hiệu **Start** = 1 được kích hoạt, cho phép nhập dữ liệu đầu vào với các giá trị **A = -3** và **B = -4**.
- Vì đây là đầu vào đầu tiên, mạch cần toàn bộ 13 chu kỳ để hoàn thành tính toán.
- Sau 13 chu kỳ, mạch xuất kết quả **Output = 5**, đồng thời kích hoạt tín hiệu **Done = 1** báo hiệu hoàn thành.
- Input thứ hai được nhập vào với **A = -6** và **B = 8** (với cơ chế pipeline).
- Nhờ **pipeline**, mạch chỉ cần 6 chu kỳ để xử lý đầu vào này, nhanh hơn đáng kể so với chu kỳ đầu tiên.
- Sau 6 chu kỳ, mạch cho ra **Output = 10**, và **Done = 1** được kích hoạt báo hiệu kết quả sẵn sàng.